

AI/BigData 인텐시브 과정

강사장철원

Section 0

코스소개

DAY1

DAY2

DAY3

DAY4

DAY5

데이터 분석을 위한 기초 수학

데이터 집계 및 시각화

DAY6

DAY7

DAY8

DAY9

DAY10

미니
프로젝트

확률분포, AB테스트

DAY11

DAY12

DAY13

DAY14

DAY15

머신러닝

DAY16

DAY17

DAY18

DAY19

DAY20

딥러닝

파이널 프로젝트

□ 행렬 기초

□ 행렬식

□ 역행렬

□ 내적

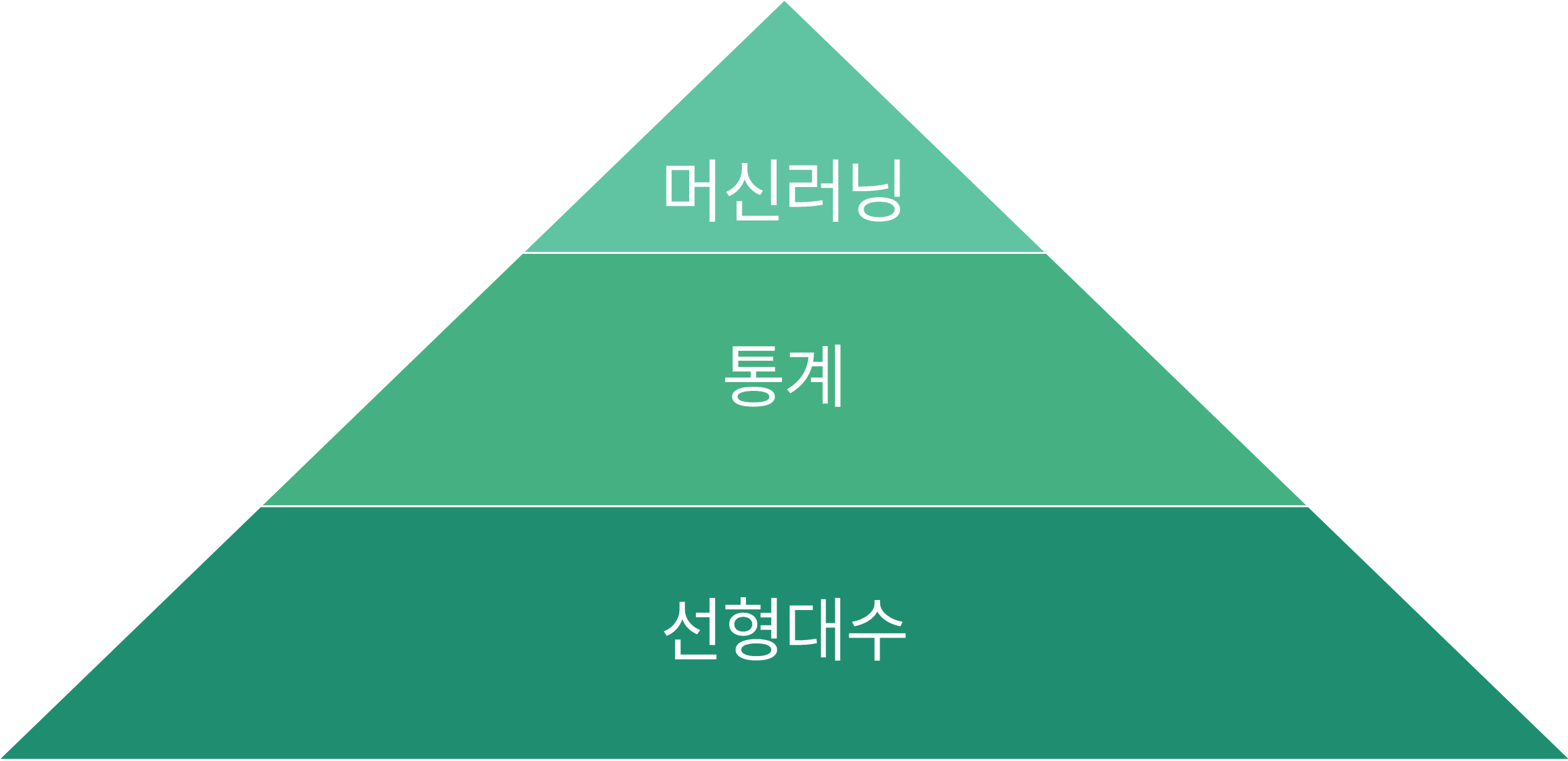
□ 고윳값, 고유벡터

□ 대각화

AI/BigData 인텐시브 과정

Section 1. 선형대수

Section 1-1. 행렬 기초



머신러닝

통계

선형대수

선형대수를 공부하는 이유

$$G = \tilde{Y}\tilde{Y}^\top = (Y - b\mathbf{1}^\top)(Y - b\mathbf{1}^\top)^\top = n(I - b\mathbf{1}^\top - \mathbf{1}b^\top + Kbb^\top).$$

If b is aligned with $\mathbf{1}$, i.e., they live in the same line, then $-b\mathbf{1}^\top - \mathbf{1}b^\top + Kbb^\top$ is a rank-1 matrix and G has $K - 1$ eigenvalues being n and the rest eigenvalue being $n(1 - \mathbf{1}^\top b)^2$. On the other hand, if b is not aligned with $\mathbf{1}$, then $-b\mathbf{1}^\top - \mathbf{1}b^\top + Kbb^\top$ is a rank-2 matrix and G has $K - 2$ eigenvalues being n . In this case, the rest of the two eigenvalues, denoted by π_1 and π_K , correspond to the eigenvectors within the subspace spanned by $\mathbf{1}$ and b .

To estimate the largest eigenvalues π_1 , we construct two orthonormal vectors within this subspace spanned by $\mathbf{1}$ and b and compute the corresponding Rayleigh quotient. Specifically, we first compute the Rayleigh quotient along the direction $\mathbf{1}$ as

$$\frac{\mathbf{1}^\top G \mathbf{1}}{\mathbf{1}^\top \mathbf{1}} = \frac{n}{K} \left(K - 2K\mathbf{1}^\top b + K(\mathbf{1}^\top b)^2 \right) = n \left(1 - 2\mathbf{1}^\top b + (\mathbf{1}^\top b)^2 \right) = n(1 - \mathbf{1}^\top b)^2.$$

Use Gram-Schmidt orthonormalization to obtain the other direction as $a = b - \frac{1}{K}\mathbf{1}^\top b\mathbf{1}$, which gives the following Rayleigh quotient:

$$\frac{a^\top G a}{a^\top a} = \frac{n}{\|a\|^2} \left(\|a\|^2 + K \left(\|b\|^2 - \frac{1}{K} (\mathbf{1}^\top b)^2 \right)^2 \right) = n + nK \left(\|b\|^2 - \frac{1}{K} (\mathbf{1}^\top b)^2 \right),$$

데이터 = 행렬

나이	성별	몸무게	시력
10	0	30	0.2
20	1	70	1.0
14	1	45	0.7
32	0	50	1.5
56	0	55	0.8

개발자가 왼쪽 데이터를 보는 관점



행(row) 위주

데이터 = 행렬

나이	성별	몸무게	시력
10	0	30	0.2
20	1	70	1.0
14	1	45	0.7
32	0	50	1.5
56	0	55	0.8

지금부터는 **행렬** 로 바라보기

- 데이터 전체
- 행(row)
- 열(column)
- 스칼라

스칼라

나이	성별	몸무게	시력
10	0	30	0.2
20	1	70	1.0
14	1	45	0.7
32	0	50	1.5
56	0	55	0.8

스칼라: 크기 만으로 나타낼수 있는 물리량

$$x = 1.5$$

스칼라 실습

```
In [1]: ▶ 1 import numpy as np  
        2 import pandas as pd
```

```
In [36]: ▶ 1 df = pd.read_csv("./data/house_prices.csv")
```

```
In [37]: ▶ 1 df
```

Out[37]:

	0 CRIM	1 ZN	2 INDUS	3 CHAS	4 NOX	5 RM	6 AGE	7 DIS
0	0.00632	18.0	2.31	0	0.538	6.575	65.2	4.0900
1	0.02731	0.0	7.07	0	0.469	6.421	78.9	4.9671
2	0.02729	0.0	7.07	0	0.469	7.185	61.1	4.9671
3	0.03237	0.0	2.18	0	0.458	6.998	45.8	6.0622
4	0.06905	0.0	2.18	0	0.458	7.147	54.2	6.0622

```
In [40]: ▶ 1 df.iloc[2,4]
```

Out[40]: 0.469

```
In [41]: ▶ 1 df.iloc[3,5]
```

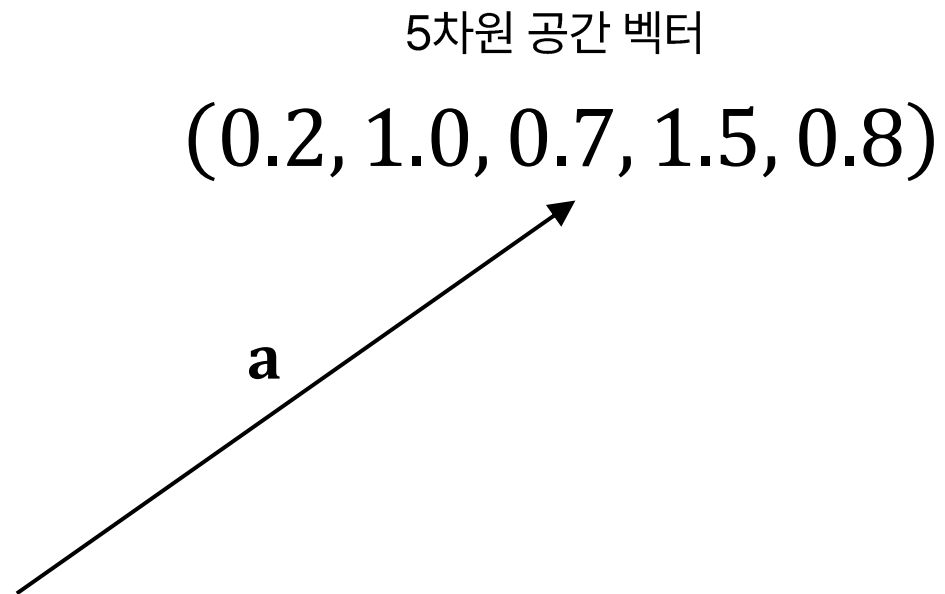
Out[41]: 6.998

```
In [42]: ▶ 1 df.iloc[2,3] + df.iloc[4,5]
```

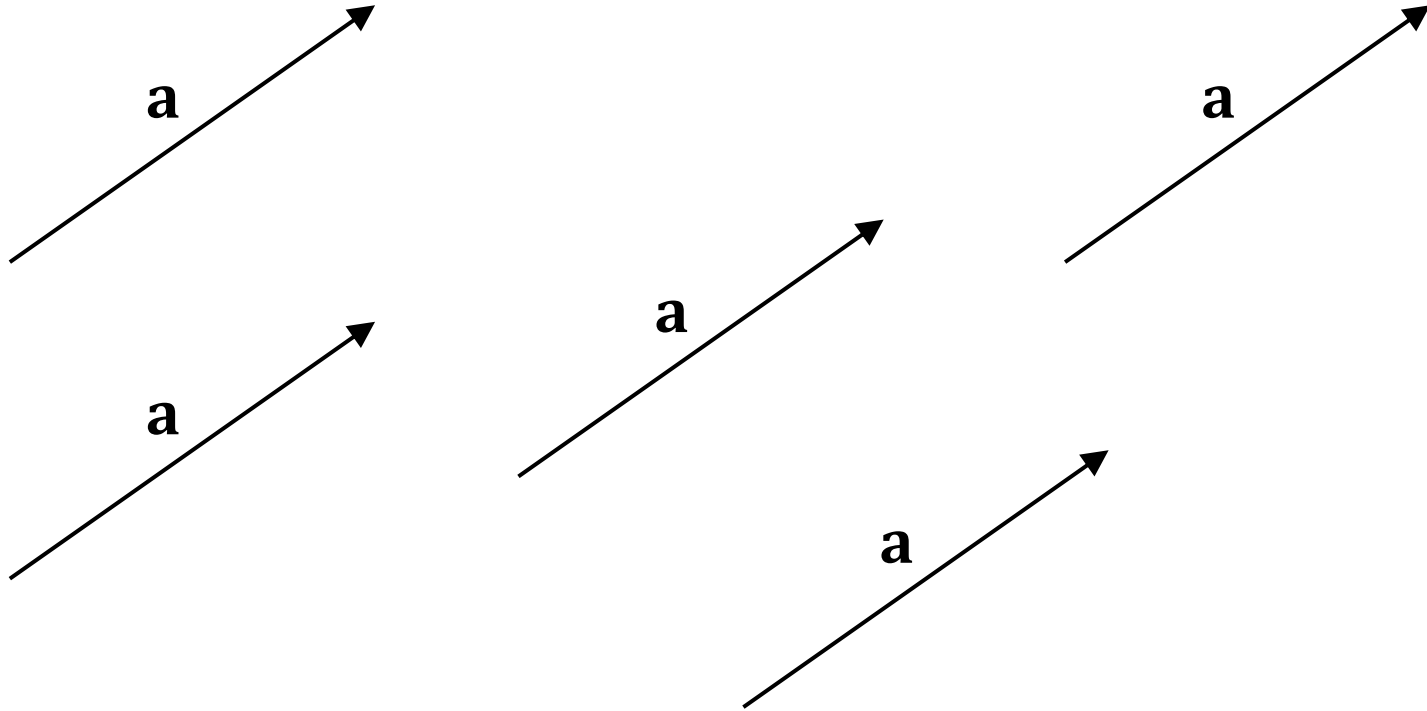
Out[42]: 7.147

벡터(vector)

나이	성별	몸무게	시력
10	0	30	0.2
20	1	70	1.0
14	1	45	0.7
32	0	50	1.5
56	0	55	0.8



벡터(vector)



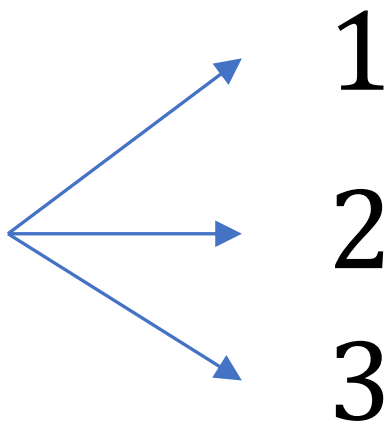
크기와 방향이 같다면 모두 동일한 벡터

스칼라 vs 벡터

스칼라

숫자 하나로 구성

파이썬
변수

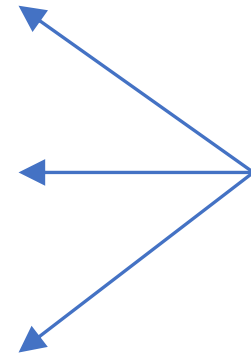


벡터

두 개 일수도,
세 개 일수도,
네 개 일수도...

[1, 2]
[1, 2, 4]
[1, 2, 4, 8]

파이썬
리스트



벡터(vector)

나이	성별	몸무게	시력
10	0	30	0.2
20	1	70	1.0
14	1	45	0.7
32	0	50	1.5
56	0	55	0.8

열 벡터

나이	성별	몸무게	시력
10	0	30	0.2
20	1	70	1.0
14	1	45	0.7
32	0	50	1.5
56	0	55	0.8

행 벡터

Section
행렬기초

$$\mathbf{a} = \begin{pmatrix} 1 \\ 2 \\ 3 \end{pmatrix}, \quad \mathbf{b} = \begin{pmatrix} 4 \\ 5 \\ 6 \end{pmatrix}$$

$$\mathbf{a} + \mathbf{b} = \begin{pmatrix} 1 \\ 2 \\ 3 \end{pmatrix} + \begin{pmatrix} 4 \\ 5 \\ 6 \end{pmatrix}$$

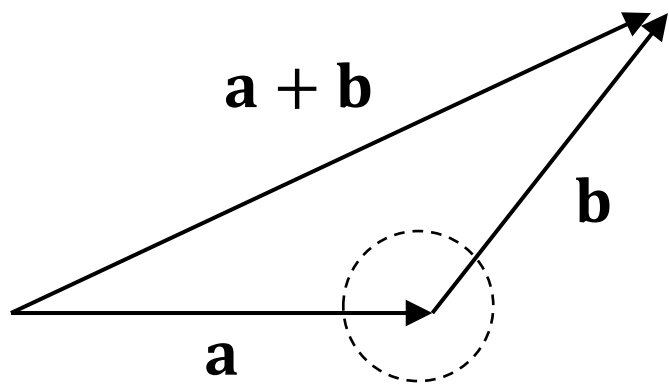
$$= \begin{pmatrix} 1 + 4 \\ 2 + 5 \\ 3 + 6 \end{pmatrix}$$

$$= \begin{pmatrix} 5 \\ 7 \\ 9 \end{pmatrix}$$

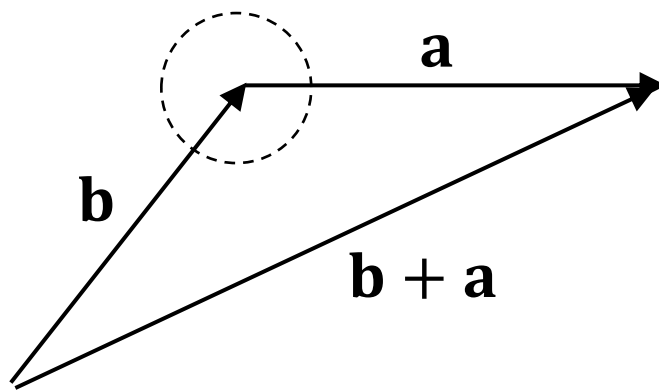
a	b
1	4
2	5
3	6

a		b		a+b
1		4		5
2		5		7
3		6		9

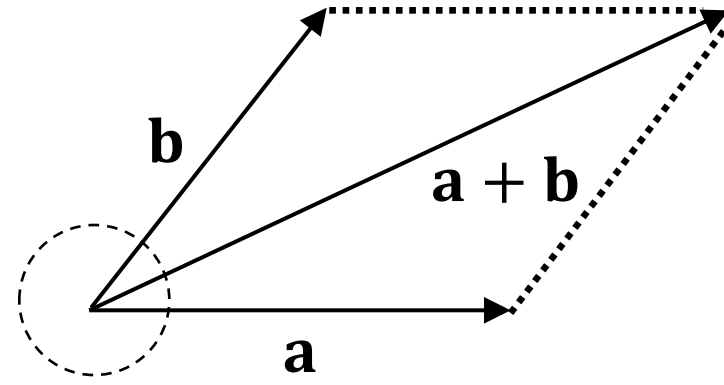
벡터의 덧셈



(a)



(b)



(c)

벡터의 뺄셈

$$\mathbf{a} = \begin{pmatrix} 7 \\ 3 \\ 9 \end{pmatrix}, \quad \mathbf{b} = \begin{pmatrix} 2 \\ 5 \\ 7 \end{pmatrix}$$

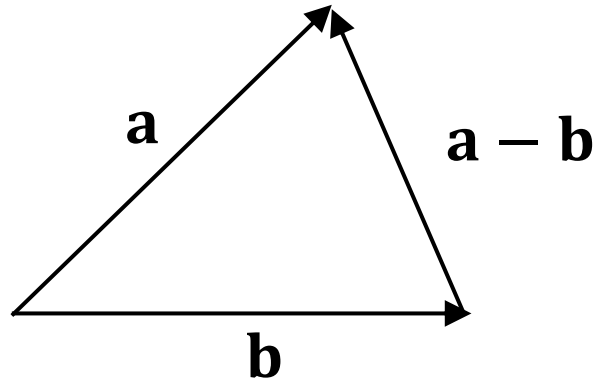
$$\mathbf{a} - \mathbf{b} = \begin{pmatrix} 7 \\ 3 \\ 9 \end{pmatrix} - \begin{pmatrix} 2 \\ 5 \\ 7 \end{pmatrix}$$

$$= \begin{pmatrix} 7 - 2 \\ 3 - 5 \\ 9 - 7 \end{pmatrix}$$

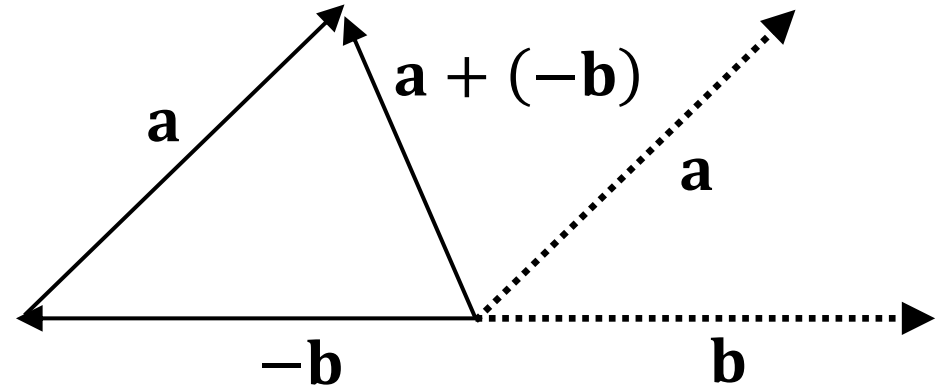
$$= \begin{pmatrix} 5 \\ -2 \\ 2 \end{pmatrix}$$

$$\mathbf{a} - \mathbf{b} = \mathbf{a} + (-\mathbf{b})$$

벡터의 뺄셈



$$b + (a - b) = a$$



$$a - b = a + (-b)$$

벡터 실습

1	df				
	CRIM	ZN	INDUS	CHAS	NOX
0	0.00632	18.0	2.31	0	0.538
1	0.02731	0.0	7.07	0	0.469
2	0.02729	0.0	7.07	0	0.469
3	0.03237	0.0	2.18	0	0.458
4	0.06905	0.0	2.18	0	0.458
...	...	...	...	...	...
501	0.06263	0.0	11.93	0	0.573
502	0.04527	0.0	11.93	0	0.573
503	0.06076	0.0	11.93	0	0.573
504	0.10959	0.0	11.93	0	0.573
505	0.04741	0.0	11.93	0	0.573

506 rows × 14 columns

1	df['CRIM']
0	0.00632
1	0.02731
2	0.02729
3	0.03237
4	0.06905
...	...
501	0.06263
502	0.04527
503	0.06076
504	0.10959
505	0.04741
Name: CRIM, Length: 506, dtype: float64	

1	df['NOX']
0	0.538
1	0.469
2	0.469
3	0.458
4	0.458
...	...
501	0.573
502	0.573
503	0.573
504	0.573
505	0.573
Name: NOX, Length: 506, dtype: float64	

1	df['CRIM']+df['NOX']
0	0.54432
1	0.49631
2	0.49629
3	0.49037
4	0.52705
...	...
501	0.63563
502	0.61827
503	0.63376
504	0.68259
505	0.62041
Length: 506, dtype: float64	

행렬(matrix)

나이	성별	몸무게	시력
10	0	30	0.2
20	1	70	1.0
14	1	45	0.7
32	0	50	1.5
56	0	55	0.8



$$X = \begin{pmatrix} 10 & 0 & 30 & 0.2 \\ 20 & 1 & 70 & 1.0 \\ 14 & 1 & 45 & 0.7 \\ 32 & 0 & 50 & 1.5 \\ 56 & 0 & 55 & 0.8 \end{pmatrix}$$

행렬(matrix)

4×3
행 열

$$X = \begin{pmatrix} 10 & 0 & 30 \\ 20 & 1 & 70 \\ 14 & 1 & 45 \\ 32 & 0 & 50 \end{pmatrix}$$

4행

3열

스칼라, 벡터, 행렬

$$x = 1.5$$

$$\mathbf{a} = \begin{pmatrix} 1 \\ 2 \\ 3 \end{pmatrix}$$

$$\mathbf{X} = \begin{pmatrix} 10 & 0 & 30 & 0.2 \\ 20 & 1 & 70 & 1.0 \\ 14 & 1 & 45 & 0.7 \\ 32 & 0 & 50 & 1.5 \\ 56 & 0 & 55 & 0.8 \end{pmatrix}$$

reshape

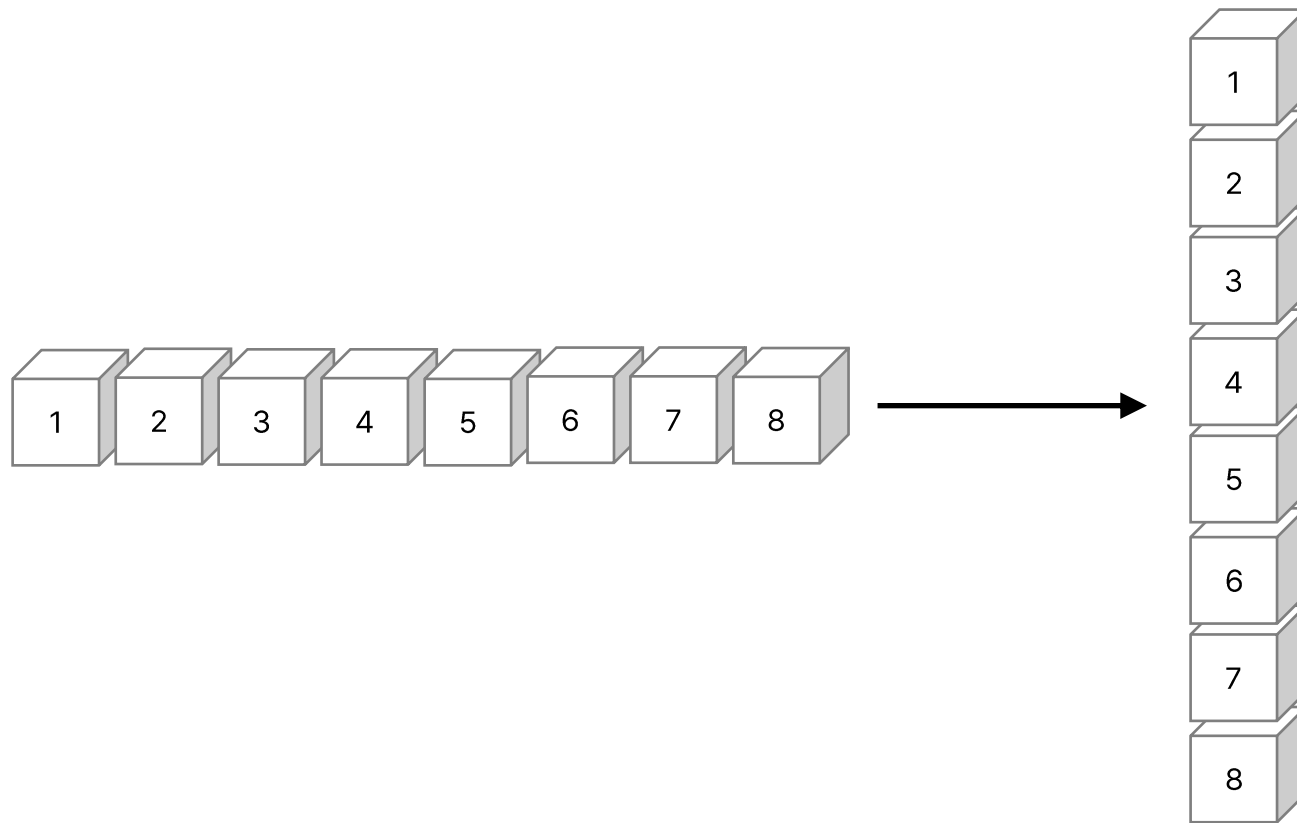
```
In [1]: 1 import numpy as np
```

```
In [2]: 1 A = np.array([1,2,3,4,5,6,7,8])  
2 print(A)  
[1 2 3 4 5 6 7 8]
```

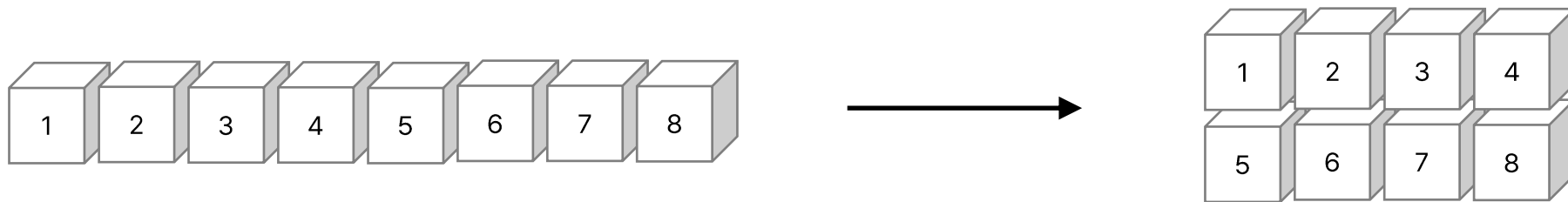
```
In [3]: 1 A1 = A.reshape(8,1)  
2 print(A1)  
[[1]  
 [2]  
 [3]  
 [4]  
 [5]  
 [6]  
 [7]  
 [8]]
```

```
In [4]: 1 A1.shape
```

```
Out[4]: (8, 1)
```



reshape



In [5]:

```
1 A2 = A.reshape(2,4)
2 print(A2)
```

```
[[1 2 3 4]
 [5 6 7 8]]
```

In [6]:

```
1 A3 = np.reshape(A, (2,4))
2 print(A3)
```

```
[[1 2 3 4]
 [5 6 7 8]]
```

In [7]:

```
1 A4 = A.reshape(-1,4)
2 print(A4)
```

```
[[1 2 3 4]
 [5 6 7 8]]
```

In [8]:

```
1 A5 = A.reshape(2, -1)
2 print(A5)
```

```
[[1 2 3 4]
 [5 6 7 8]]
```


reshape

In [10]:

```
1 print(A5)
```

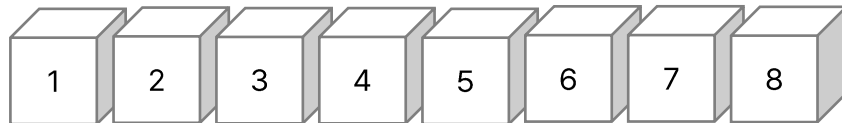
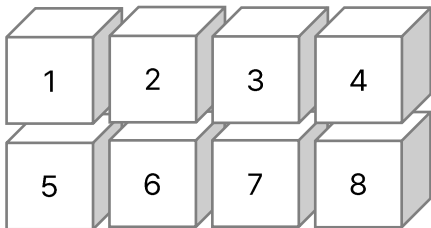
```
[[1 2 3 4]
 [5 6 7 8]]
```

In [9]:

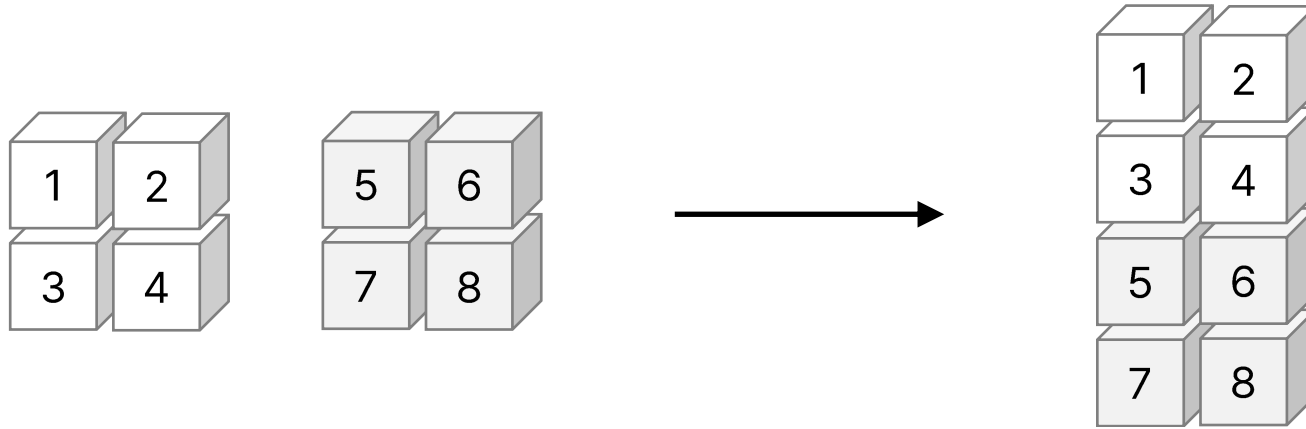
```
1 A6 = A5.reshape(-1)
```

```
2 print(A6)
```

```
[1 2 3 4 5 6 7 8]
```



concatenate



```
In [11]: 1 A = np.array([[1,2], [3,4]])  
        2 B = np.array([[5,6], [7,8]])
```

```
In [12]: 1 print(A)  
[[1 2]  
 [3 4]]
```

```
In [13]: 1 print(B)  
[[5 6]  
 [7 8]]
```

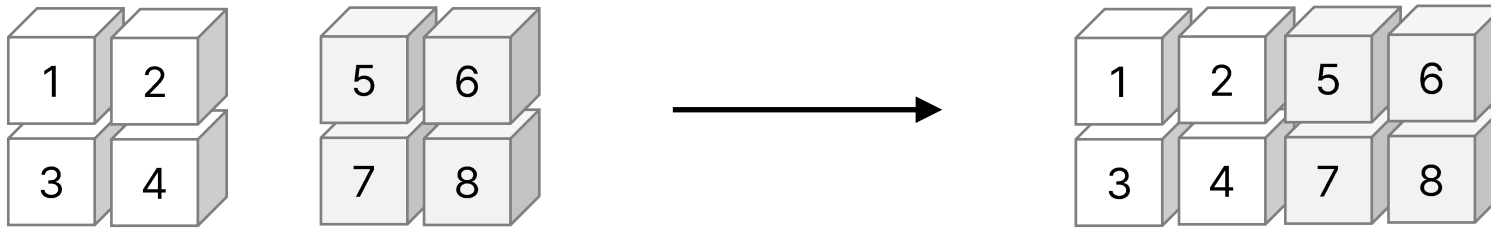
```
In [14]: 1 C = np.concatenate((A, B), axis=0)  
        2 print(C)
```

```
[[1 2]  
 [3 4]  
 [5 6]  
 [7 8]]
```

```
In [15]: 1 C.shape
```

```
Out[15]: (4, 2)
```

concatenate



```
In [17]: 1 D = np.concatenate((A, B), axis=1)
          2 print(D)
          [[1 2 5 6]
           [3 4 7 8]]
```

AI/BigData 인텐시브 과정

Section 1. 선형대수

Section 1-2. 여러 가지 행렬

행렬(matrix)

나이	성별	몸무게	시력
10	0	30	0.2
20	1	70	1.0
14	1	45	0.7
32	0	50	1.5
56	0	55	0.8



$$X = \begin{pmatrix} 10 & 0 & 30 & 0.2 \\ 20 & 1 & 70 & 1.0 \\ 14 & 1 & 45 & 0.7 \\ 32 & 0 & 50 & 1.5 \\ 56 & 0 & 55 & 0.8 \end{pmatrix}$$

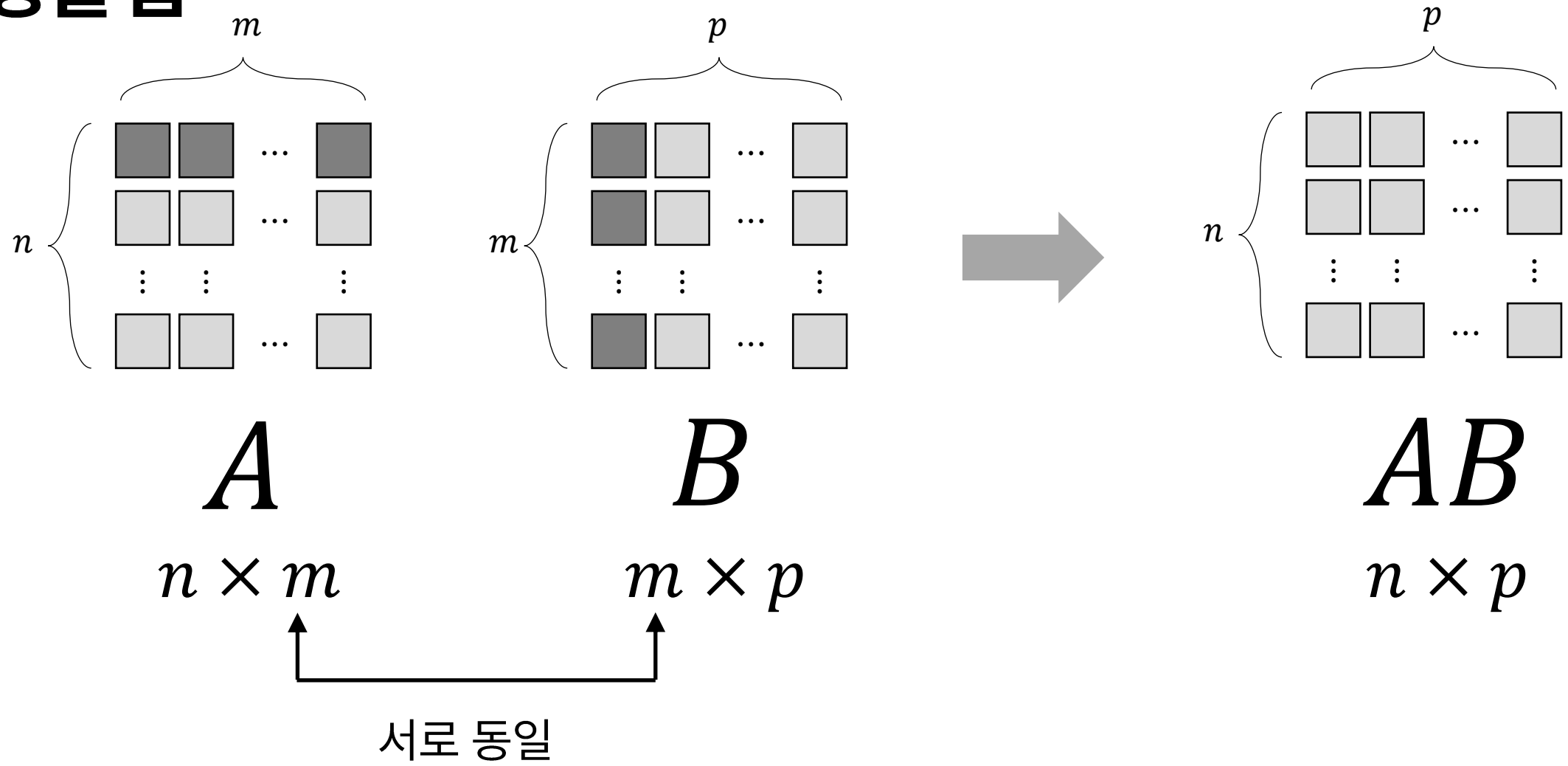
행렬(matrix)

$$\mathbf{A} = \begin{pmatrix} 2 & 7 \\ 3 & 4 \\ 6 & 1 \end{pmatrix}, \quad \mathbf{B} = \begin{pmatrix} 1 & 4 \\ 4 & -1 \\ 2 & 5 \end{pmatrix}$$

$$\mathbf{A} + \mathbf{B} = \begin{pmatrix} 2 & 7 \\ 3 & 4 \\ 6 & 1 \end{pmatrix} + \begin{pmatrix} 1 & 4 \\ 4 & -1 \\ 2 & 5 \end{pmatrix} = \begin{pmatrix} 2+1 & 7+4 \\ 3+4 & 4+(-1) \\ 6+2 & 1+5 \end{pmatrix} = \begin{pmatrix} 3 & 11 \\ 7 & 3 \\ 8 & 6 \end{pmatrix}$$

$$\mathbf{A} - \mathbf{B} = \begin{pmatrix} 2 & 7 \\ 3 & 4 \\ 6 & 1 \end{pmatrix} - \begin{pmatrix} 1 & 4 \\ 4 & -1 \\ 2 & 5 \end{pmatrix} = \begin{pmatrix} 2-1 & 7-4 \\ 3-4 & 4-(-1) \\ 6-2 & 1-5 \end{pmatrix} = \begin{pmatrix} 1 & 3 \\ -1 & 5 \\ 4 & -4 \end{pmatrix}$$

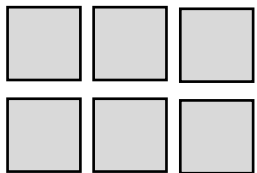
행렬 곱



행렬 곱

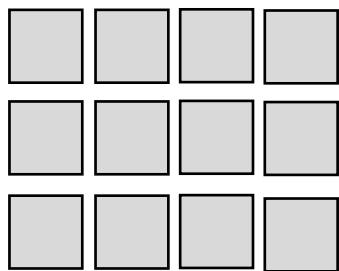
$$\begin{array}{c} (2 \times 3) + (7 \times -1) = -1 \\ \downarrow \\ \mathbf{AB} = \begin{pmatrix} 2 & 7 \\ 3 & 4 \\ 5 & 2 \end{pmatrix} \begin{pmatrix} 3 & -3 & 5 \\ -1 & 2 & 1 \end{pmatrix} = \begin{pmatrix} -1 & 8 & 17 \\ 5 & -1 & 19 \\ 13 & -11 & 27 \end{pmatrix} \\ \begin{matrix} 3 \times 2 & 2 \times 3 & 3 \times 3 \end{matrix} \end{array}$$

행렬 곱이 성립하지 않는 경우



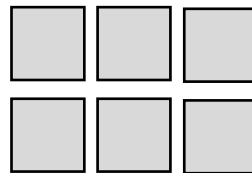
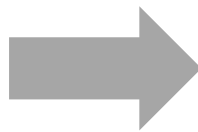
A

$$2 \times 3$$



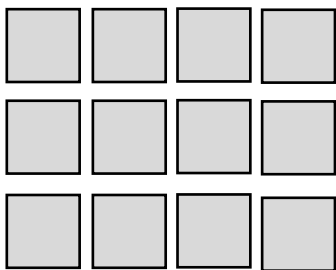
B

$$3 \times 4$$



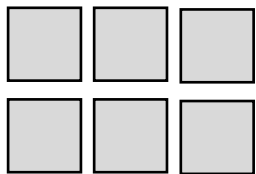
AB

$$2 \times 4$$



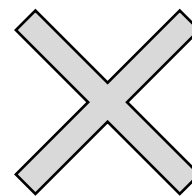
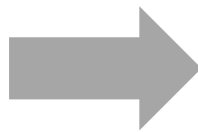
B

$$3 \times 4$$



A

$$2 \times 3$$



BA

행렬 곱은 교환 법칙이 성립하지 않습니다.

$$AB \neq BA$$

행렬 원소 곱

$$A = \begin{pmatrix} 1 & 0 \\ 2 & 3 \end{pmatrix} \qquad B = \begin{pmatrix} 2 & 5 \\ 1 & 3 \end{pmatrix}$$

$$A \odot B = \begin{pmatrix} 1 & 0 \\ 2 & 3 \end{pmatrix} \odot \begin{pmatrix} 2 & 5 \\ 1 & 3 \end{pmatrix} = \begin{pmatrix} 1 \times 2 & 0 \times 5 \\ 2 \times 1 & 3 \times 3 \end{pmatrix} = \begin{pmatrix} 2 & 0 \\ 2 & 9 \end{pmatrix}$$

전치 행렬(transpose matrix)

$$U = \begin{pmatrix} 1 & 5 \\ 3 & 4 \\ 6 & 2 \end{pmatrix} \quad U^T = \begin{pmatrix} 1 & 3 & 6 \\ 5 & 4 & 2 \end{pmatrix}$$

$$\begin{pmatrix} u_{11} & u_{12} \\ u_{21} & u_{22} \\ u_{31} & u_{32} \end{pmatrix} \rightarrow \begin{pmatrix} u_{11} & u_{21} & u_{31} \\ u_{12} & u_{22} & u_{32} \end{pmatrix}$$

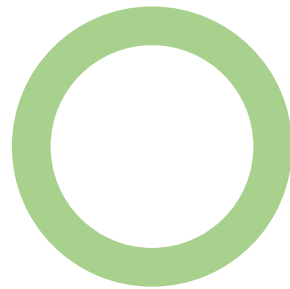
전치 행렬 표기법

행렬 U 의 전치 행렬

$$U = \begin{pmatrix} 1 & 5 \\ 3 & 4 \\ 6 & 2 \end{pmatrix}$$

$$U^T = \begin{pmatrix} 1 & 3 & 6 \\ 5 & 4 & 2 \end{pmatrix}$$

$$U' = \begin{pmatrix} 1 & 3 & 6 \\ 5 & 4 & 2 \end{pmatrix}$$



전치 행렬의 성질

$$(a) \quad (A^T)^T = A$$

$$(b) \quad (A + B)^T = A^T + B^T$$

$$(c) \quad (A - B)^T = A^T - B^T$$

$$(d) \quad (aA)^T = aA^T$$

$$(e) \quad (AB)^T = B^T A^T$$

대칭 행렬(symmmetric matrix)

$$A = \begin{pmatrix} a & b & c \\ b & d & e \\ c & e & f \end{pmatrix}$$

전치 행렬 & 대칭 행렬 실습

```
In [47]: 1 df.columns

Out[47]: Index(['CRIM', 'ZN', 'INDUS', 'CHAS', 'NOX', 'RM', 'AGE', 'DIS', 'RAD', 'TAX',
               'PTRATIO', 'B', 'LSTAT', 'MEDV'],
              dtype='object')
```

```
In [49]: 1 X = df[['CRIM', 'NOX', 'RM', 'AGE', 'TAX']]
          2 X

Out[49]:
```

	CRIM	NOX	RM	AGE	TAX
0	0.00632	0.538	6.575	65.2	296
1	0.02731	0.469	6.421	78.9	242
2	0.02729	0.469	7.185	61.1	242
3	0.03237	0.458	6.998	45.8	222
4	0.06905	0.458	7.147	54.2	222
...	...	...	...	...	...
501	0.06263	0.573	6.593	69.1	273
502	0.04527	0.573	6.120	76.7	273
503	0.06076	0.573	6.976	91.0	273
504	0.10959	0.573	6.794	89.3	273
505	0.04741	0.573	6.030	80.8	273

506 rows x 5 columns

X

전치 행렬 & 대칭 행렬 실습

In [53]: ▶

```
1 X_t = np.transpose(X)
2 X_t
```

Out[53]:

	0	1	2	3	4	5	6	7	8	9	...
CRIM	0.00632	0.02731	0.02729	0.03237	0.06905	0.02985	0.08829	0.14455	0.21124	0.17004	...
NOX	0.53800	0.46900	0.46900	0.45800	0.45800	0.45800	0.52400	0.52400	0.52400	0.52400	...
RM	6.57500	6.42100	7.18500	6.99800	7.14700	6.43000	6.01200	6.17200	5.63100	6.00400	...
AGE	65.20000	78.90000	61.10000	45.80000	54.20000	58.70000	66.60000	96.10000	100.00000	85.90000	...
TAX	296.00000	242.00000	242.00000	222.00000	222.00000	222.00000	311.00000	311.00000	311.00000	311.00000	...

5 rows × 506 columns

◀

In [54]: ▶

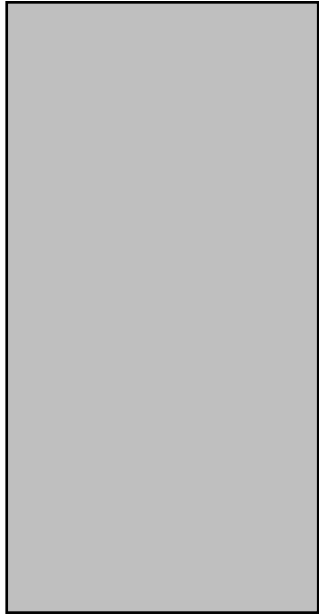
```
1 X_t = X.T
2 X_t
```

Out[54]:

	0	1	2	3	4	5	6	7	8	9	...
CRIM	0.00632	0.02731	0.02729	0.03237	0.06905	0.02985	0.08829	0.14455	0.21124	0.17004	...
NOX	0.53800	0.46900	0.46900	0.45800	0.45800	0.45800	0.52400	0.52400	0.52400	0.52400	...
RM	6.57500	6.42100	7.18500	6.99800	7.14700	6.43000	6.01200	6.17200	5.63100	6.00400	...
AGE	65.20000	78.90000	61.10000	45.80000	54.20000	58.70000	66.60000	96.10000	100.00000	85.90000	...
TAX	296.00000	242.00000	242.00000	222.00000	222.00000	222.00000	311.00000	311.00000	311.00000	311.00000	...

5 rows × 506 columns

전치 행렬 & 대칭 행렬 실습



X



X^T

행렬 곱 실습

```
In [55]: ▶ 1 X_t.dot(X)
```

Out[55]:

	CRIM	NOX	RM	AGE	TAX
CRIM	4.397034e+04	1226.123170	1.082195e+04	1.685150e+05	1.173073e+06
NOX	1.226123e+03	162.470380	1.751519e+03	2.045220e+04	1.211706e+05
RM	1.082195e+04	1751.519414	2.023460e+04	2.156702e+05	1.280740e+06
AGE	1.685150e+05	20452.201630	2.156702e+05	2.779615e+06	1.537874e+07
TAX	1.173073e+06	121170.623200	1.280740e+06	1.537874e+07	9.867314e+07

```
In [56]: ▶ 1 X_t@X
```

Out[56]:

	CRIM	NOX	RM	AGE	TAX
CRIM	4.397034e+04	1226.123170	1.082195e+04	1.685150e+05	1.173073e+06
NOX	1.226123e+03	162.470380	1.751519e+03	2.045220e+04	1.211706e+05
RM	1.082195e+04	1751.519414	2.023460e+04	2.156702e+05	1.280740e+06
AGE	1.685150e+05	20452.201630	2.156702e+05	2.779615e+06	1.537874e+07
TAX	1.173073e+06	121170.623200	1.280740e+06	1.537874e+07	9.867314e+07

$$X^T X$$

전치 행렬 & 대칭 행렬 실습

The diagram illustrates the multiplication of two matrices, X^T and X , to produce the matrix $X^T X$. On the left, a horizontal gray rectangle labeled 5×506 represents X^T , and a vertical gray rectangle labeled 506×5 represents X . An equals sign follows, leading to a square gray rectangle labeled 5×5 representing the result $X^T X$. Below each rectangle is its corresponding matrix notation: X^T under the first rectangle, X under the second, and $X^T X$ under the result rectangle.

$$\begin{matrix} 5 \times 506 & 506 \times 5 & = & 5 \times 5 \\ X^T & X & & X^T X \end{matrix}$$

대각 행렬(diagonal matrix)

$$\mathbf{D} = \begin{pmatrix} d_{11} & 0 & 0 \\ 0 & d_{22} & 0 \\ 0 & 0 & d_{33} \end{pmatrix}$$

주 대각 성분을 제외한 모든 행렬 성분이 0인 정 사각 행렬

단위 행렬(항등 행렬, identity matrix)

$$\mathbf{I} = \begin{pmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \end{pmatrix}$$

단위 행렬(항등 행렬, identity matrix)

$$A = \begin{pmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \\ 7 & 8 & 9 \end{pmatrix}, \quad I = \begin{pmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \end{pmatrix}$$

$$AI = \begin{pmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \\ 7 & 8 & 9 \end{pmatrix} \begin{pmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \end{pmatrix} = \begin{pmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \\ 7 & 8 & 9 \end{pmatrix} = A$$

$$IA = \begin{pmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \end{pmatrix} \begin{pmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \\ 7 & 8 & 9 \end{pmatrix} = \begin{pmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \\ 7 & 8 & 9 \end{pmatrix} = A$$

$$AI = IA = A$$

삼각 행렬

$$A = \begin{pmatrix} a_{11} & a_{12} & a_{13} \\ 0 & a_{22} & a_{23} \\ 0 & 0 & a_{33} \end{pmatrix}$$

상 삼각 행렬
(upper triangular matrix)

$$A = \begin{pmatrix} a_{11} & 0 & 0 \\ a_{21} & a_{22} & 0 \\ a_{31} & a_{32} & a_{33} \end{pmatrix}$$

하 삼각 행렬
(lower triangular matrix)

영 행렬(zero matrix)

$$\mathbf{0} = \begin{pmatrix} 0 & 0 & 0 \\ 0 & 0 & 0 \\ 0 & 0 & 0 \end{pmatrix}$$

$$\mathbf{0} = \begin{pmatrix} 0 & 0 \\ 0 & 0 \\ 0 & 0 \end{pmatrix}$$

$$\mathbf{0} = \begin{pmatrix} 0 & 0 & 0 \\ 0 & 0 & 0 \end{pmatrix}$$

원 벡터(one vector)

$$\mathbf{1} = \begin{pmatrix} 1 \\ 1 \\ 1 \end{pmatrix}$$

$$\mathbf{1}^T = (1 \quad 1 \quad 1)$$

$$\mathbf{1} = \begin{pmatrix} 1 \\ 1 \end{pmatrix}$$

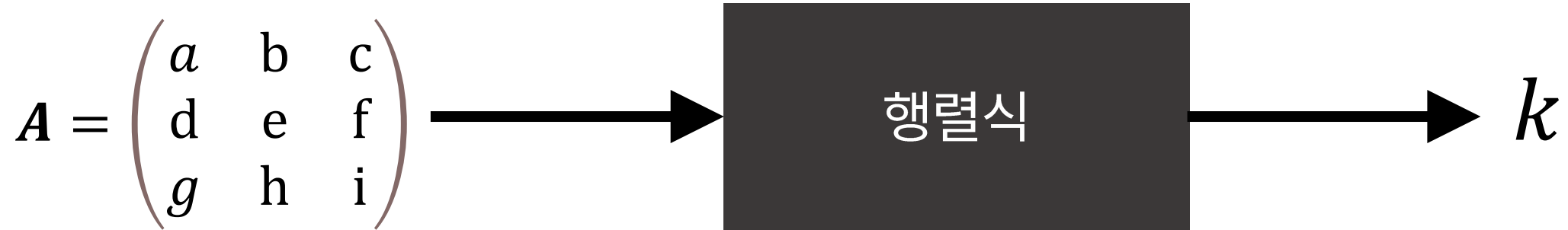
$$\mathbf{1}^T = (1 \quad 1)$$

AI/BigData 인텐시브 과정

Section 1. 선형대수

Section 1-3. 행렬식

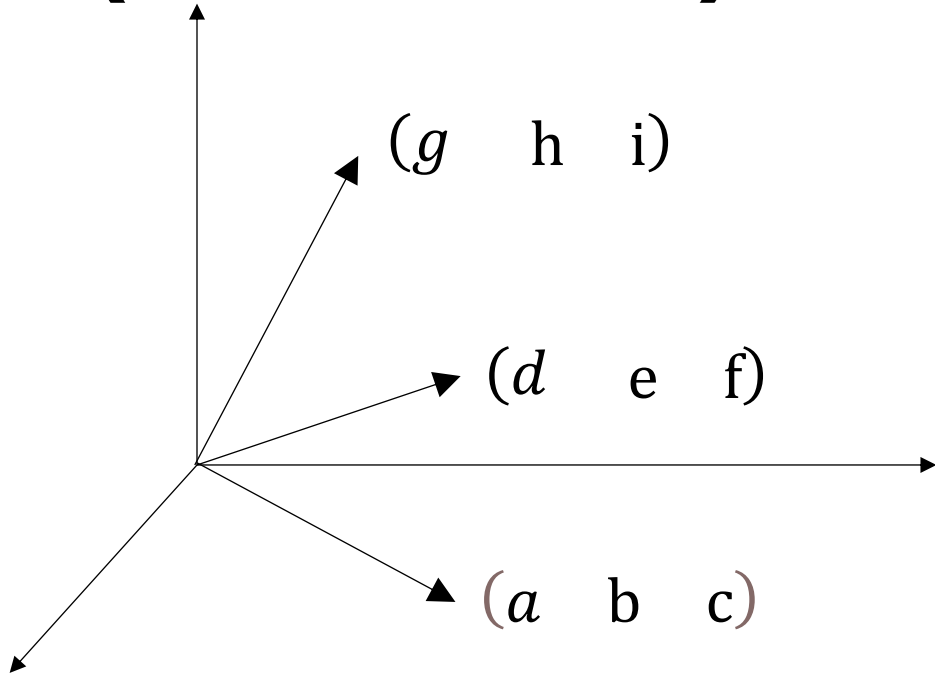
행렬식(determinant)



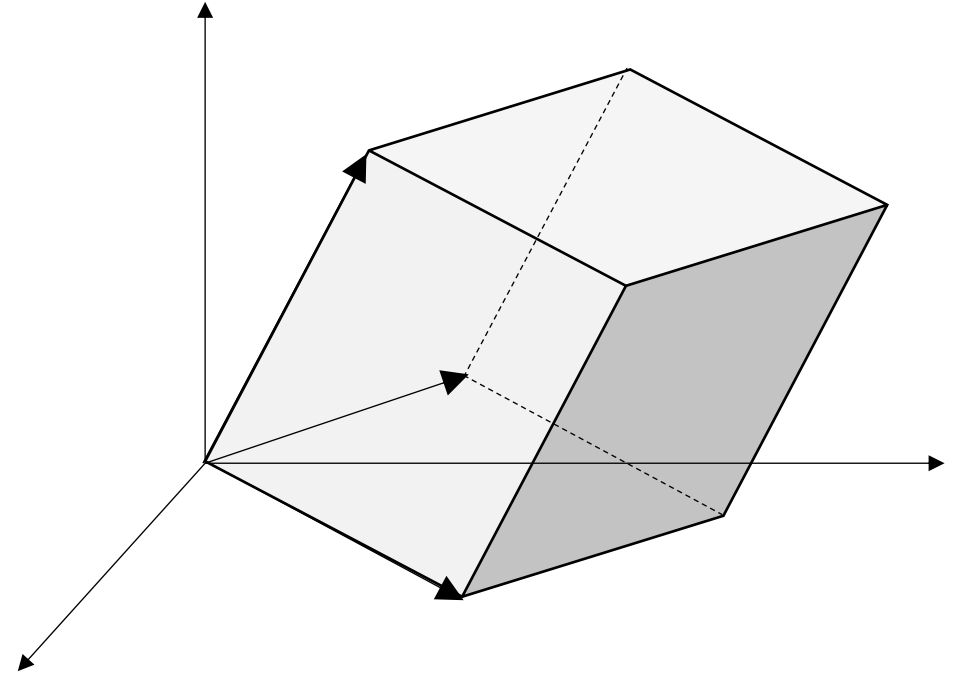
행렬식: 정사각 행렬을 스칼라로 변환하는 함수

- 정사각 행렬에만 해당
- 행렬을 스칼라로 변환

행렬식(determinant)



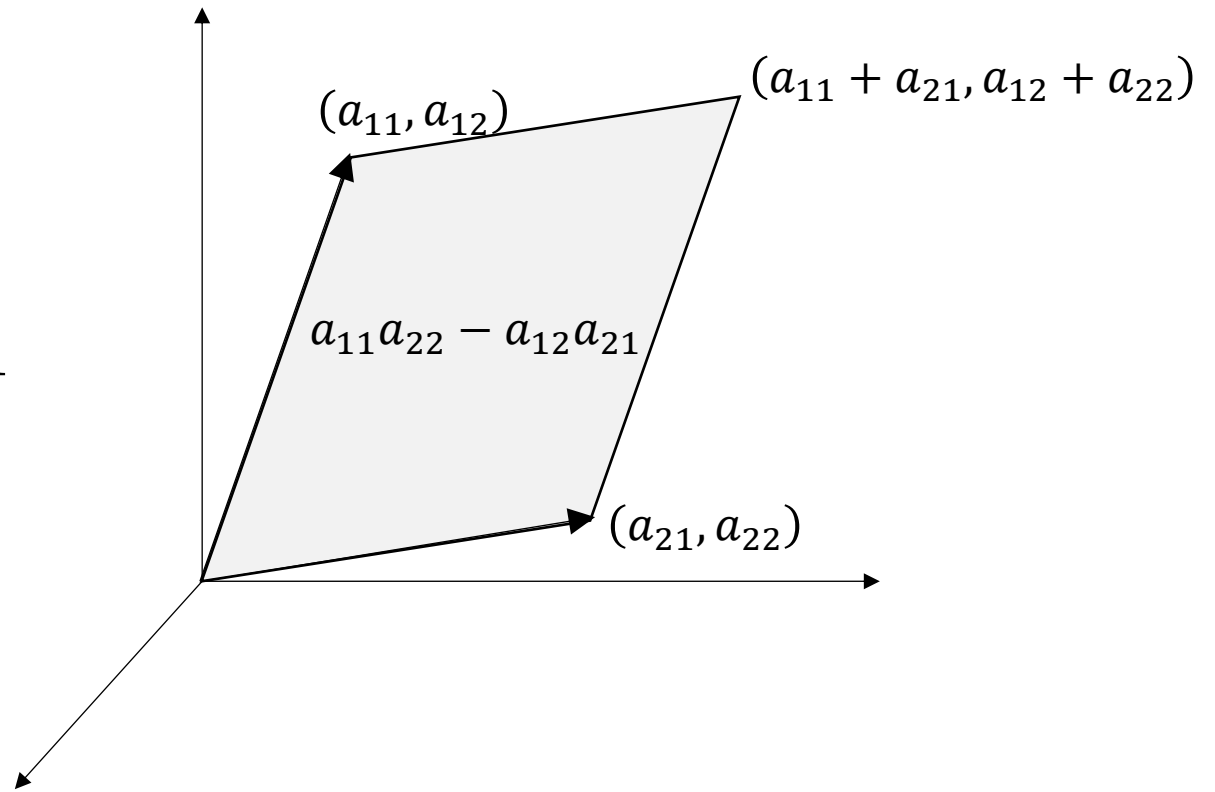
$$A = \begin{pmatrix} a & b & c \\ d & e & f \\ g & h & i \end{pmatrix}$$



$$|A| = \begin{vmatrix} a & b & c \\ d & e & f \\ g & h & i \end{vmatrix}$$

행렬식(determinant)

$$\begin{vmatrix} a_{11} & a_{12} \\ a_{21} & a_{22} \end{vmatrix} = a_{11}a_{22} - a_{12}a_{21}$$



행렬식(determinant)

$$A = \begin{pmatrix} 1 & 2 & 3 \\ 2 & 4 & 6 \\ 1 & 0 & 1 \end{pmatrix}$$

$$\det(A) = 0$$

$$B = \begin{pmatrix} 1 & 2 & 2 \\ 2 & 4 & 0 \\ 3 & 6 & 1 \end{pmatrix}$$

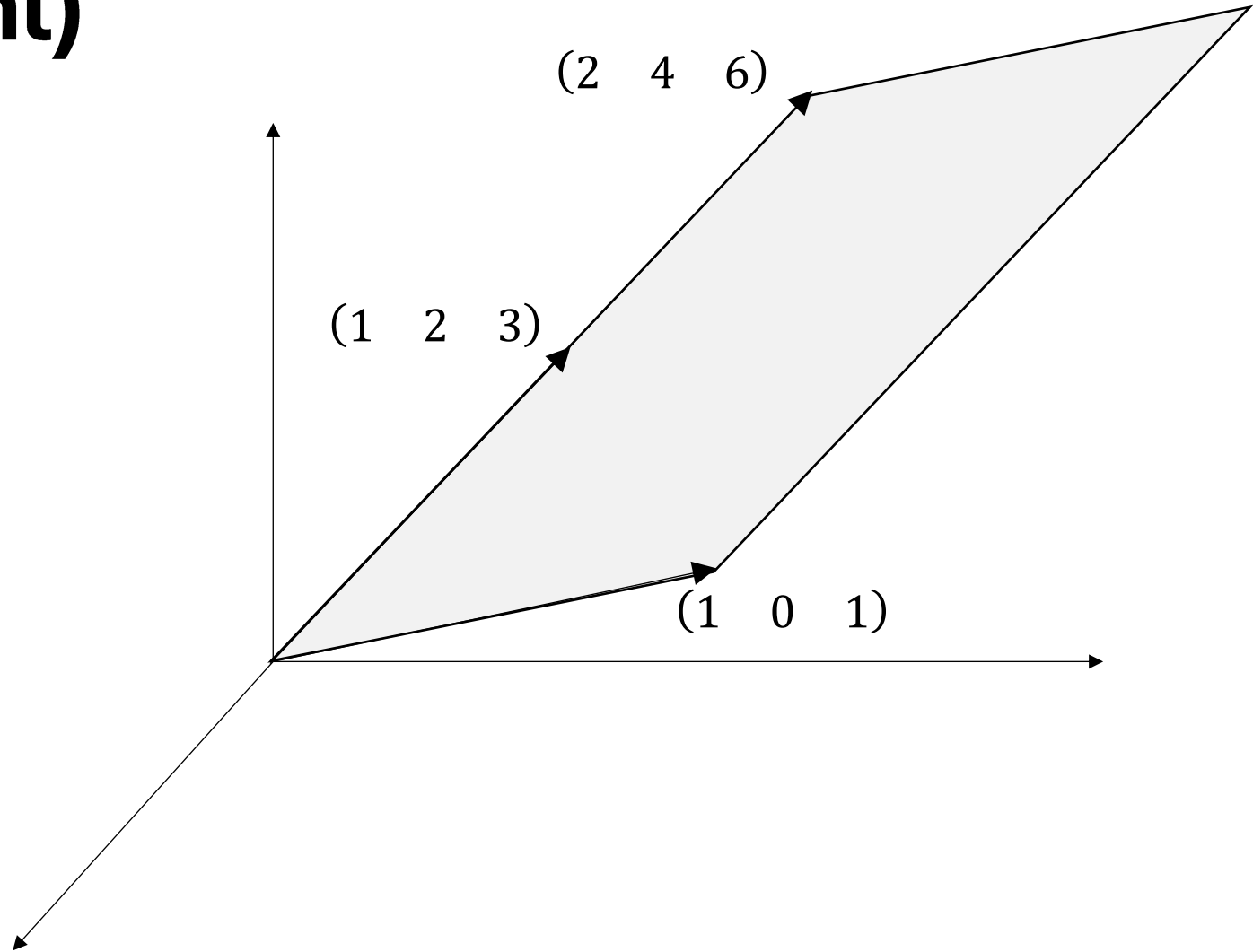
$$\det(B) = 0$$

Why?

행렬식(determinant)

$$A = \begin{pmatrix} 1 & 2 & 3 \\ 2 & 4 & 6 \\ 1 & 0 & 1 \end{pmatrix}$$

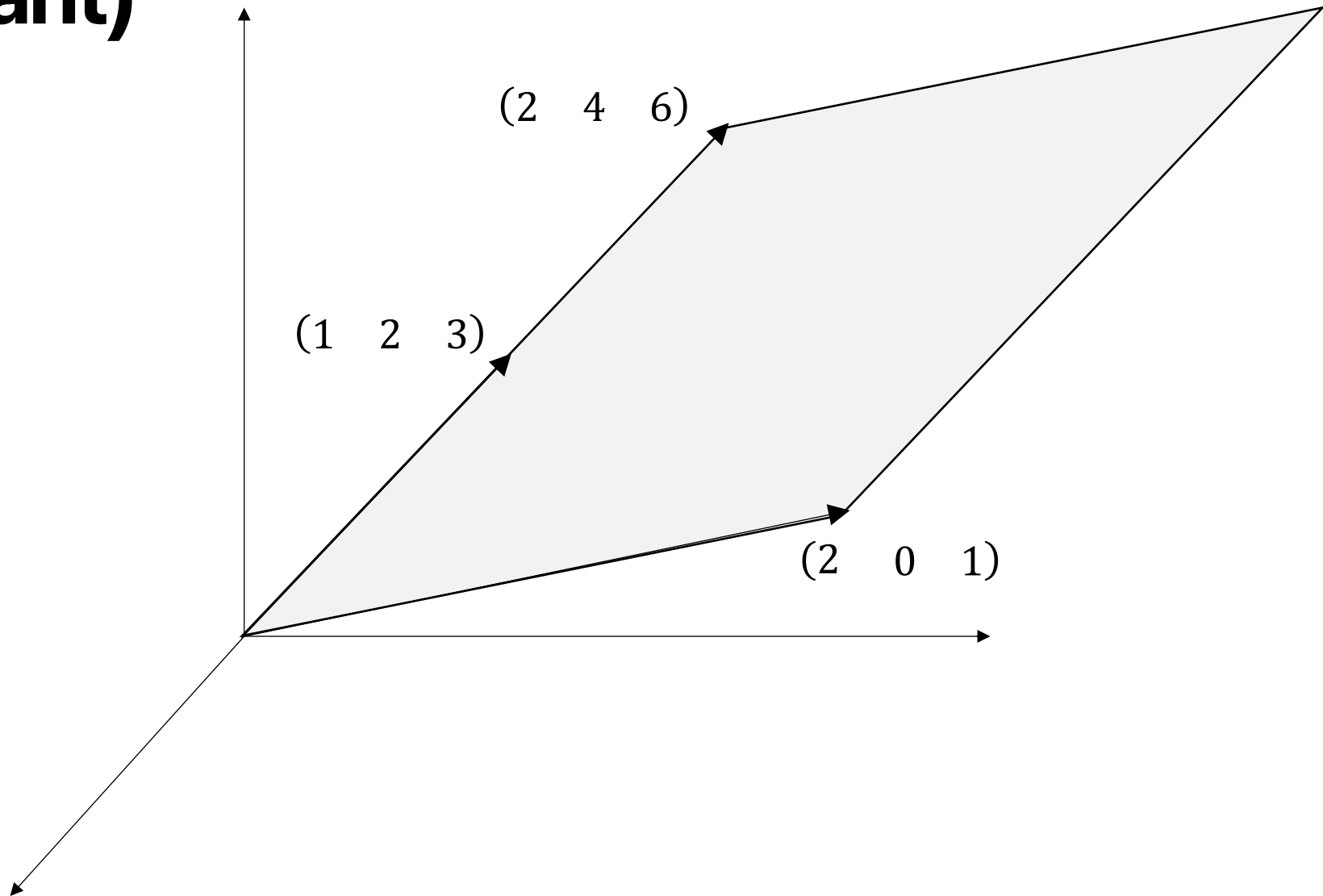
$$\det(A) = 0$$



행렬식(determinant)

$$\mathbf{B} = \begin{pmatrix} 1 & 2 & 2 \\ 2 & 4 & 0 \\ 3 & 6 & 1 \end{pmatrix}$$

$$\det(\mathbf{B}) = 0$$



행렬식(determinant)

```
In [57]: ▶ 1 XtX = X_t@X
```

```
In [58]: ▶ 1 np.linalg.det(XtX)
```

```
Out[58]: 7.91431657769047e+21
```

행렬식(determinant)

x1	x2	x3	x4	x5

$$x_2 = x_5$$

x1	x2	x3	x4	x5

$$\det(X^T X) = 0$$

$$x_1 = x_4 * 3$$

x1	x2	x3	x4	x5

$$\det(X^T X) = 0$$

$$x_2 = x_5$$

x1	x2	x3	x4	x5

$$\det(X^T X) = 0$$

```
In [39]: 1 A = df[['CRIM', 'NOX', 'NOX']]
```

```
In [40]: 1 AtA = A.T@A
```

```
In [48]: 1 AtA
```

Out[48]:

	CRIM	NOX	NOX
CRIM	43970.343555	1226.12317	1226.12317
NOX	1226.123170	162.47038	162.47038
NOX	1226.123170	162.47038	162.47038

```
In [41]: 1 np.linalg.det(AtA)
```

Out[41]: -3.2062532690792516e-07

$x_1 = x_4 \times 3$

x1	x2	x3	x4	x5

$\det(X^T X) = 0$

```
In [42]: 1 B = df[['CRIM', 'NOX']]
```

```
In [43]: 1 B['NOX3'] = B['NOX']*3
```

C:\Users\Wstois\AppData\Local\Temp\Wipyke
WithCopyWarning:
A value is trying to be set on a copy of a DataFrame or Series resulting in a new object being created rather than performing an in-place modification.
Try using .loc[row_indexer,col_indexer] for better performance.

See the caveats in the documentation: https://pandas.pydata.org/pandas-docs/stable/user_guide/indexing.html#returning-a-copy-versus-modifying-in-place
B['NOX3'] = B['NOX']*3

```
In [44]: 1 B
```

Out[44]:

	CRIM	NOX	NOX3
0	0.00632	0.538	1.614
1	0.02731	0.469	1.407
2	0.02729	0.469	1.407
3	0.03237	0.458	1.374
4	0.06905	0.458	1.374
...	...	...	...

```
In [45]: 1 BtB = B.T@B
```

```
In [47]: 1 BtB
```

Out[47]:

	CRIM	NOX	NOX3
CRIM	43970.343555	1226.12317	3678.369511
NOX	1226.123170	162.47038	487.411140
NOX3	3678.369511	487.41114	1462.233421

```
In [46]: 1 np.linalg.det(BtB)
```

Out[46]: 0.0

AI/BigData 인텐시브 과정

Section 1. 선형대수

Section 1-4. 역행렬

역행렬(inverse matrix)

$$AA^{-1} = A^{-1}A = I$$

역행렬(inverse matrix)

$$A = \begin{pmatrix} a_{11} & a_{12} \\ a_{21} & a_{22} \end{pmatrix}$$

$$A^{-1} = \frac{1}{a_{11}a_{22} - a_{12}a_{21}} \begin{pmatrix} a_{22} & -a_{12} \\ -a_{21} & a_{11} \end{pmatrix}$$

↓
행렬식

역행렬(inverse matrix)

역행렬이 존재함 $\iff$ 행렬식 $\neq 0$ $\iff$ nonsingular matrix

역행렬이 존재하지 않음 $\iff$ 행렬식 $= 0$ $\iff$ singular matrix

역행렬(inverse matrix)

```
In [59]: ▶ 1 XtX_inv = np.linalg.inv(XtX)
```

```
In [61]: ▶ 1 print(XtX_inv)
```

```
[[ 4.07613803e-05  1.32526683e-04  4.76832932e-05 -1.01889386e-06
   -1.10744437e-06]
 [ 1.32526683e-04  3.44301609e-01 -1.30762150e-02 -8.56047279e-04
   -1.21234099e-04]
 [ 4.76832932e-05 -1.30762150e-02  9.05341934e-04  1.73338529e-05
    1.03814433e-06]
 [-1.01889386e-06 -8.56047279e-04  1.73338529e-05  5.35211609e-06
    4.19638894e-09]
 [-1.10744437e-06 -1.21234099e-04  1.03814433e-06  4.19638894e-09
    1.58047038e-07]]
```

역행렬(inverse matrix)

In [62]: ▶ 1 XtX @ XtX_inv

Out[62]:

	0	1	2	3	4
CRIM	1.000000e+00	0.000000e+00	3.108624e-15	7.806256e-17	-5.551115e-17
NOX	-2.775558e-17	1.000000e+00	-3.885781e-16	-8.023096e-18	0.000000e+00
RM	-4.440892e-16	-8.526513e-14	1.000000e+00	-1.396452e-16	2.775558e-17
AGE	0.000000e+00	-6.821210e-13	3.907985e-14	1.000000e+00	0.000000e+00
TAX	0.000000e+00	3.637979e-12	8.526513e-14	-4.052314e-15	1.000000e+00

In [63]: ▶ 1 XtX_inv @ XtX

Out[63]:

	CRIM	NOX	RM	AGE	TAX
0	1.000000e+00	0.000000e+00	8.881784e-16	0.000000e+00	1.421085e-14
1	-1.989520e-13	1.000000e+00	-2.557954e-13	-2.046363e-12	-1.637090e-11
2	1.310063e-14	1.110223e-15	1.000000e+00	1.367795e-13	1.151079e-12
3	2.966377e-16	7.372575e-18	1.908196e-17	1.000000e+00	2.814415e-14
4	2.775558e-17	0.000000e+00	2.775558e-17	4.440892e-16	1.000000e+00

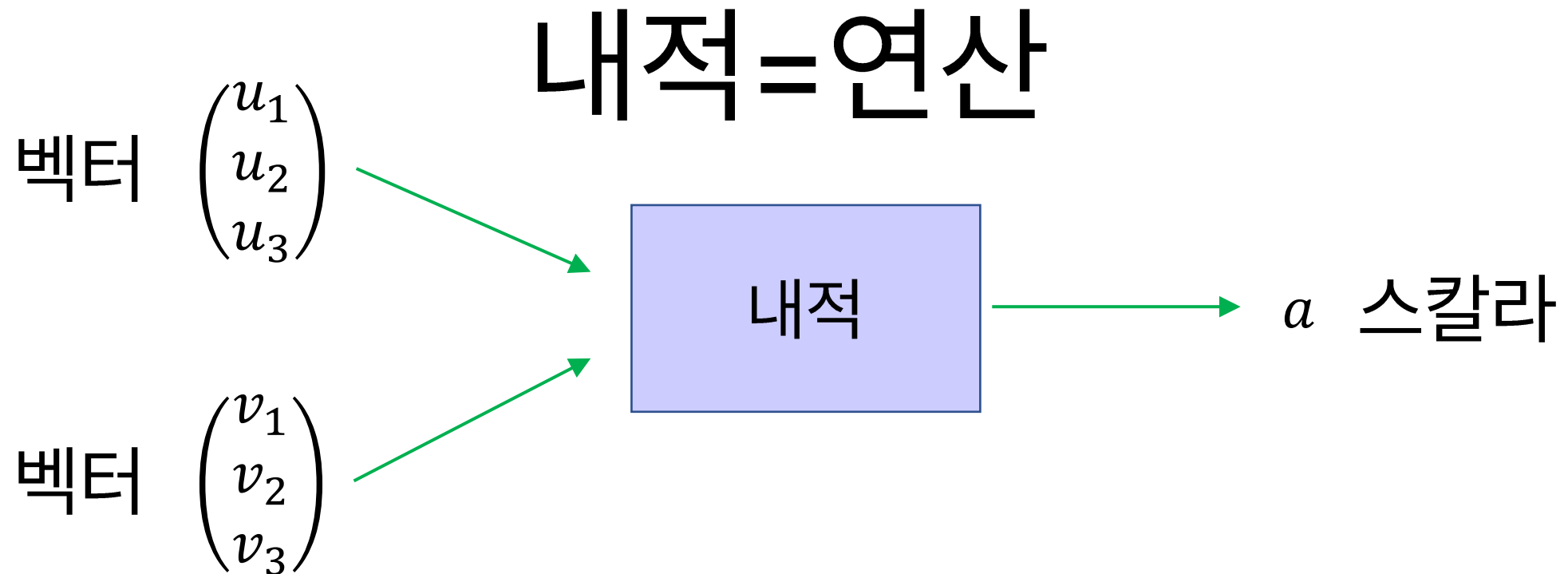
$$XX^{-1} = X^{-1}X = I$$

AI/BigData 인텐시브 과정

Section 1. 선형대수

Section 1-5. 내적

내적(inner product)



내적(inner product)

$$\langle \mathbf{u}, \mathbf{v} \rangle = \mathbf{u} \cdot \mathbf{v} = u_1 v_1 + u_2 v_2 + \cdots + u_n v_n$$

$$\mathbf{u} = \begin{pmatrix} u_1 \\ u_2 \\ u_3 \end{pmatrix}, \quad \mathbf{v} = \begin{pmatrix} v_1 \\ v_2 \\ v_3 \end{pmatrix}$$

$$\langle \mathbf{u}, \mathbf{v} \rangle = \mathbf{u}^T \mathbf{v} = (u_1 \quad u_2 \quad u_3) \begin{pmatrix} v_1 \\ v_2 \\ v_3 \end{pmatrix} = u_1 v_1 + u_2 v_2 + \cdots + u_n v_n$$

내적(inner product)

$$\langle \mathbf{u}, \mathbf{v} \rangle = \mathbf{u}^T \mathbf{v}$$

$$\mathbf{u} = \begin{pmatrix} 1 \\ 2 \\ 3 \end{pmatrix}, \quad \mathbf{v} = \begin{pmatrix} 3 \\ 2 \\ 3 \end{pmatrix}$$

$$= (1 \quad 2 \quad 3) \begin{pmatrix} 3 \\ 2 \\ 3 \end{pmatrix}$$

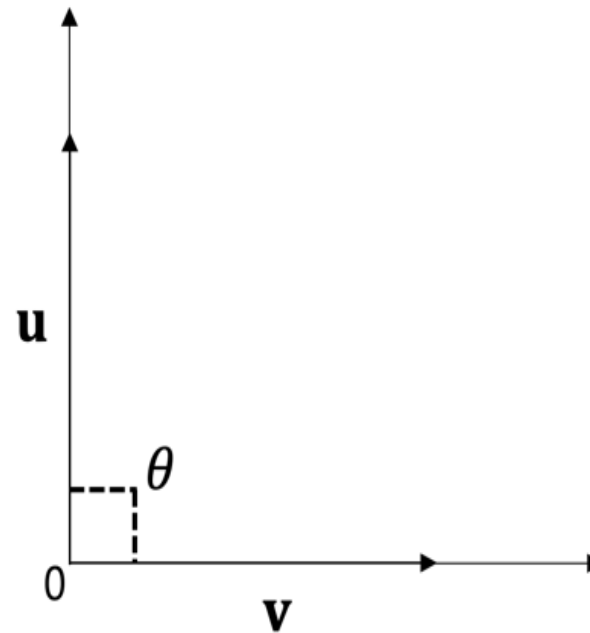
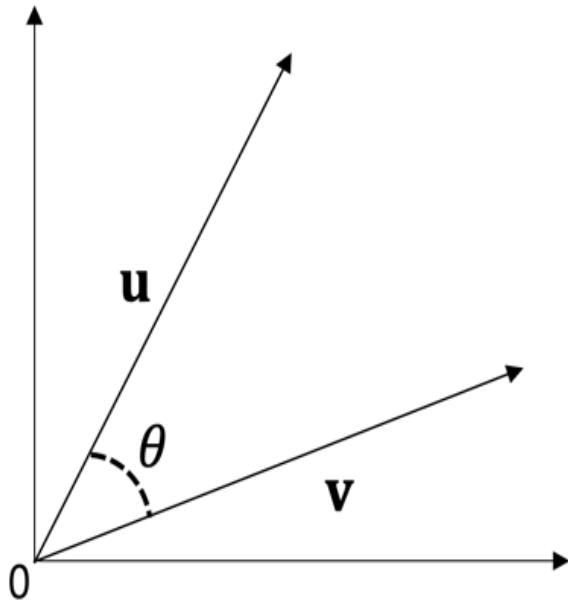
$$= 1 \times 3 + 2 \times 2 + 3 \times 3$$

$$= 3 + 4 + 9$$

$$= 16$$

내적(inner product)

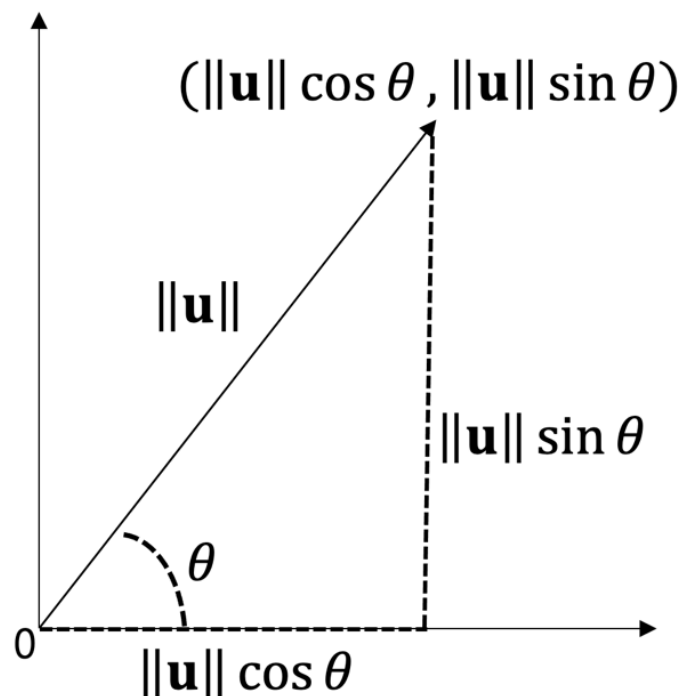
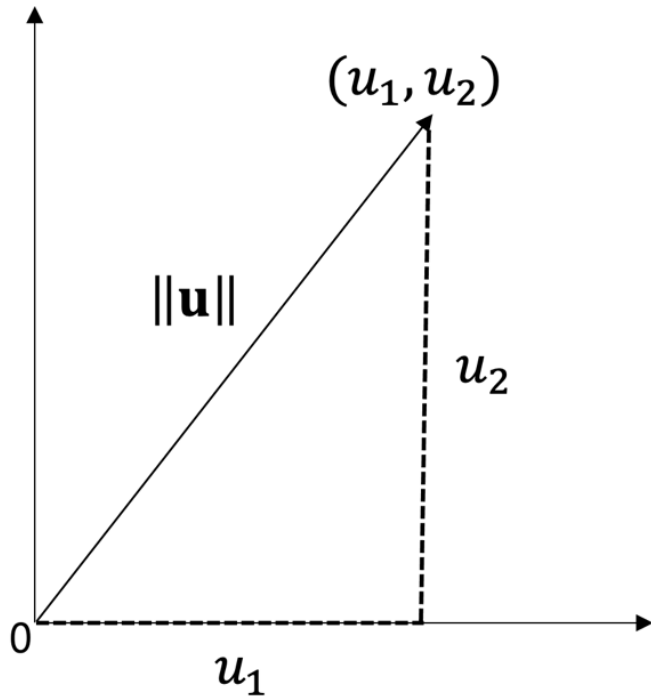
- (1) 내적 > 0 이면, 두 벡터 사이의 각도 < 90
- (2) 내적 < 0 이면, 두 벡터 사이의 각도 > 90
- (3) 내적 $= 0$ 이면, 두 벡터 사이의 각도 $= 90$



노름(norm)

$$\mathbf{u} = (u_1, u_2, \dots, u_n)$$

$$\|\mathbf{u}\| = \sqrt{u_1^2 + u_2^2 + \dots + u_n^2}$$



p-norm

$$\|x\|_p = \left(\sum_{i=1}^n |x_i|^p \right)^{1/p}$$

내적(inner product)

In [64]: ▶

```
1 crim = df['CRIM']  
2 nox = df['NOX']
```

In [66]: ▶

```
1 res = np.inner(crim, nox)  
2 print(res)
```

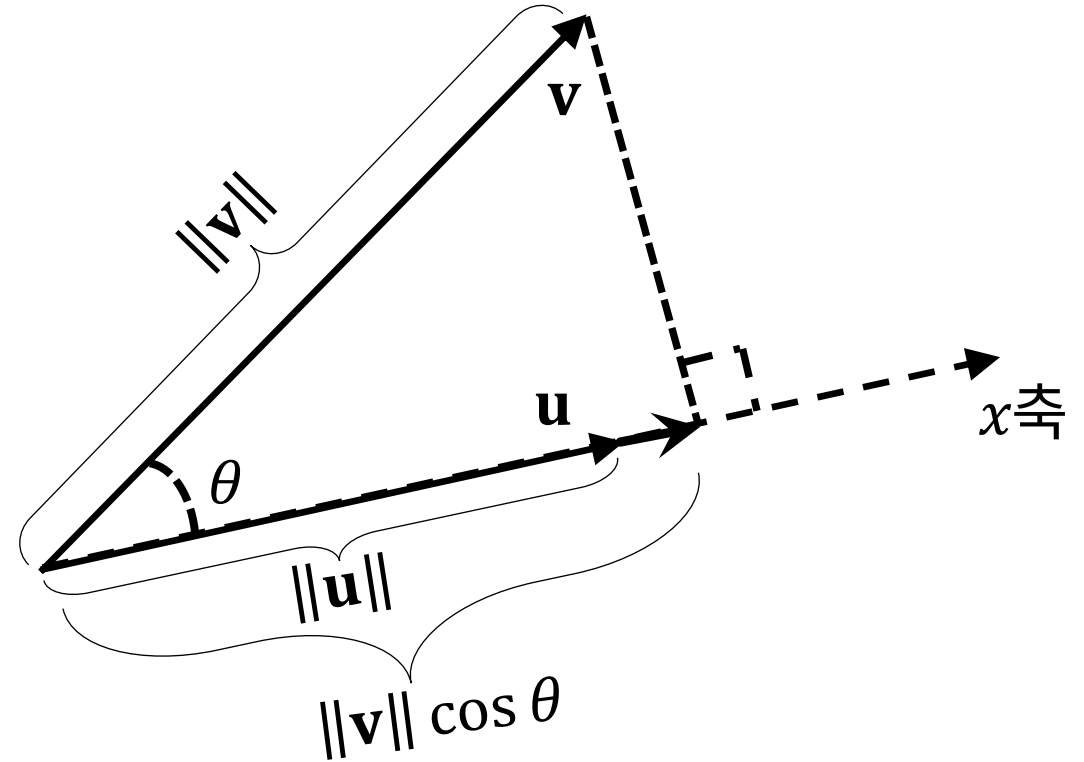
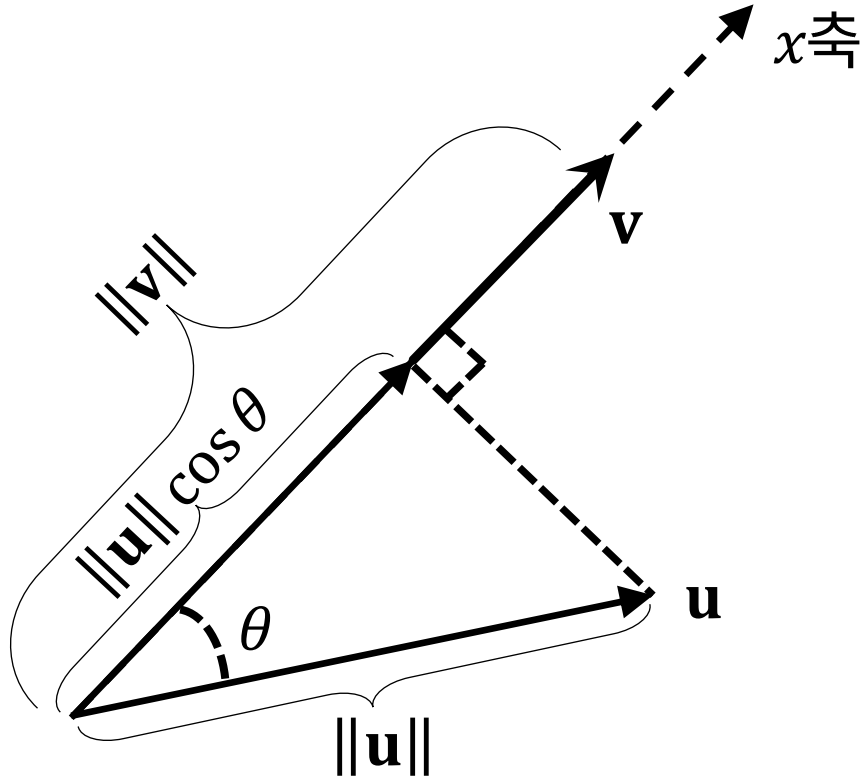
1226.1231702310001

벡터의 길이

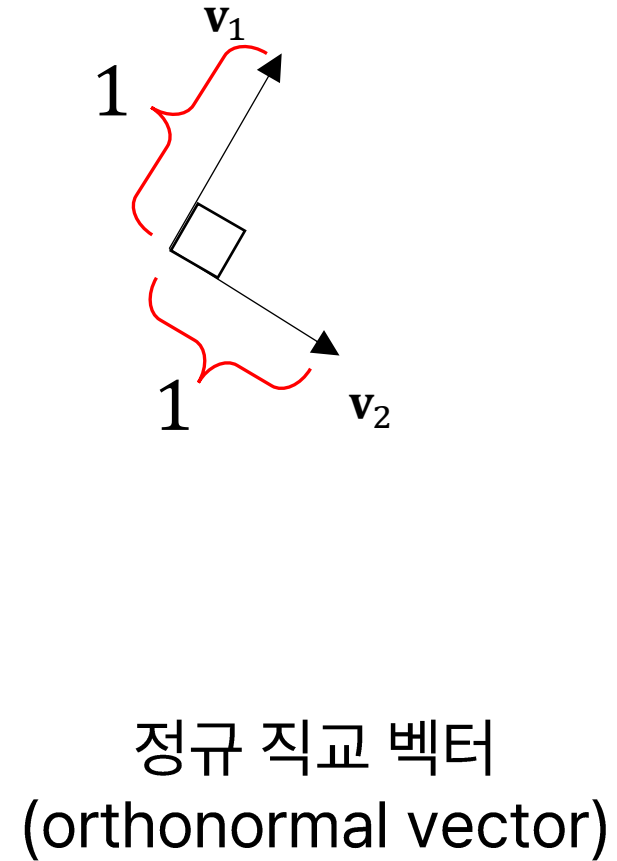
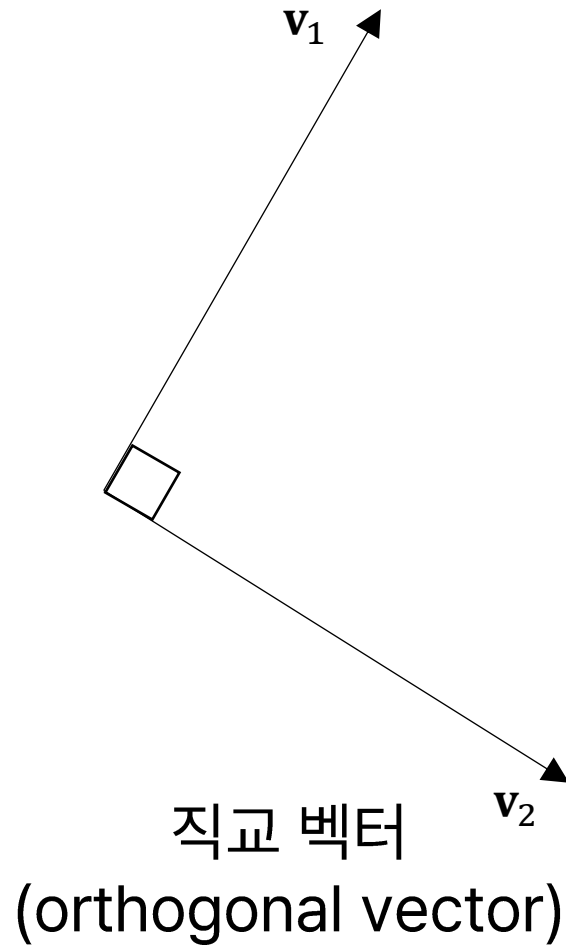
$$\mathbf{u} \cdot \mathbf{v} = \|\mathbf{u}\| \|\mathbf{v}\| \cos \theta$$

$$\cos \theta = \frac{\mathbf{u} \cdot \mathbf{v}}{\|\mathbf{u}\| \|\mathbf{v}\|}$$

정사영(projection)



정규 직교 벡터(orthonormal vector)

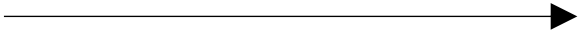


AI/BigData 인텐시브 과정

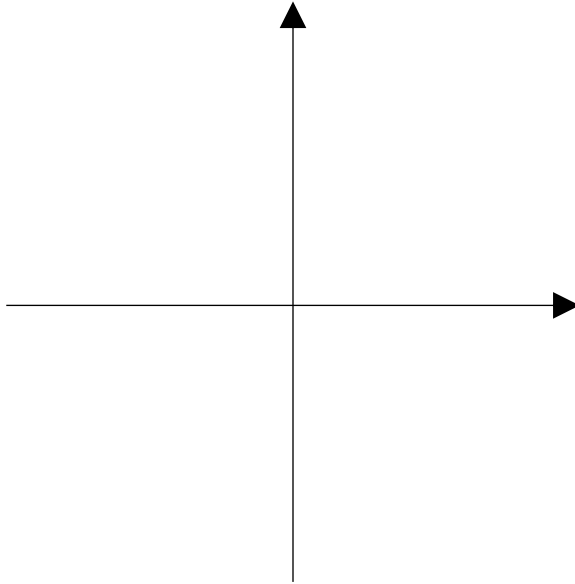
Section 1. 선형대수

Section 1-6. 기저와 차원

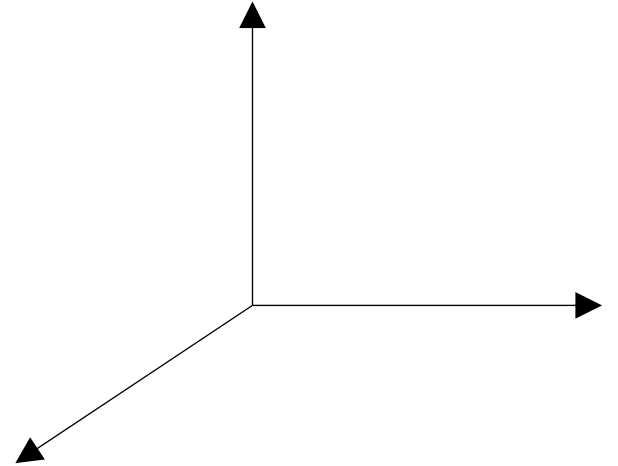
기저(basis)와 차원(dimension)



(a) 1차원

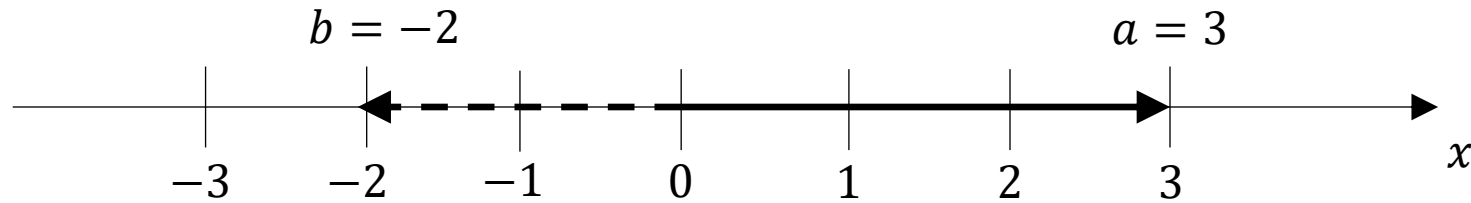


(b) 2차원



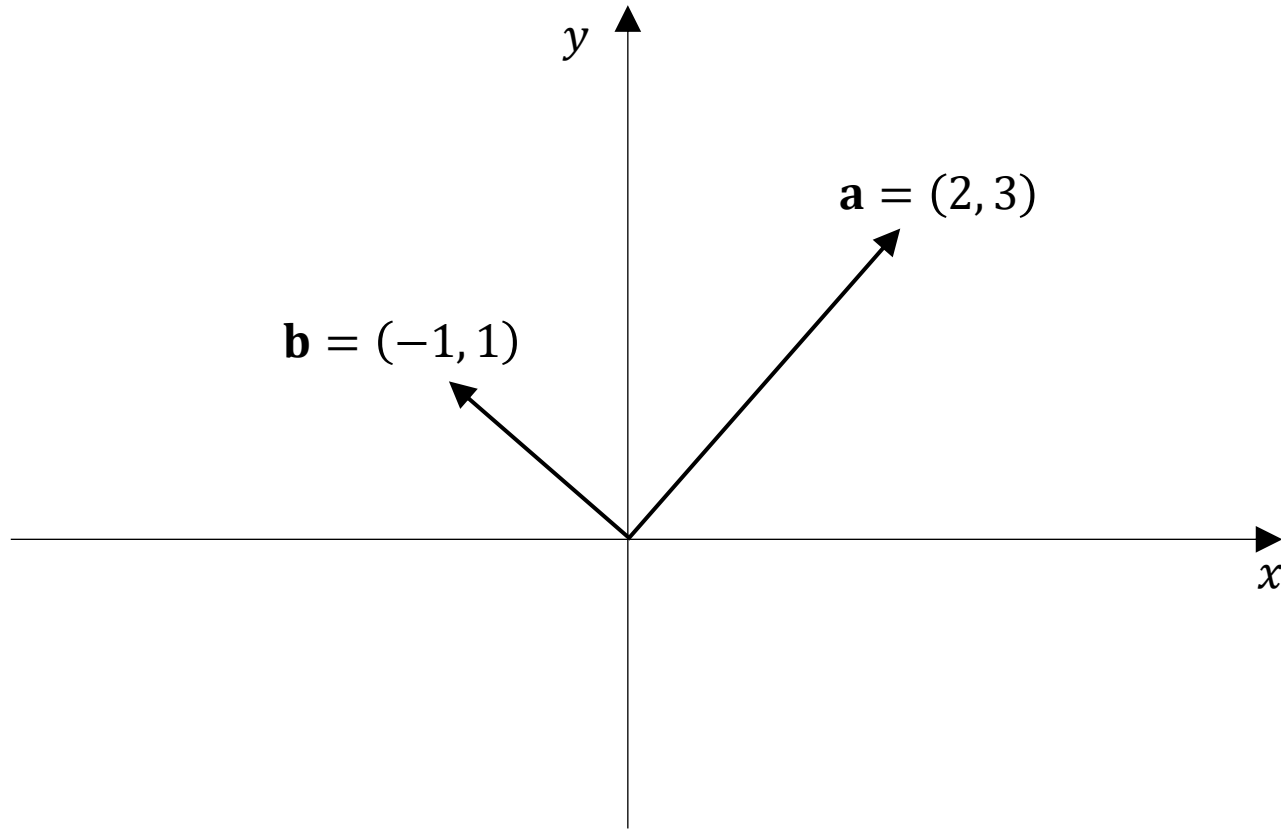
(c) 3차원

기저(basis)와 차원(dimension)



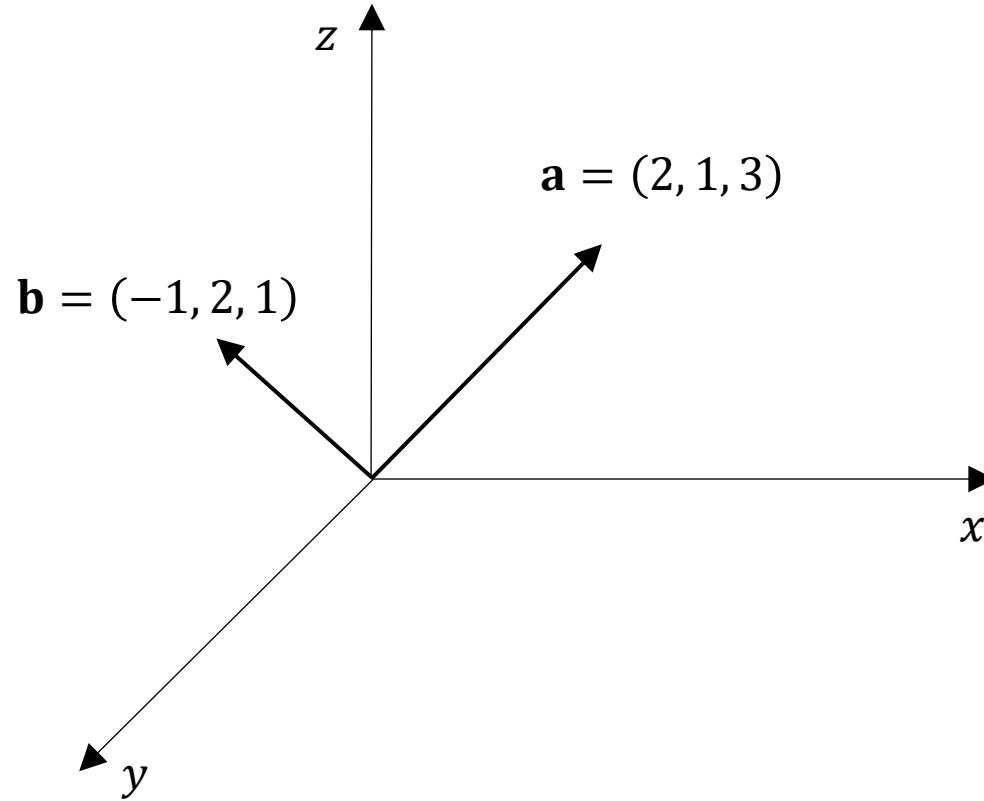
1차원 공간에서는 1개의 좌표값 필요

기저와 차원



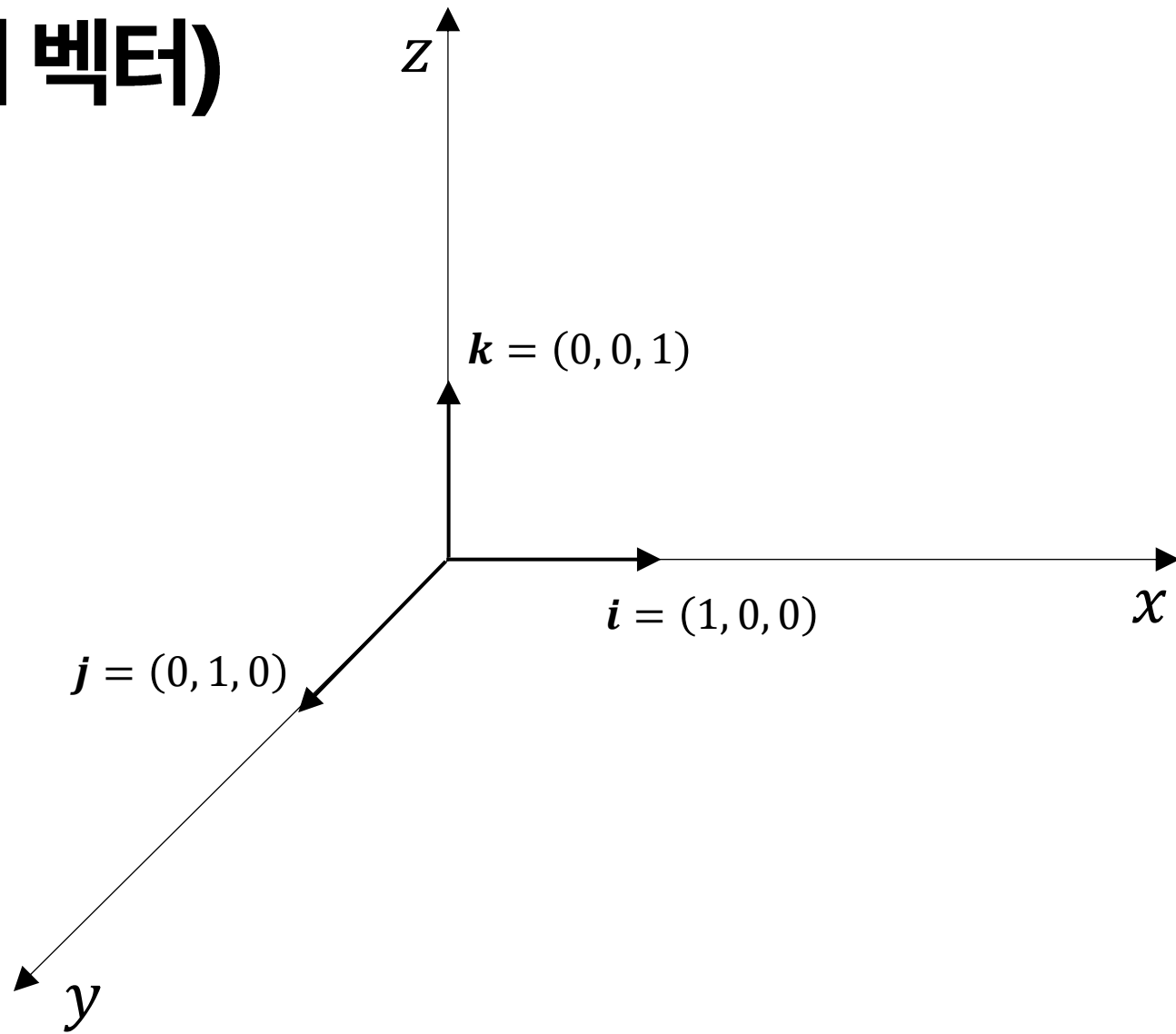
2차원 공간에서는 2개의 좌표값 필요

기저와 차원

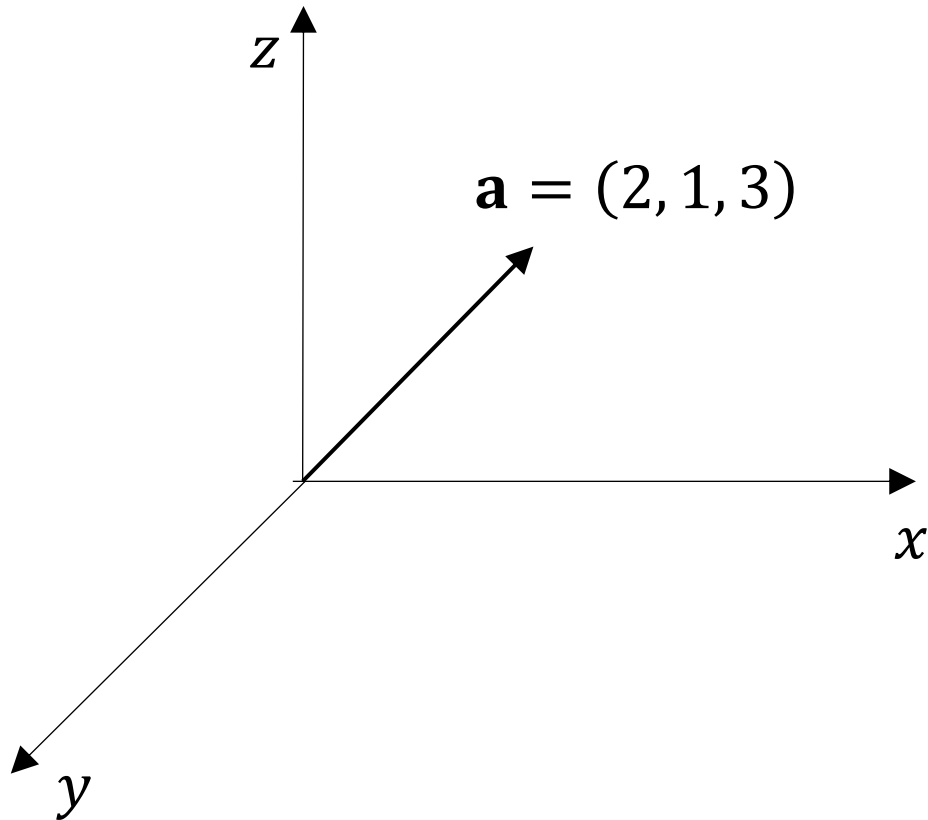


3차원 공간에서는 3개의 좌표값 필요

유닛 벡터(단위 벡터)



선형 조합(linear combination)



$$\mathbf{i} = (1, 0, 0) \quad \mathbf{j} = (0, 1, 0) \quad \mathbf{k} = (0, 0, 1)$$

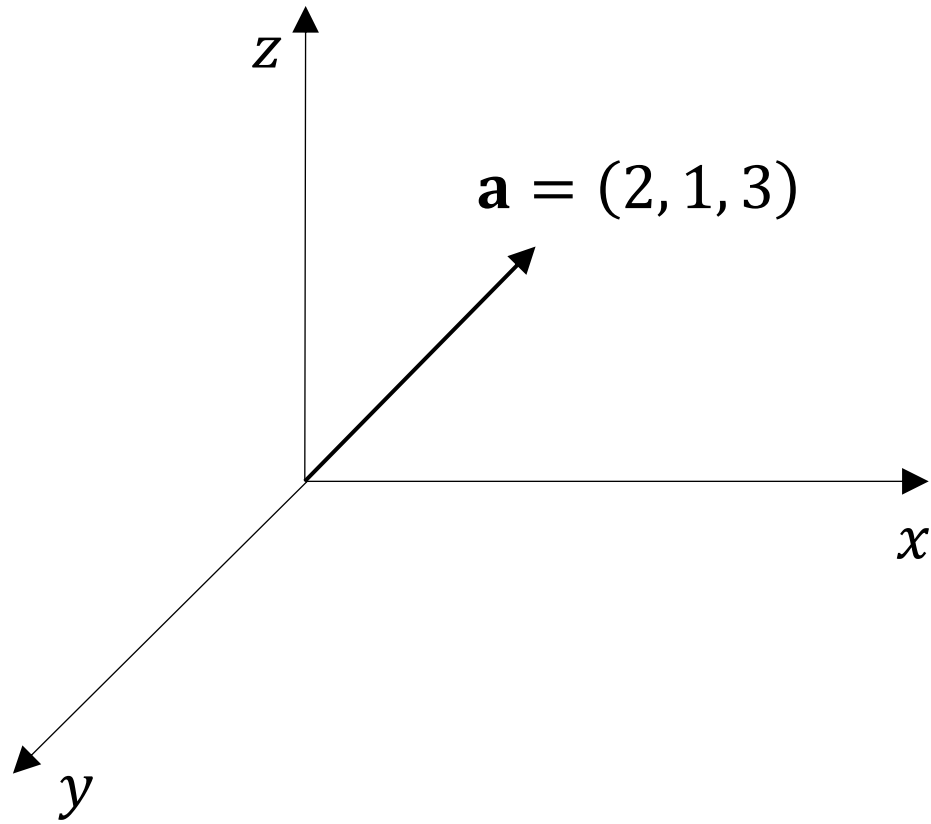
$$\mathbf{a} = (2, 1, 3)$$

$$= (2, 0, 0) + (0, 1, 0) + (0, 0, 3)$$

$$= 2(1, 0, 0) + 1(0, 1, 0) + 3(0, 0, 1)$$

$$= 2\mathbf{i} + \mathbf{j} + 3\mathbf{k}$$

선형 조합(linear combination)

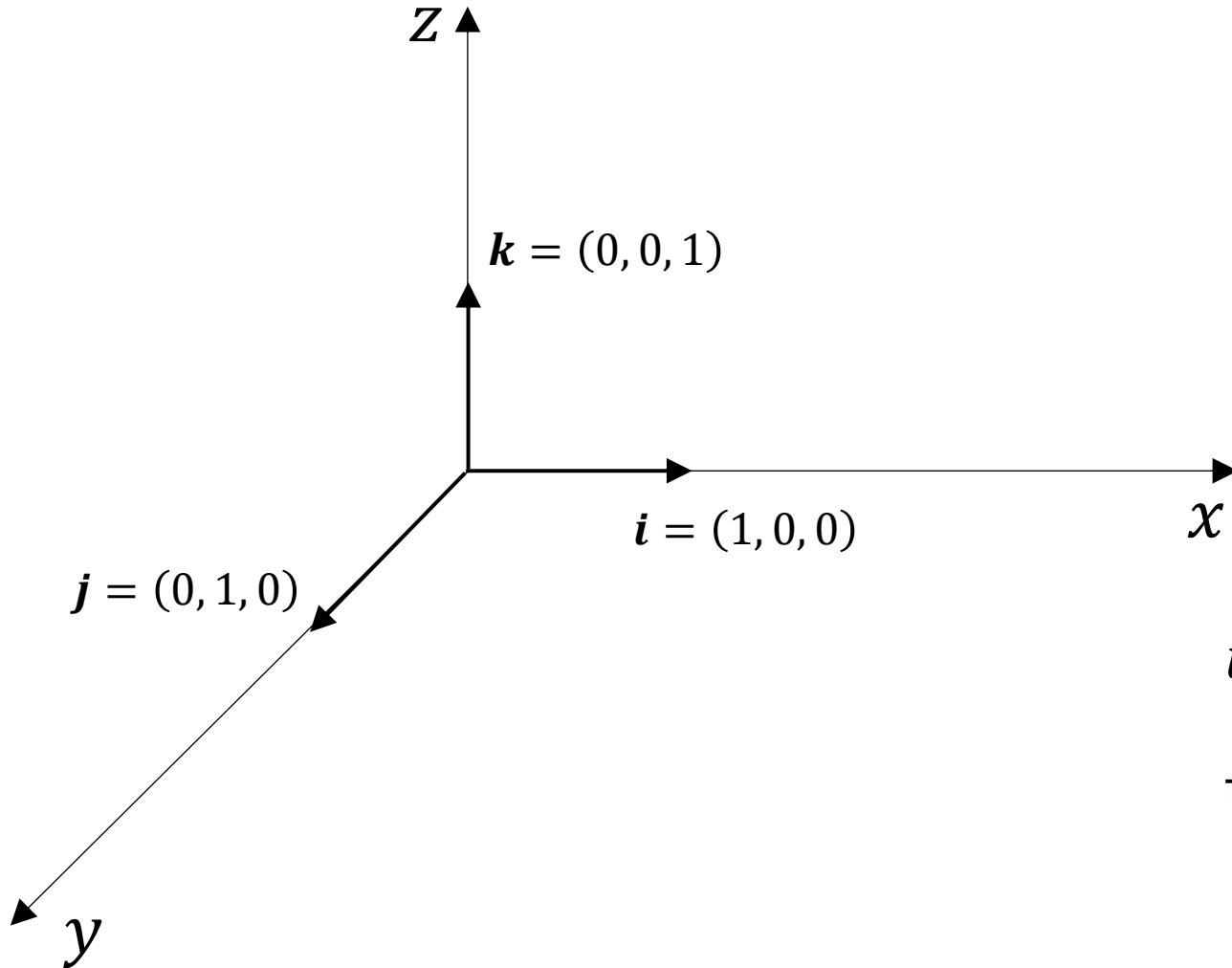


$$\mathbf{i} = (1, 0, 0) \quad \mathbf{j} = (0, 1, 0) \quad \mathbf{k} = (0, 0, 1)$$

$$\mathbf{a} = (2, 1, 3) = 2\mathbf{i} + \mathbf{j} + 3\mathbf{k}$$

$$\mathbf{a} = 2\mathbf{i} + \mathbf{j} + 3\mathbf{k}$$

유닛 벡터(단위 벡터)



$$i = (1, 0, 0) \quad j = (0, 1, 0) \quad k = (0, 0, 1)$$

로 모든 3차원 공간 좌표 표현 가능

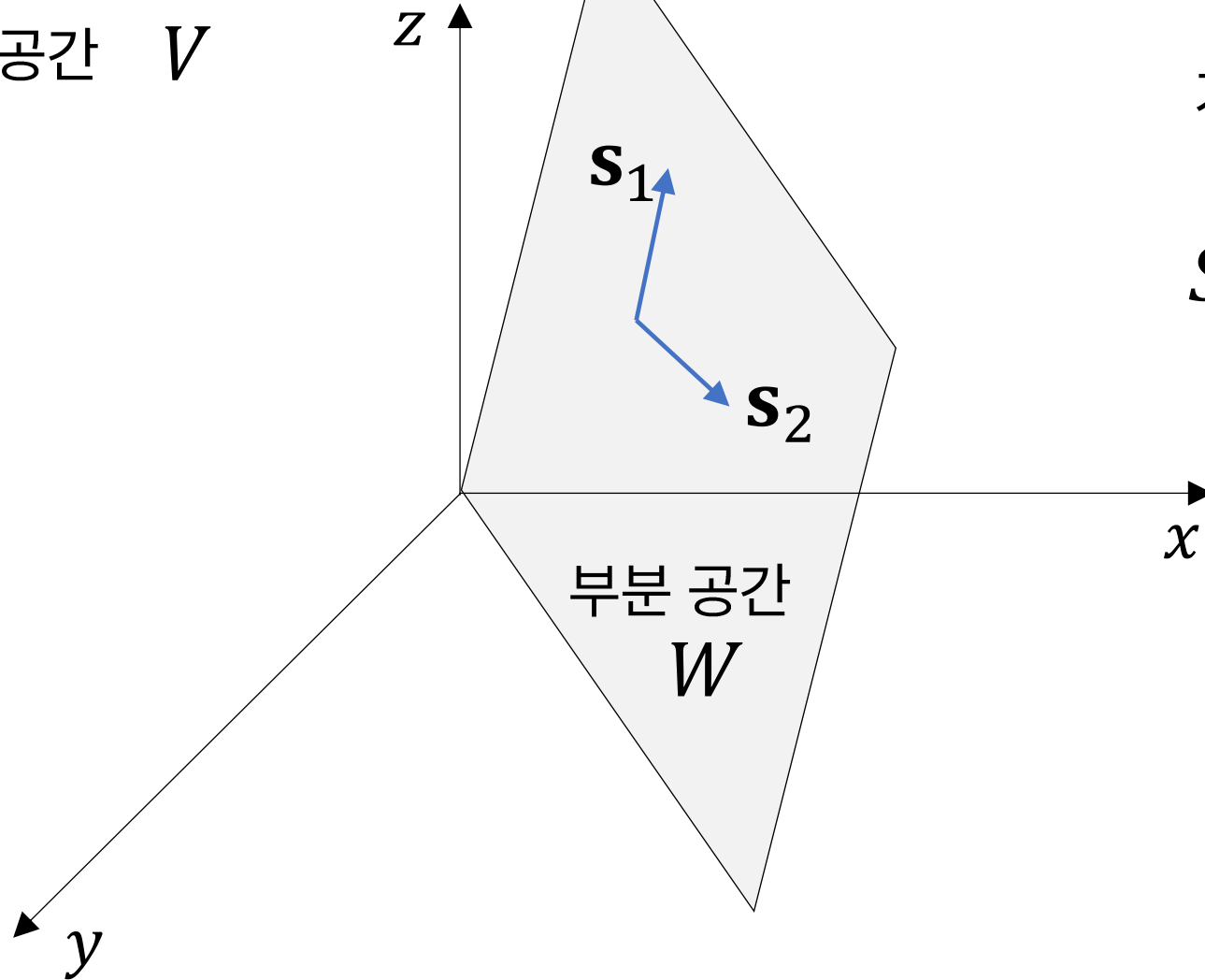


$$i = (1, 0, 0) \quad j = (0, 1, 0) \quad k = (0, 0, 1)$$

는 기저 벡터(basis vector)

부분 공간(subspace)

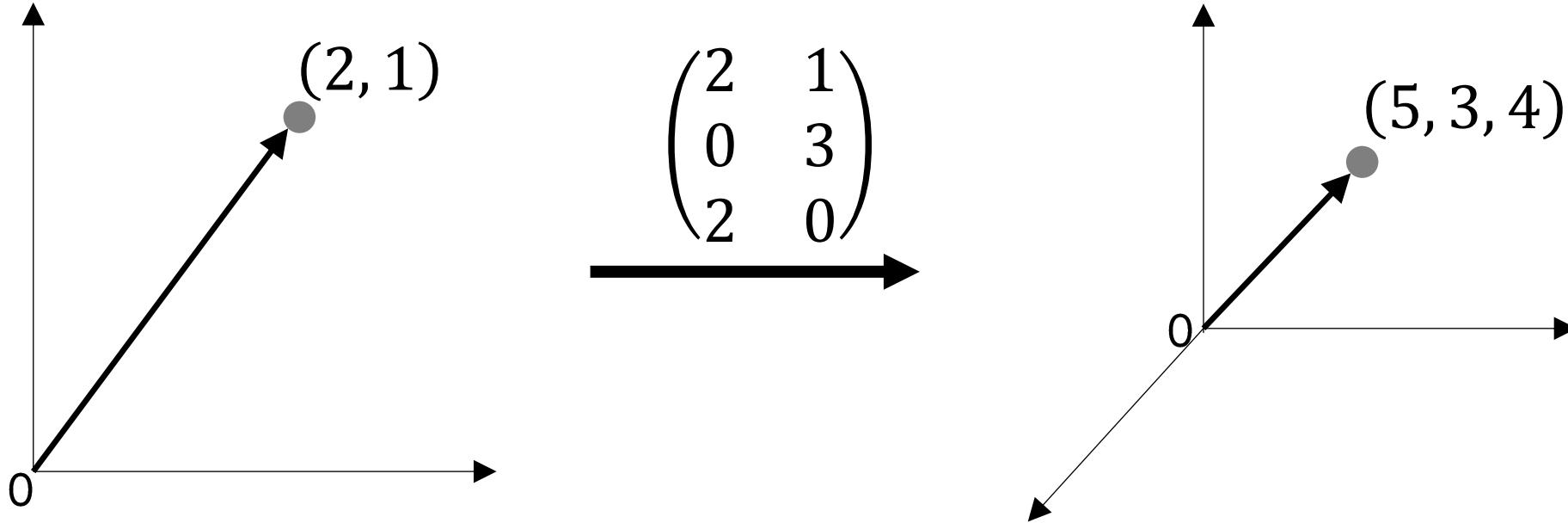
벡터 공간 V



기저 벡터 $S = \{\mathbf{s}_1, \mathbf{s}_2\}$

S 는 부분 공간 W 를 span한다

선형 변환(linear transformation)



2차원 벡터를 3차원 벡터로 선형 변환 시킴

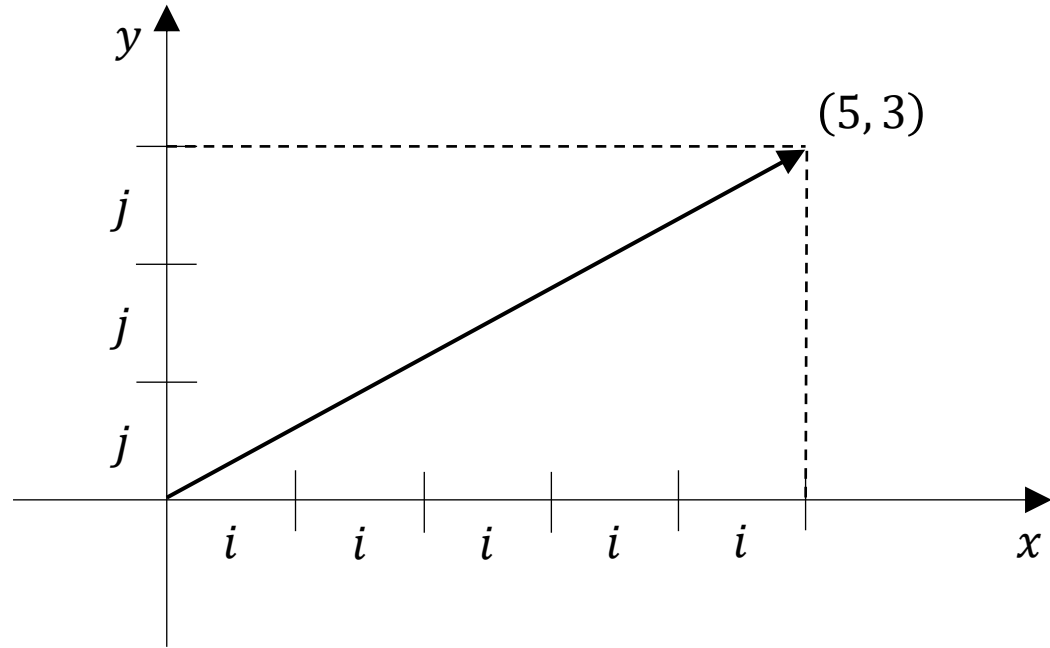
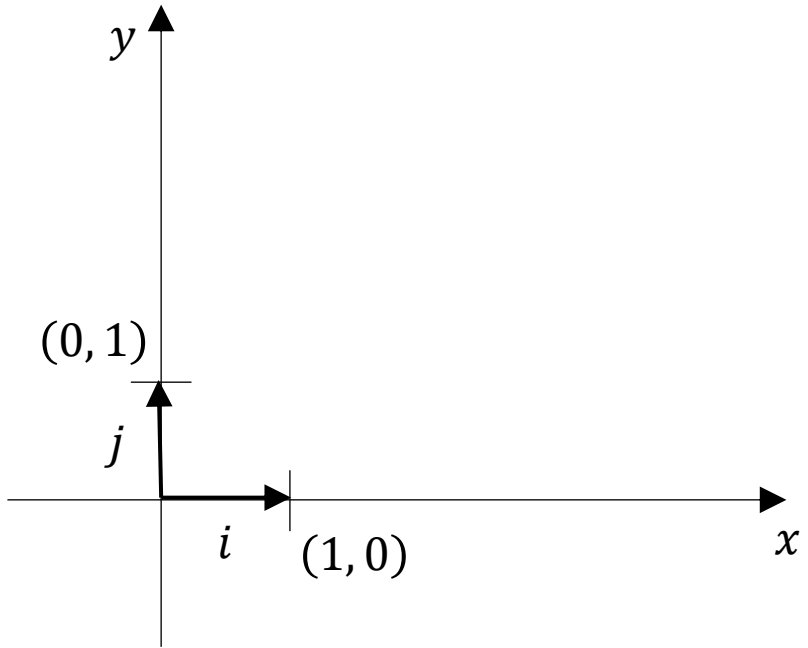
$$A\mathbf{x} = \begin{pmatrix} 2 & 1 \\ 0 & 3 \\ 2 & 0 \end{pmatrix} \begin{pmatrix} 2 \\ 1 \end{pmatrix} = \begin{pmatrix} 5 \\ 3 \\ 4 \end{pmatrix}$$

선형 조합(linear combination)

$$\mathbf{w} = a_1 \mathbf{u}_1 + a_2 \mathbf{u}_2 + \cdots + a_n \mathbf{u}_n$$

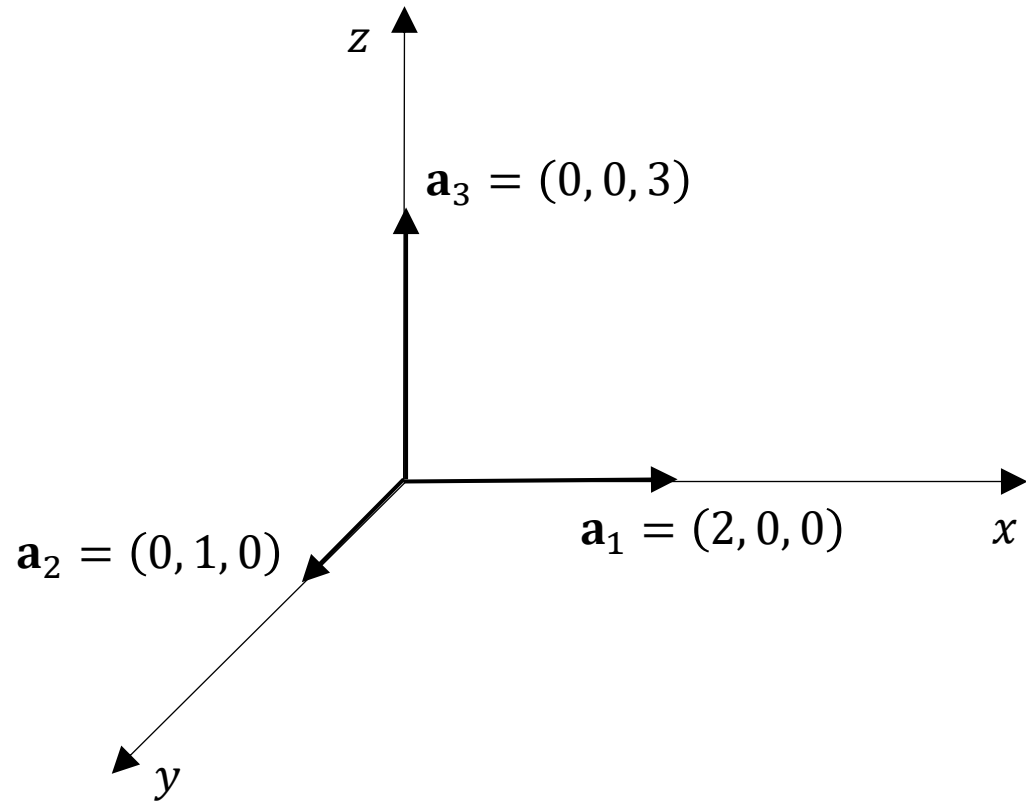
벡터 $\mathbf{w}$ 를 벡터 $\mathbf{u}_1, \mathbf{u}_2, \cdots, \mathbf{u}_n$ 의 선형 조합으로 나타냄

선형 조합(linear combination)



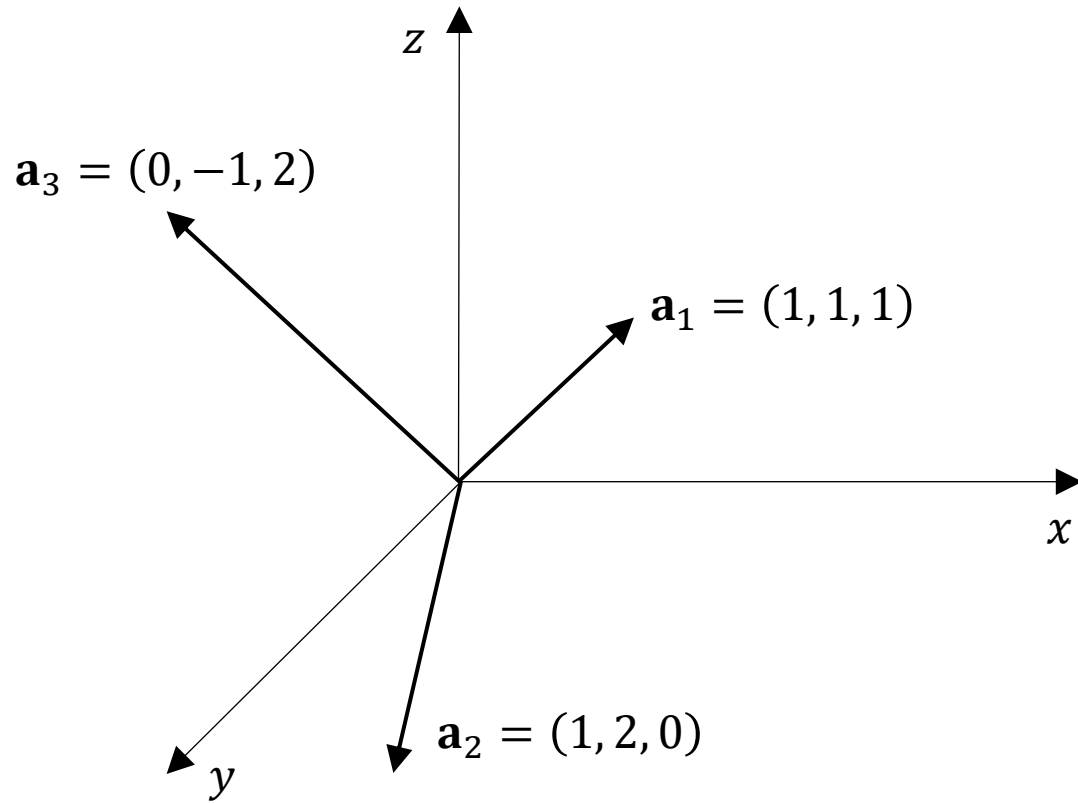
$$\begin{pmatrix} 5 \\ 3 \end{pmatrix} = 5 \begin{pmatrix} 1 \\ 0 \end{pmatrix} + 3 \begin{pmatrix} 0 \\ 1 \end{pmatrix}$$

선형 독립 = 선형 조합으로 표현할 수 없음



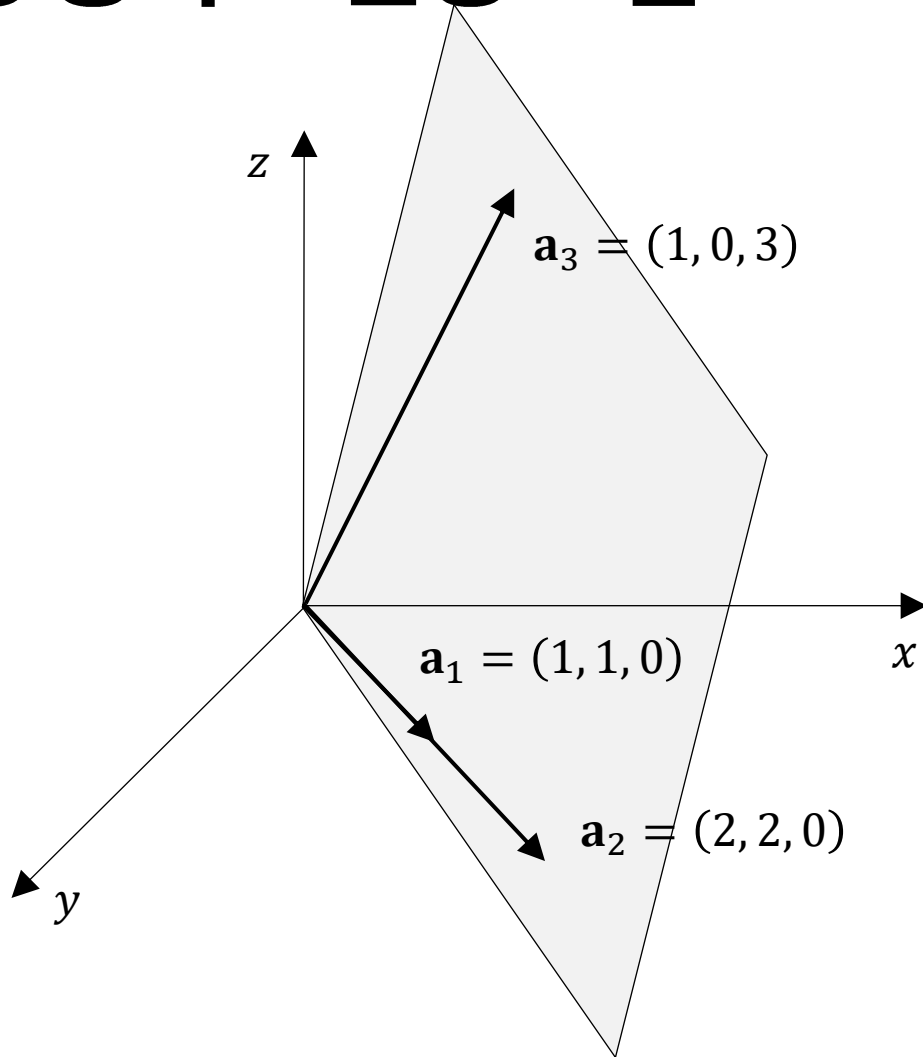
$$\mathbf{a}_1 = \begin{pmatrix} 2 \\ 0 \\ 0 \end{pmatrix}, \quad \mathbf{a}_2 = \begin{pmatrix} 0 \\ 1 \\ 0 \end{pmatrix}, \quad \mathbf{a}_3 = \begin{pmatrix} 0 \\ 0 \\ 3 \end{pmatrix}$$

선형 독립(linear independent)



$$\mathbf{a}_1 = \begin{pmatrix} 1 \\ 1 \\ 1 \end{pmatrix}, \quad \mathbf{a}_2 = \begin{pmatrix} 1 \\ 2 \\ 0 \end{pmatrix}, \quad \mathbf{a}_3 = \begin{pmatrix} 0 \\ -1 \\ 2 \end{pmatrix}$$

선형 종속 = 선형 조합으로 표현할 수 있음



$$\mathbf{a}_1 = \begin{pmatrix} 1 \\ 1 \\ 0 \end{pmatrix}, \quad \mathbf{a}_2 = \begin{pmatrix} 2 \\ 2 \\ 0 \end{pmatrix}, \quad \mathbf{a}_3 = \begin{pmatrix} 1 \\ 0 \\ 3 \end{pmatrix}$$

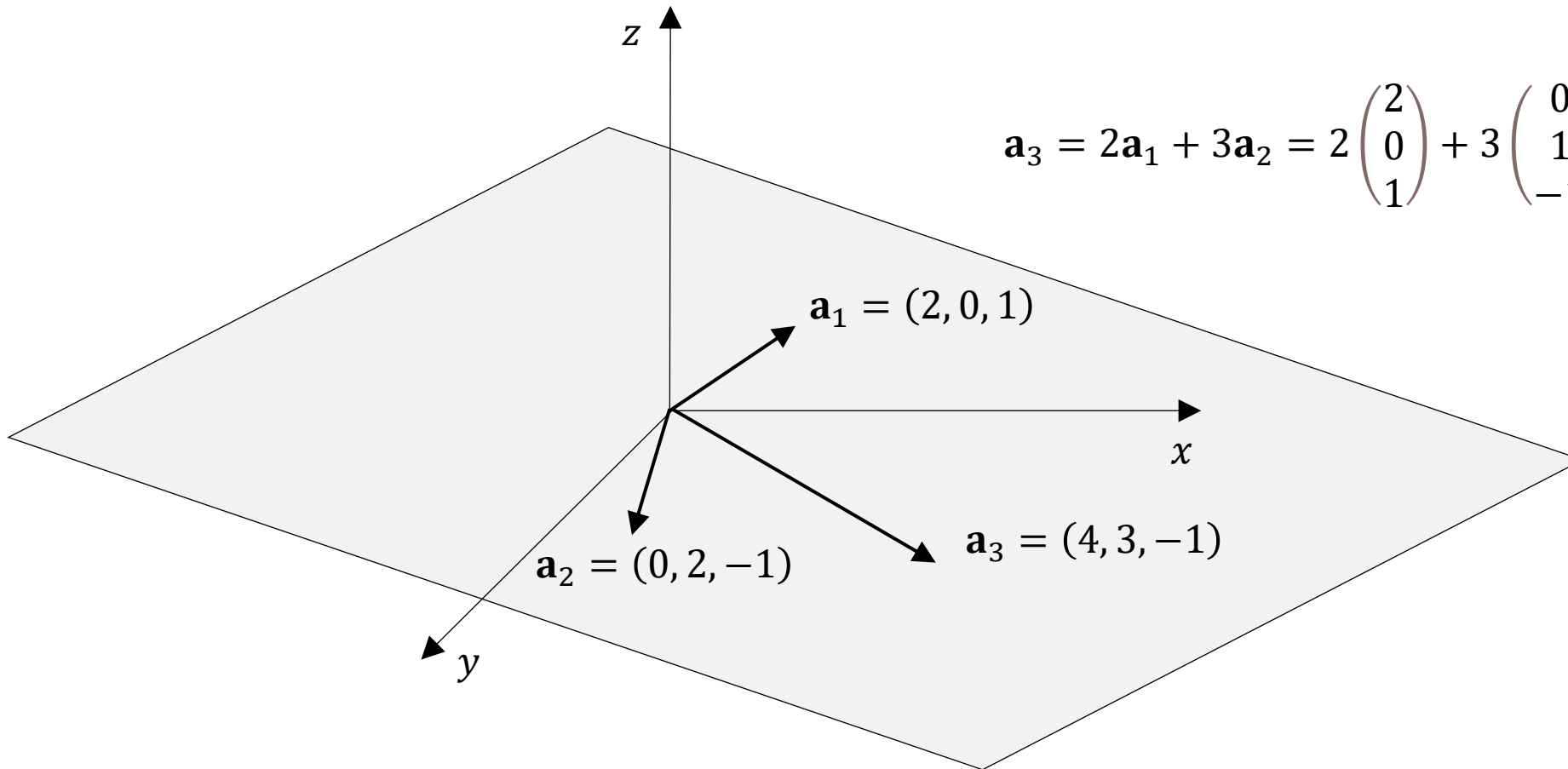
$$\mathbf{a}_2 = \begin{pmatrix} 2 \\ 2 \\ 0 \end{pmatrix} = 2 \begin{pmatrix} 1 \\ 1 \\ 0 \end{pmatrix} = 2\mathbf{a}_1$$

$$\mathbf{a}_2 = 2\mathbf{a}_1$$

선형 종속

$$\mathbf{a}_1 = \begin{pmatrix} 2 \\ 0 \\ 1 \end{pmatrix}, \quad \mathbf{a}_2 = \begin{pmatrix} 0 \\ 1 \\ -1 \end{pmatrix}, \quad \mathbf{a}_3 = \begin{pmatrix} 4 \\ 3 \\ -1 \end{pmatrix}$$

$$\mathbf{a}_3 = 2\mathbf{a}_1 + 3\mathbf{a}_2 = 2\begin{pmatrix} 2 \\ 0 \\ 1 \end{pmatrix} + 3\begin{pmatrix} 0 \\ 1 \\ -1 \end{pmatrix} = \begin{pmatrix} 4 \\ 0 \\ 2 \end{pmatrix} + \begin{pmatrix} 0 \\ 3 \\ -3 \end{pmatrix} = \begin{pmatrix} 4 \\ 3 \\ -1 \end{pmatrix}$$



기저(basis)

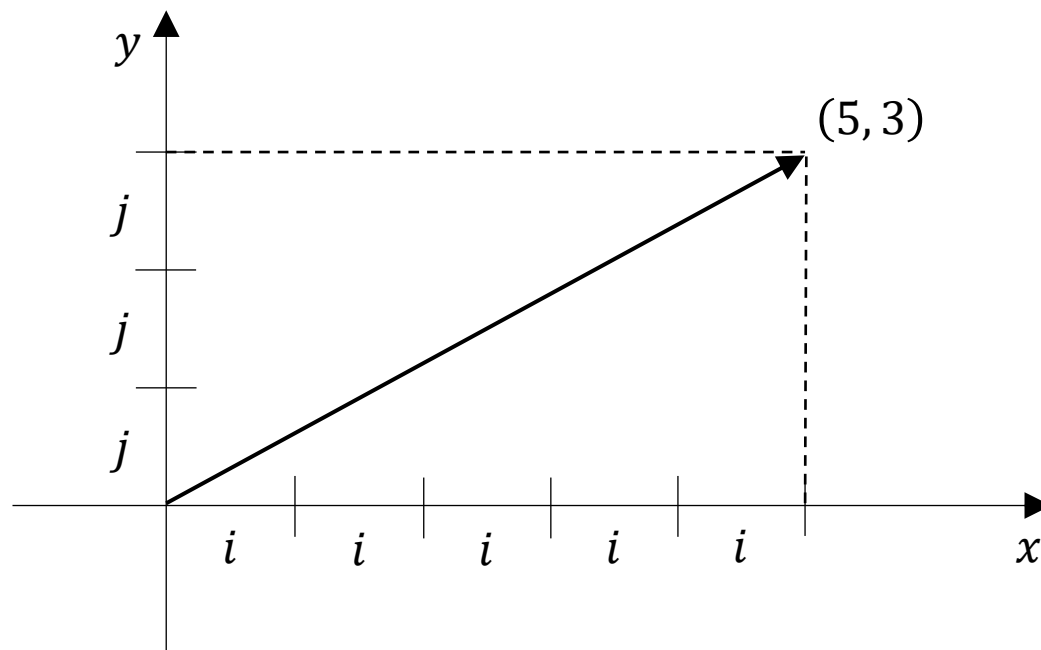
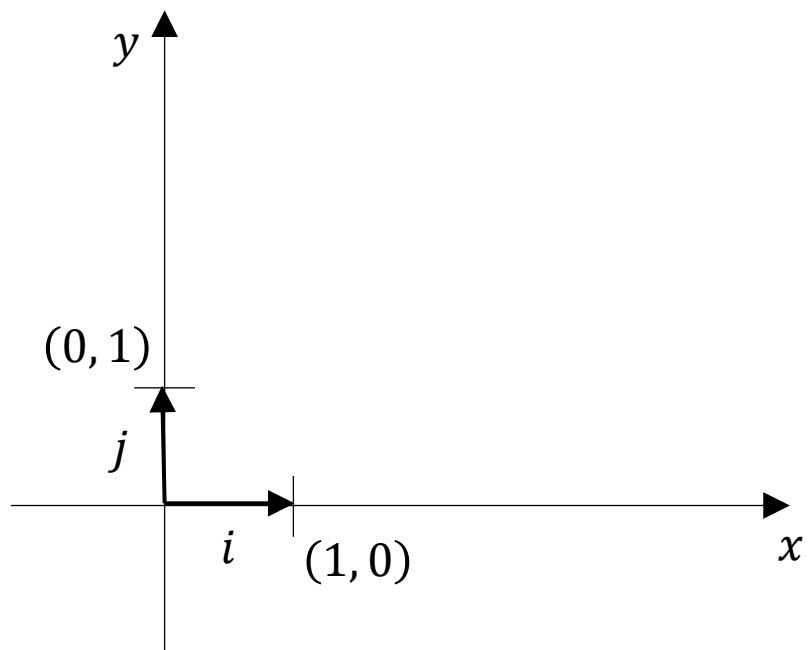
기저(basis): 벡터 공간을 생성하는 선형 독립인 벡터들

기저의 조건

(1) $\{s_1, s_2, \dots, s_n\}$ 가 공간 S 를 생성합니다.

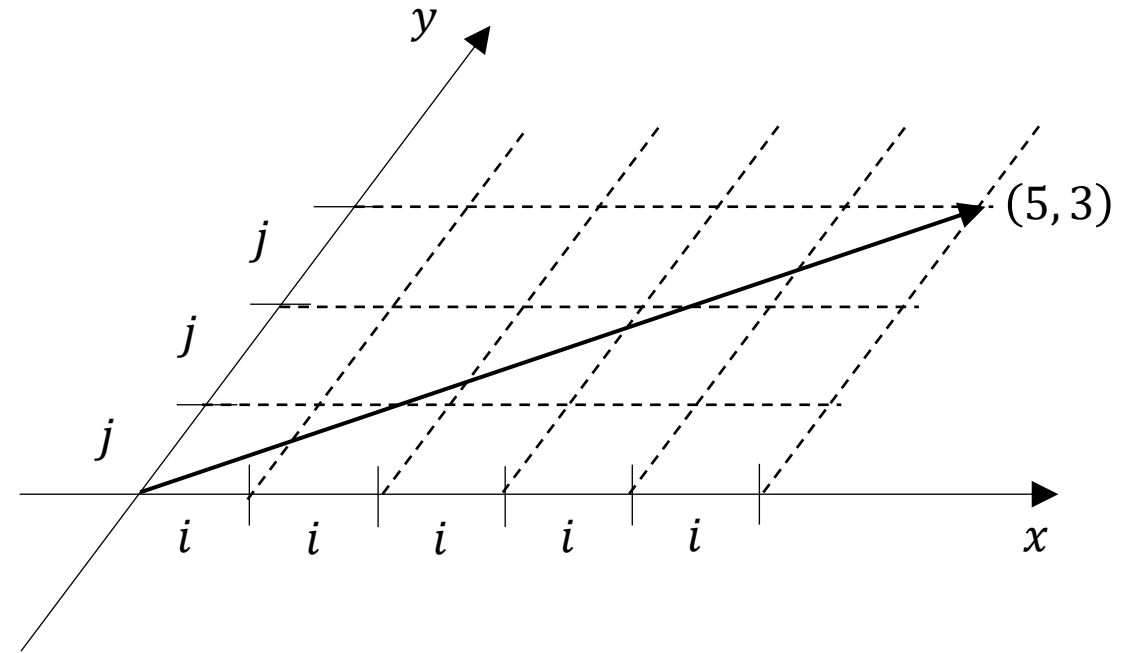
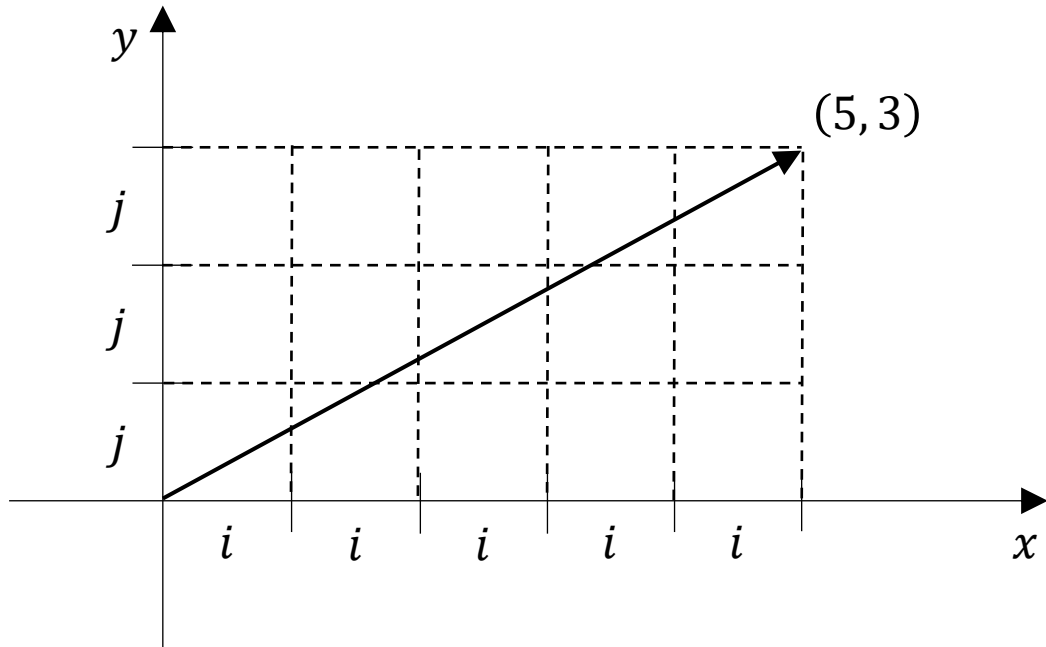
(2) $s_1, s_2, \dots, s_n$ 는 선형 독립(linearly independent)

기저와 차원



$$\begin{pmatrix} 5 \\ 3 \end{pmatrix} = 5 \begin{pmatrix} 1 \\ 0 \end{pmatrix} + 3 \begin{pmatrix} 0 \\ 1 \end{pmatrix}$$

기저는 항상 직교해야할까?

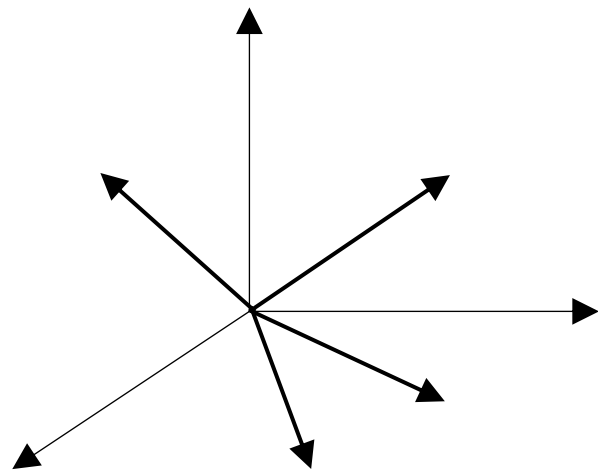


- 기저가 항상 직교일 필요는 없습니다.
- 기저는 유일하지 않습니다.

기저의 성질

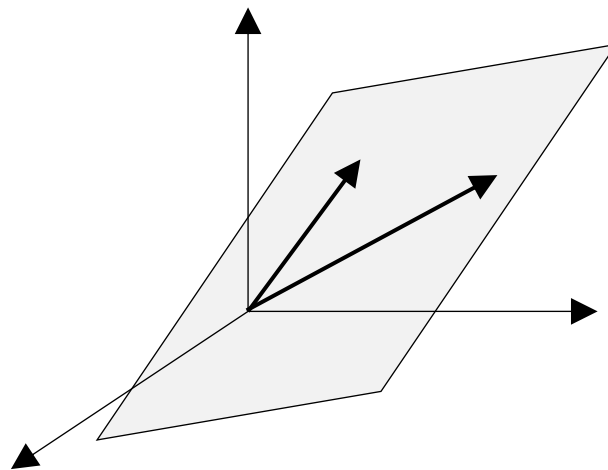
n 차원 벡터 공간 S 에서

- (a) n 개가 넘는 벡터가 만드는 집합은 선형 종속입니다.
- (b) n 개 미만의 벡터가 만드는 집합은 벡터 공간 S 를 생성할 수 없습니다.
- (c) 벡터 공간의 모든 기저 벡터의 개수는 동일하다.



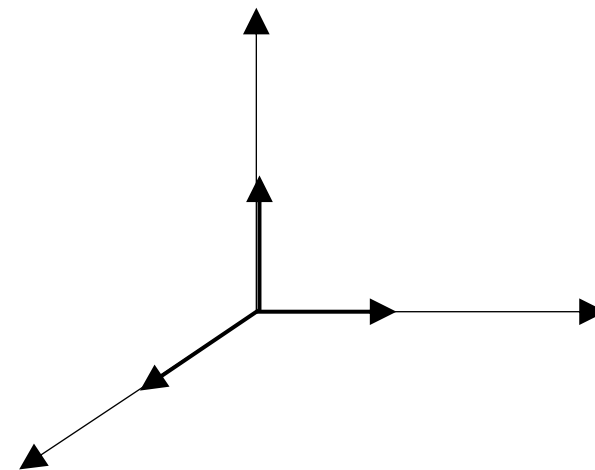
(a)

n 개가 넘는 벡터가
만드는 집합은 선형 종속



(b)

n 개 미만의 벡터가 만드는
집합은 벡터 공간 S 를
생성할 수 없습니다.



(c)

벡터 공간의
모든 기저 벡터의
개수는 동일하다.

차원과 랭크

차원(dimension): 해당 공간을 구성하는 기저 벡터의 개수

랭크(rank): 행 공간과 열 공간의 공통 차원

행 공간과 열 공간

$$A = \begin{pmatrix} a_{11} & a_{12} & \cdots & a_{1p} \\ a_{21} & a_{22} & \cdots & a_{2p} \\ \vdots & \vdots & \ddots & \vdots \\ a_{n1} & a_{n2} & \cdots & a_{np} \end{pmatrix}$$

$$\begin{pmatrix} a_{11} & a_{12} & \cdots & a_{1p} \\ a_{21} & a_{22} & \cdots & a_{2p} \\ \vdots & \vdots & \ddots & \vdots \\ a_{n1} & a_{n2} & \cdots & a_{np} \end{pmatrix} \rightarrow \begin{pmatrix} (a_{11}, a_{12}, \cdots, a_{1p}) \\ (a_{21}, a_{22}, \cdots, a_{2p}) \\ \vdots \\ (a_{n1}, a_{n2}, \cdots, a_{np}) \end{pmatrix}$$

행 공간(row spaces)

행 벡터로 span할 수 있는 공간

$$\begin{pmatrix} a_{11} & a_{12} & \cdots & a_{1p} \\ a_{21} & a_{22} & \cdots & a_{2p} \\ \vdots & \vdots & \ddots & \vdots \\ a_{n1} & a_{n2} & \cdots & a_{np} \end{pmatrix} \rightarrow \begin{pmatrix} a_{11} \\ a_{21} \\ \vdots \\ a_{n1} \end{pmatrix}, \begin{pmatrix} a_{12} \\ a_{22} \\ \vdots \\ a_{n2} \end{pmatrix}, \cdots, \begin{pmatrix} a_{1p} \\ a_{2p} \\ \vdots \\ a_{np} \end{pmatrix}$$

열 공간(column spaces)

열 벡터로 span할 수 있는 공간

행 공간과 열 공간

$$A = \begin{pmatrix} 1 & 5 \\ 3 & 4 \\ 5 & 2 \end{pmatrix}$$

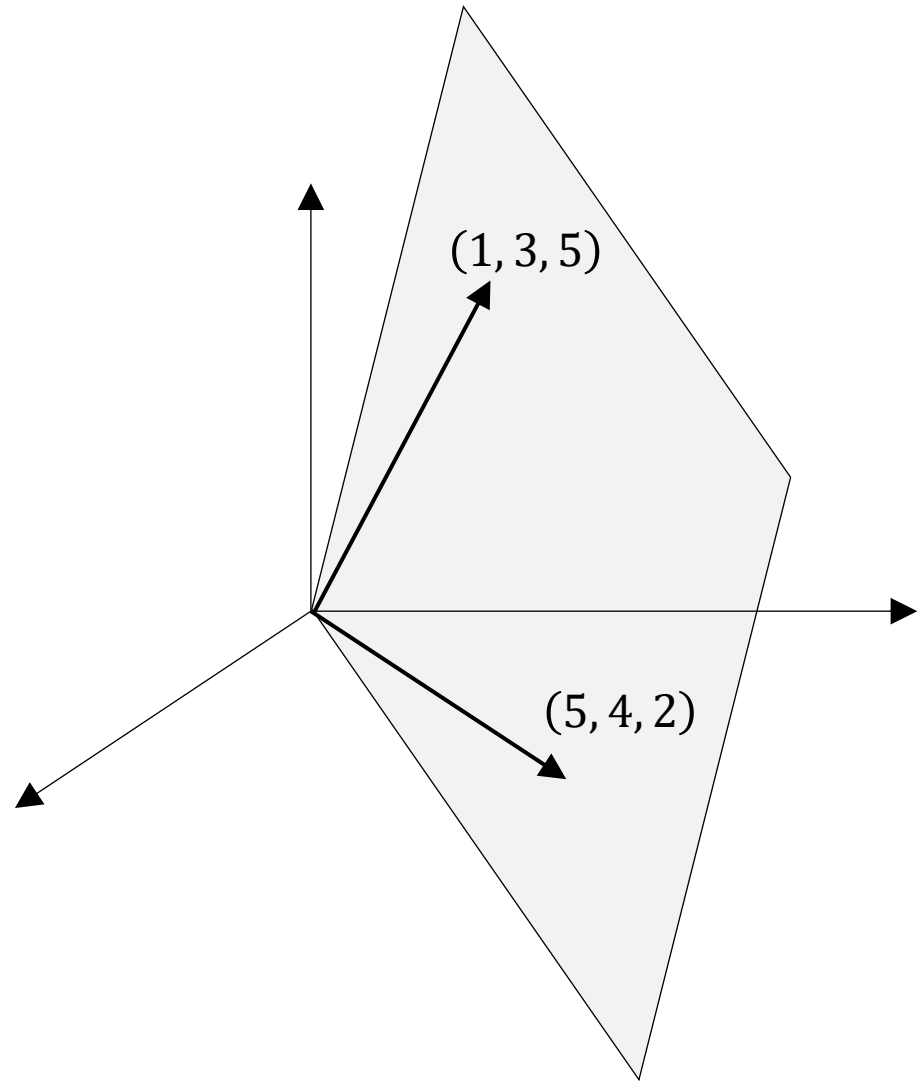
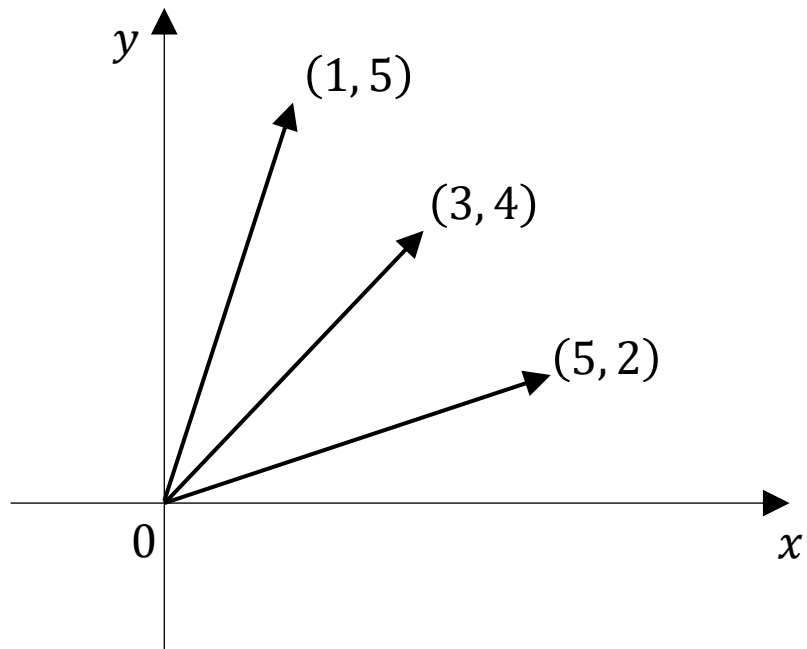
$$\begin{pmatrix} 1 & 5 \end{pmatrix}$$

$$\begin{pmatrix} 3 & 4 \end{pmatrix}$$

$$\begin{pmatrix} 5 & 2 \end{pmatrix}$$

$$\begin{pmatrix} 1 \\ 3 \\ 5 \end{pmatrix}, \quad \begin{pmatrix} 5 \\ 4 \\ 2 \end{pmatrix}$$

행 공간과 열 공간



영 공간(null space): $\mathbf{Ax} = \mathbf{0}$ 을 만족하는 모든 벡터 $\mathbf{x}$ 의 집합

널리티(nullity): 영 공간의 차원

기본 행 연산(elementary row operation)

(a) 한 행에 영이 아닌 상수를 모두 곱한다.

(b) 두 행을 교환 한다.

(c) 한 행의 배수를 다른 행에 더한다.

기본 행 연산

(a) 한 행에 영이 아닌 상수를 모두 곱한다.

$$\begin{pmatrix} 1 & 2 & 3 & 4 \\ 2 & 3 & 4 & 5 \\ 0 & 1 & 2 & 1 \\ 3 & 2 & 0 & 1 \end{pmatrix}$$

$$\begin{pmatrix} 1 & 2 & 3 & 4 \\ 2 \times 3 & 3 \times 3 & 4 \times 3 & 5 \times 3 \\ 0 & 1 & 2 & 1 \\ 3 & 2 & 0 & 1 \end{pmatrix} = \begin{pmatrix} 1 & 2 & 3 & 4 \\ 6 & 9 & 12 & 15 \\ 0 & 1 & 2 & 1 \\ 3 & 2 & 0 & 1 \end{pmatrix}$$

기본 행 연산

(b) 두 행을 교환 한다.

$$\begin{pmatrix} 1 & 2 & 3 & 4 \\ 6 & 9 & 12 & 15 \\ 0 & 1 & 2 & 1 \\ 3 & 2 & 0 & 1 \end{pmatrix} \rightarrow \begin{pmatrix} 6 & 9 & 12 & 15 \\ 1 & 2 & 3 & 4 \\ 0 & 1 & 2 & 1 \\ 3 & 2 & 0 & 1 \end{pmatrix}$$

기본 행 연산

(c) 한 행의 배수를 다른 행에 더한다.

$$\begin{pmatrix} 1 & 2 & 3 & 4 \\ 2 & 3 & 4 & 5 \\ 0 & 1 & 2 & 1 \\ 3 & 2 & 0 & 1 \end{pmatrix}$$

$$\begin{pmatrix} 1 & 2 & 3 & 4 \\ 2 & 3 & 4 & 5 \\ 0 & 1 & 2 & 1 \\ 3 + (1 \times -3) & 2 + (2 \times -3) & 0 + (3 \times -3) & 1 + (4 \times -3) \end{pmatrix}$$

행 공간과 열 공간의 성질

- (a) 기본 행 연산은 행렬의 열 공간을 변화시키지 않습니다.
- (b) 기본 행 연산은 행렬의 행 공간을 변화시키지 않습니다.
- (c) 행렬 A 의 행 공간과 열 공간의 차원은 동일합니다.

행 공간과 열 공간의 성질

- (a) 기본 행 연산은 행렬의 열 공간을 변화시키지 않습니다.
- (b) 기본 행 연산은 행렬의 행 공간을 변화시키지 않습니다.
- (c) 행렬 A 의 행 공간과 열 공간의 차원은 동일합니다.

$$A = \begin{pmatrix} \boxed{r_1} \\ \boxed{r_2} \\ \boxed{r_3} \\ \boxed{r_4} \\ \boxed{r_5} \end{pmatrix}$$

$$B = \begin{pmatrix} \boxed{r_2} \\ \boxed{r_1} \\ \boxed{r_3} \\ \boxed{r_4} \\ \boxed{r_5} \end{pmatrix}$$

행 공간과 열 공간의 성질

- (a) 기본 행 연산은 행렬의 열 공간을 변화시키지 않습니다.
- (b) 기본 행 연산은 행렬의 행 공간을 변화시키지 않습니다.
- (c) 행렬 A 의 행 공간과 열 공간의 차원은 동일합니다.

$$A = \begin{pmatrix} r_1 \\ r_2 \\ r_3 \\ r_4 \\ r_5 \end{pmatrix}$$

$$B = \begin{pmatrix} r_1 \\ r_2 \\ r_3 \\ r_4 \\ r_5 + 3r_1 \end{pmatrix}$$

행 공간과 열 공간의 성질

(c) 행렬 A 의 행 공간과 열 공간의 차원은 동일합니다.

x1	x2	x3	x4	x5

우리가 다루는 직사각 행렬 형태의 데이터들 또한

행 공간과 열 공간의 차원은 동일합니다.

AI/BigData 인텐시브 과정

Section 1. 선형대수

Section 1-7. 고윳값과 고유벡터

고윳값(eigenvalue)과 고유 벡터(eigen vector)

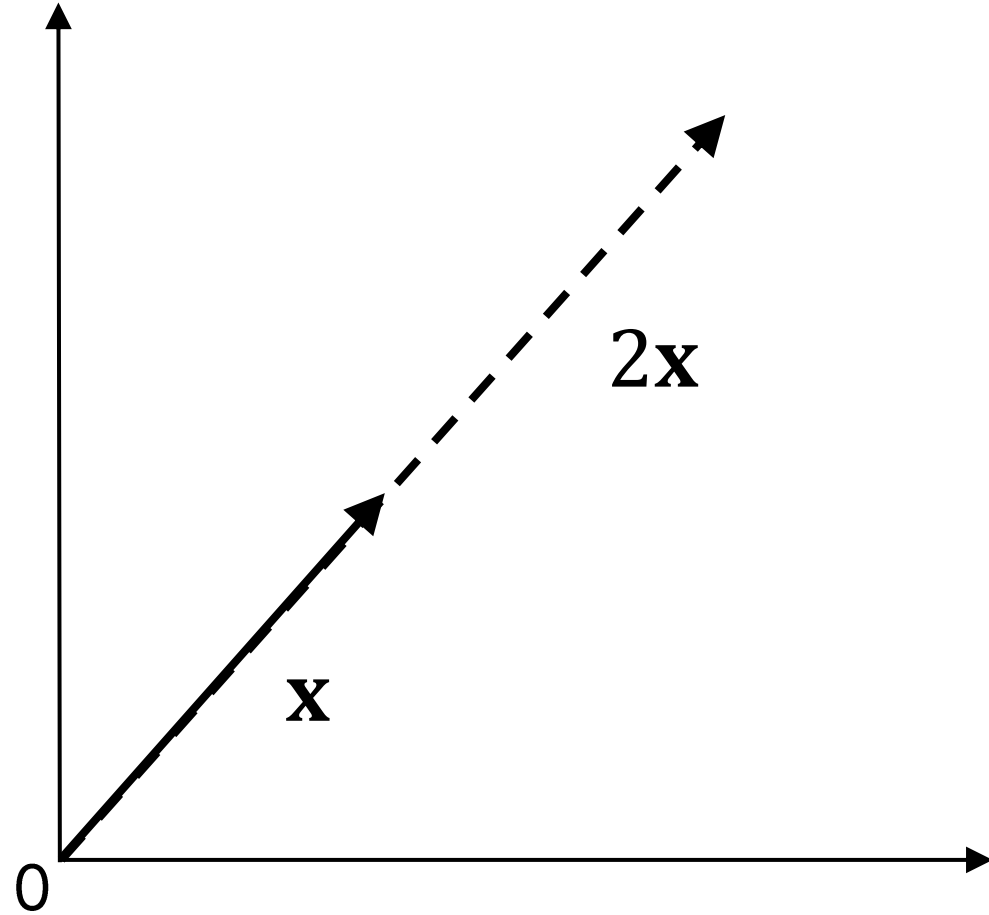
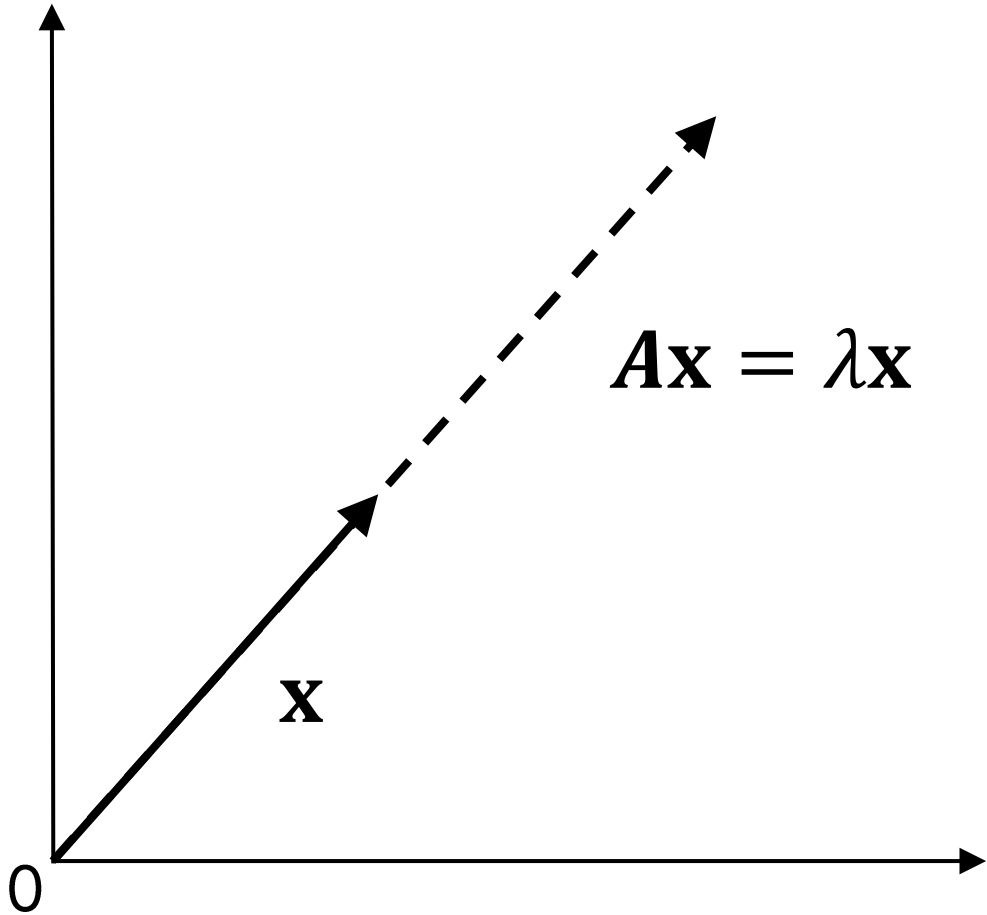
The diagram illustrates the eigenvalue equation $Ax = \lambda x$. The matrix A and the vector x on the left are enclosed in light blue ovals. The scalar λ on the right is enclosed in a light red oval. An arrow points from the red oval to the Korean text '고윳값' (eigenvalue) above it. Two arrows point from the blue ovals to the Korean text '고유 벡터' (eigenvector) below them.

$$Ax = \lambda x$$

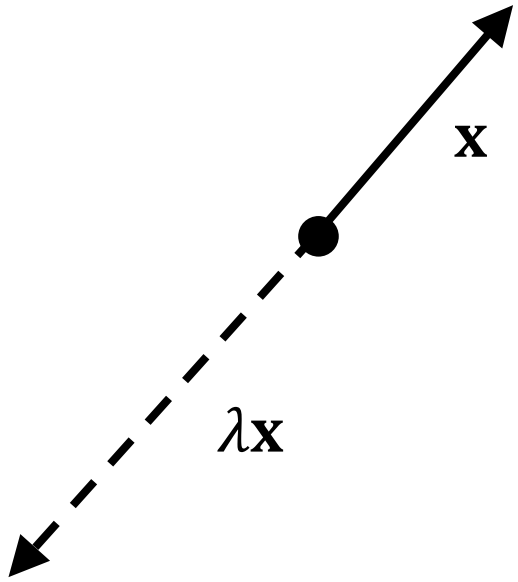
고윳값

고유 벡터

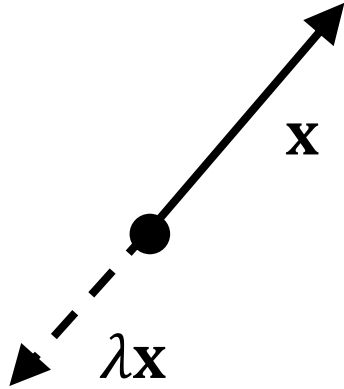
고윳값(eigenvalue)과 고유 벡터(eigen vector)



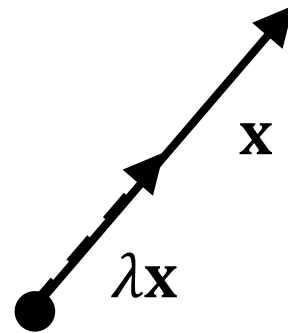
고윳값(eigenvalue)과 고유 벡터(eigen vector)



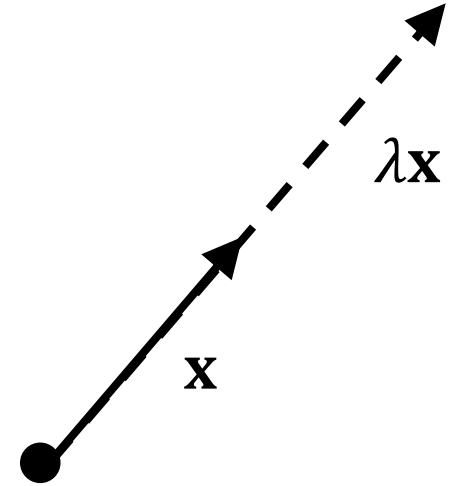
$$\lambda \leq -1$$



$$-1 \leq \lambda \leq 0$$



$$0 \leq \lambda \leq 1$$



$$\lambda \geq 1$$

고윳값과 고유 벡터

$$A\mathbf{x} = \lambda\mathbf{x}$$

$$A\mathbf{x} = \lambda\mathbf{x}$$

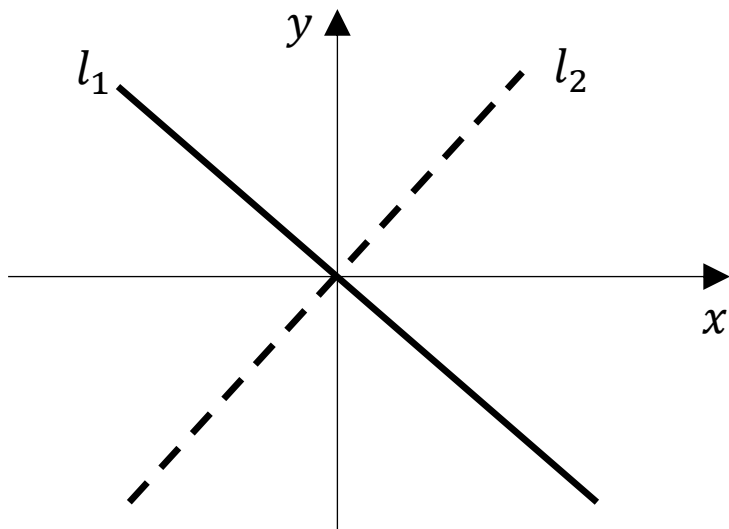
$$\Leftrightarrow A\mathbf{x} - \lambda\mathbf{x} = 0$$

$$\Leftrightarrow (A - \lambda I)\mathbf{x} = 0$$

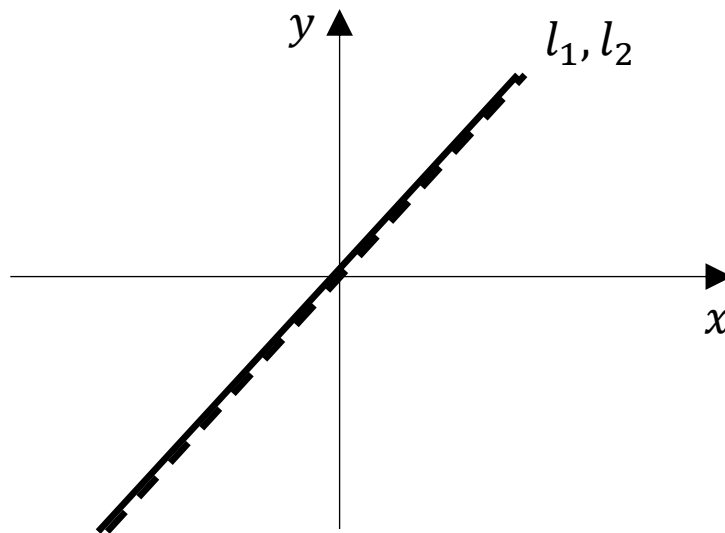
$$\det(A - \lambda I) = 0$$

Why?

동차 선형 시스템



(a) 오직 하나의 해



(b) 무한개의 해

$$A = \begin{pmatrix} 3 & 0 \\ 8 & -1 \end{pmatrix}$$

$$\begin{aligned} A - \lambda I &= \begin{pmatrix} 3 & 0 \\ 8 & -1 \end{pmatrix} - \begin{pmatrix} \lambda & 0 \\ 0 & \lambda \end{pmatrix} \\ &= \begin{pmatrix} 3 - \lambda & 0 \\ 8 & -1 - \lambda \end{pmatrix} \end{aligned}$$

$$\begin{aligned} \det(A - \lambda I) &= \begin{vmatrix} 3 - \lambda & 0 \\ 8 & -1 - \lambda \end{vmatrix} \\ &= (3 - \lambda)(-1 - \lambda) - 0 \times 8 \\ &= -3 - 3\lambda + \lambda + \lambda^2 = \lambda^2 - 2\lambda - 3 \\ &= (\lambda - 3)(\lambda + 1) \end{aligned}$$

$$\therefore \lambda = 3, \quad \lambda = -1$$

(i) $\lambda = 3$ 인 경우

$$(A - \lambda I)\mathbf{x} = 0$$

$$(A - 3I)\mathbf{x} = 0$$

$$A - 3I$$

$$= \begin{pmatrix} 3 & 0 \\ 8 & -1 \end{pmatrix} - \begin{pmatrix} 3 & 0 \\ 0 & 3 \end{pmatrix}$$

$$= \begin{pmatrix} 3-3 & 0 \\ 8 & -1-3 \end{pmatrix}$$

$$= \begin{pmatrix} 0 & 0 \\ 8 & -4 \end{pmatrix}$$

$$\begin{pmatrix} 0 & 0 \\ 8 & -4 \end{pmatrix} \begin{pmatrix} x_1 \\ x_2 \end{pmatrix} = \begin{pmatrix} 0 \\ 0 \end{pmatrix}$$

$$8x_1 - 4x_2 = 0$$

$$x_1 = s$$

$$8s - 4x_2 = 0$$

$$\Leftrightarrow 8s = 4x_2$$

$$\Leftrightarrow x_2 = 2s$$

$$\therefore x_1 = s, \quad x_2 = 2s$$

(i) $\lambda = 3$ 인 경우

$$\mathbf{x} = \begin{pmatrix} x_1 \\ x_2 \end{pmatrix} = \begin{pmatrix} s \\ 2s \end{pmatrix} = \begin{pmatrix} 1 \\ 2 \end{pmatrix} s$$

$$\mathbf{x} = \begin{pmatrix} 1 \\ 2 \end{pmatrix}$$

$$\mathbf{x} = \begin{pmatrix} 1/\sqrt{5} \\ 2/\sqrt{5} \end{pmatrix}$$

(ii) $\lambda = -1$ 인 경우

$$(A - \lambda I)\mathbf{x} = 0$$

$$A - (-1I) = \begin{pmatrix} 3 & 0 \\ 8 & -1 \end{pmatrix} - \begin{pmatrix} -1 & 0 \\ 0 & -1 \end{pmatrix} = \begin{pmatrix} 3 + 1 & 0 \\ 8 & -1 + 1 \end{pmatrix} = \begin{pmatrix} 4 & 0 \\ 8 & 0 \end{pmatrix}$$

$$(A + I)\mathbf{x} = 0$$

$$\begin{pmatrix} 4 & 0 \\ 8 & 0 \end{pmatrix} \begin{pmatrix} x_1 \\ x_2 \end{pmatrix} = \begin{pmatrix} 0 \\ 0 \end{pmatrix}$$

$$x_1 = 0$$

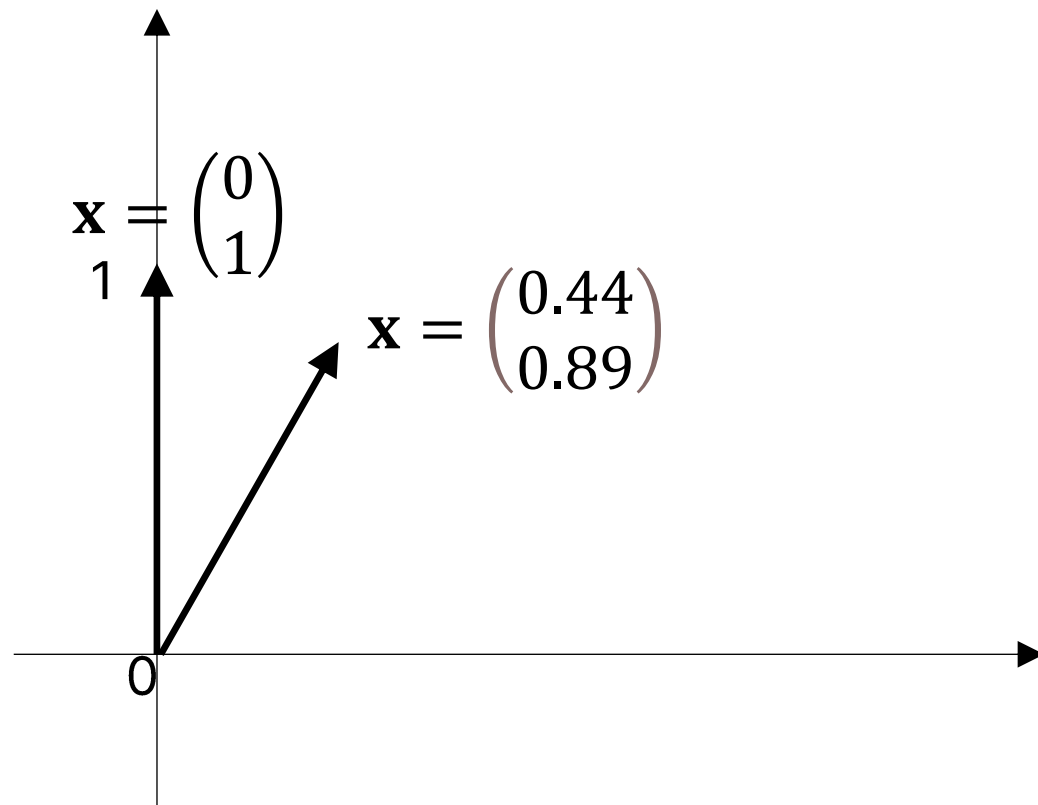
$$x_2 = s$$

(i) $\lambda = 3$ 인 경우

$$\mathbf{x} = \begin{pmatrix} 1/\sqrt{5} \\ 2/\sqrt{5} \end{pmatrix} = \begin{pmatrix} 0.44 \\ 0.89 \end{pmatrix}$$

(ii) $\lambda = -1$ 인 경우

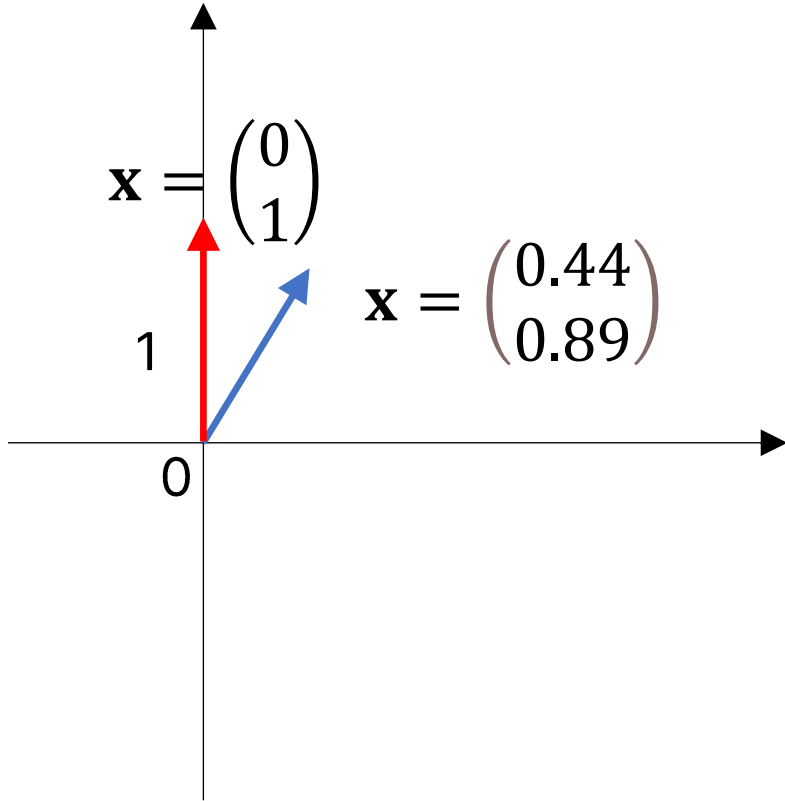
$$\mathbf{x} = \begin{pmatrix} 0 \\ 1 \end{pmatrix}$$



- 고유 벡터는 반드시 서로 직교하는 것은 아니다.
- 단, 대칭 행렬의 고유 벡터는 서로 직교한다.

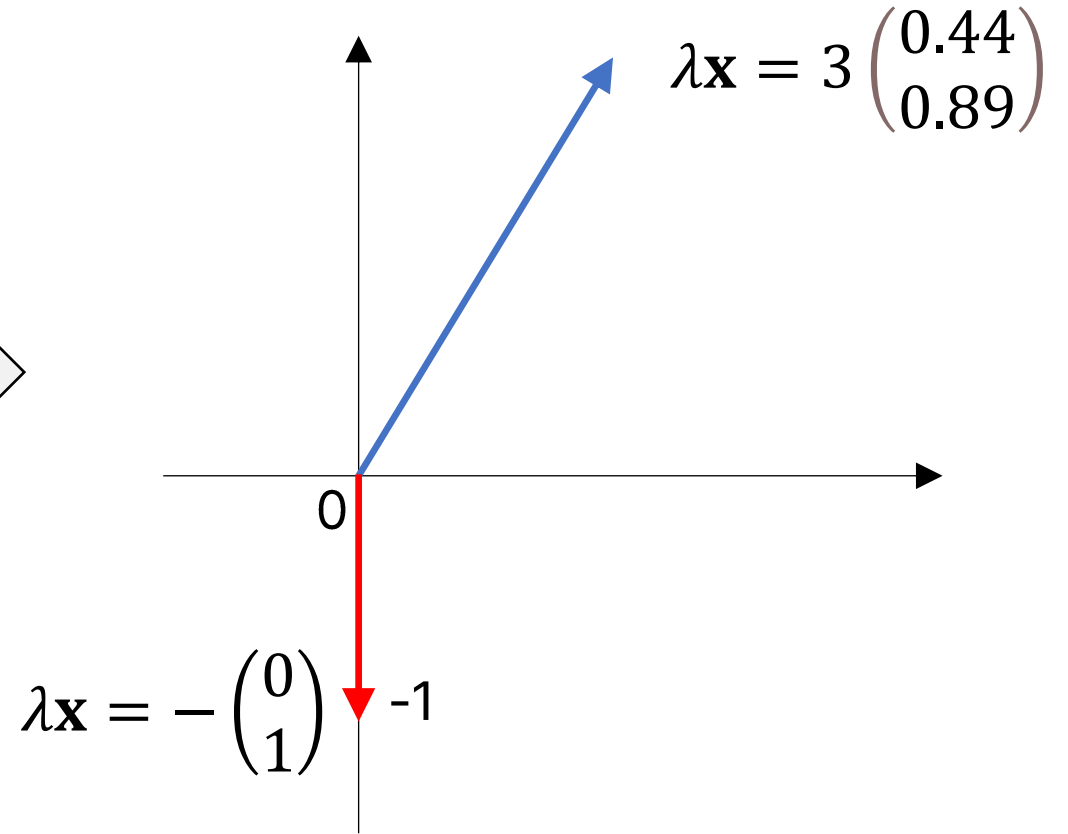
(i) $\lambda = 3$ 인 경우

$$\mathbf{x} = \begin{pmatrix} 1/\sqrt{5} \\ 2/\sqrt{5} \end{pmatrix} = \begin{pmatrix} 0.44 \\ 0.89 \end{pmatrix}$$



(ii) $\lambda = -1$ 인 경우

$$\mathbf{x} = \begin{pmatrix} 0 \\ 1 \end{pmatrix}$$



AI/BigData 인텐시브 과정

Section 1. 선형대수

Section 1-8. 대각화

대각화(diagonalization)

대각화(diagonalization): 행렬을 대각 행렬로 만드는 것

$$D = P^{-1}AP$$

가역 행렬(invertible matrix)

역행렬이 존재하는 행렬

가역 행렬 P 가 존재하면 대각화 가능

직교 대각화

$$D = P^{-1}AP$$

대칭 행렬

직교 행렬

직교 행렬 P 가 존재하면 직교 대각화 가능

고윳값 분해(eigenvalue decomposition)

고윳값 분해는 직교 대각화의 한 종류

$$\begin{pmatrix} \sigma_{11} & \sigma_{12} & \sigma_{13} \\ \sigma_{21} & \sigma_{22} & \sigma_{23} \\ \sigma_{31} & \sigma_{32} & \sigma_{33} \end{pmatrix}$$

$$= (\mathbf{u}_1 \quad \mathbf{u}_2 \quad \mathbf{u}_3) \begin{pmatrix} \lambda_1 & 0 & 0 \\ 0 & \lambda_2 & 0 \\ 0 & 0 & \lambda_3 \end{pmatrix} \begin{pmatrix} \mathbf{u}_1^T \\ \mathbf{u}_2^T \\ \mathbf{u}_3^T \end{pmatrix}$$

$$= \mathbf{P} \mathbf{D} \mathbf{P}^T$$

Section 대각화

```
In [67]: 1 cov_x = X_t.dot(X)
        2 cov_x
```

Out[67]:

	CRIM	NOX	RM	AGE	TAX
CRIM	4.397034e+04	1226.123170	1.082195e+04	1.685150e+05	1.173073e+06
NOX	1.226123e+03	162.470380	1.751519e+03	2.045220e+04	1.211706e+05
RM	1.082195e+04	1751.519414	2.023460e+04	2.156702e+05	1.280740e+06
AGE	1.685150e+05	20452.201630	2.156702e+05	2.779615e+06	1.537874e+07
TAX	1.173073e+06	121170.623200	1.280740e+06	1.537874e+07	9.867314e+07

```
In [68]: 1 e, v = np.linalg.eig(cov_x)
```

```
In [69]: 1 e
```

Out[69]: array([1.01109837e+08, 3.74919516e+05, 2.99553705e+04, 2.40312204e+03,
2.90022330e+00])

```
In [70]: 1 v
```

Out[70]: array([[1.17248568e-02, -4.34541539e-02, -9.89273583e-01,
1.38967257e-01, -3.78876853e-04],
[1.21545113e-03, 3.97729110e-03, 5.48368537e-03,
3.74537281e-02, -9.99274663e-01],
[1.28461807e-02, 4.11504926e-02, 1.37240647e-01,
9.88869506e-01, 3.79962757e-02],
[1.54544362e-01, 9.86199762e-01, -4.66395497e-02,
-3.66695593e-02, 2.48287726e-03],
[9.87832003e-01, -1.54313263e-01, 1.72471505e-02,
-8.81831838e-03, 3.51466736e-04]])

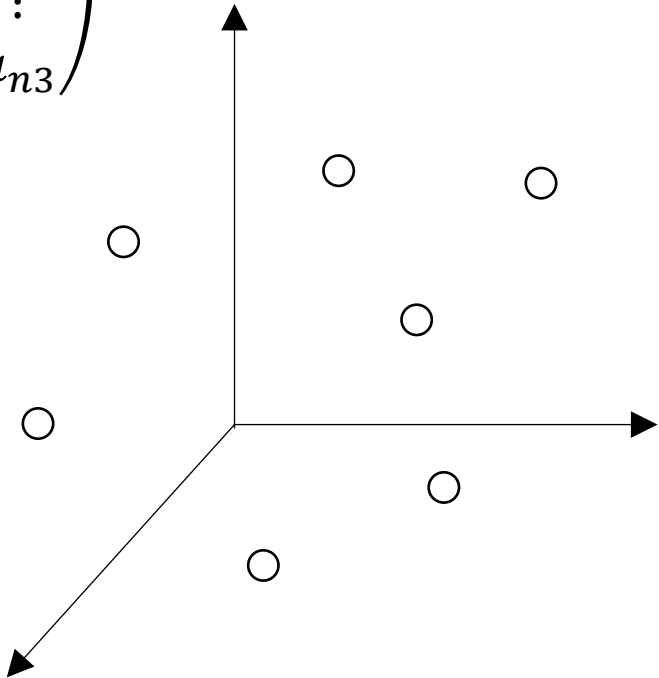
```
[58]: np.round(v, 2)
```

```
[58]: array([[ 0.01, -0.04, -0.99, 0.14, -0. ],  
[ 0. , 0. , 0.01, 0.04, -1. ],  
[ 0.01, 0.04, 0.14, 0.99, 0.04],  
[ 0.15, 0.99, -0.05, -0.04, 0. ],  
[ 0.99, -0.15, 0.02, -0.01, 0. ]])
```

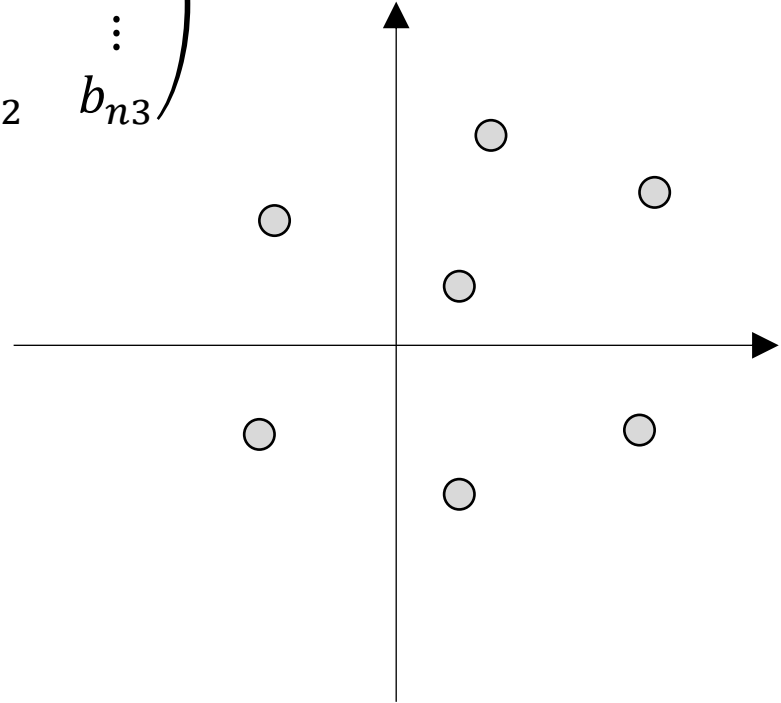
특이값 분해(singular value decomposition)

고윳값 분해를 일반화시켜 $m \times n$ 행렬에 대해서도 적용할 수 있는 행렬 분해 방법

$$\begin{pmatrix} a_{11} & a_{12} & a_{13} \\ a_{21} & a_{22} & a_{23} \\ \vdots & \vdots & \vdots \\ a_{n1} & a_{n2} & a_{n3} \end{pmatrix}$$



$$\begin{pmatrix} b_{12} & b_{13} \\ b_{22} & b_{23} \\ \vdots & \vdots \\ b_{n2} & b_{n3} \end{pmatrix}$$



특이값 분해(singular value decomposition)

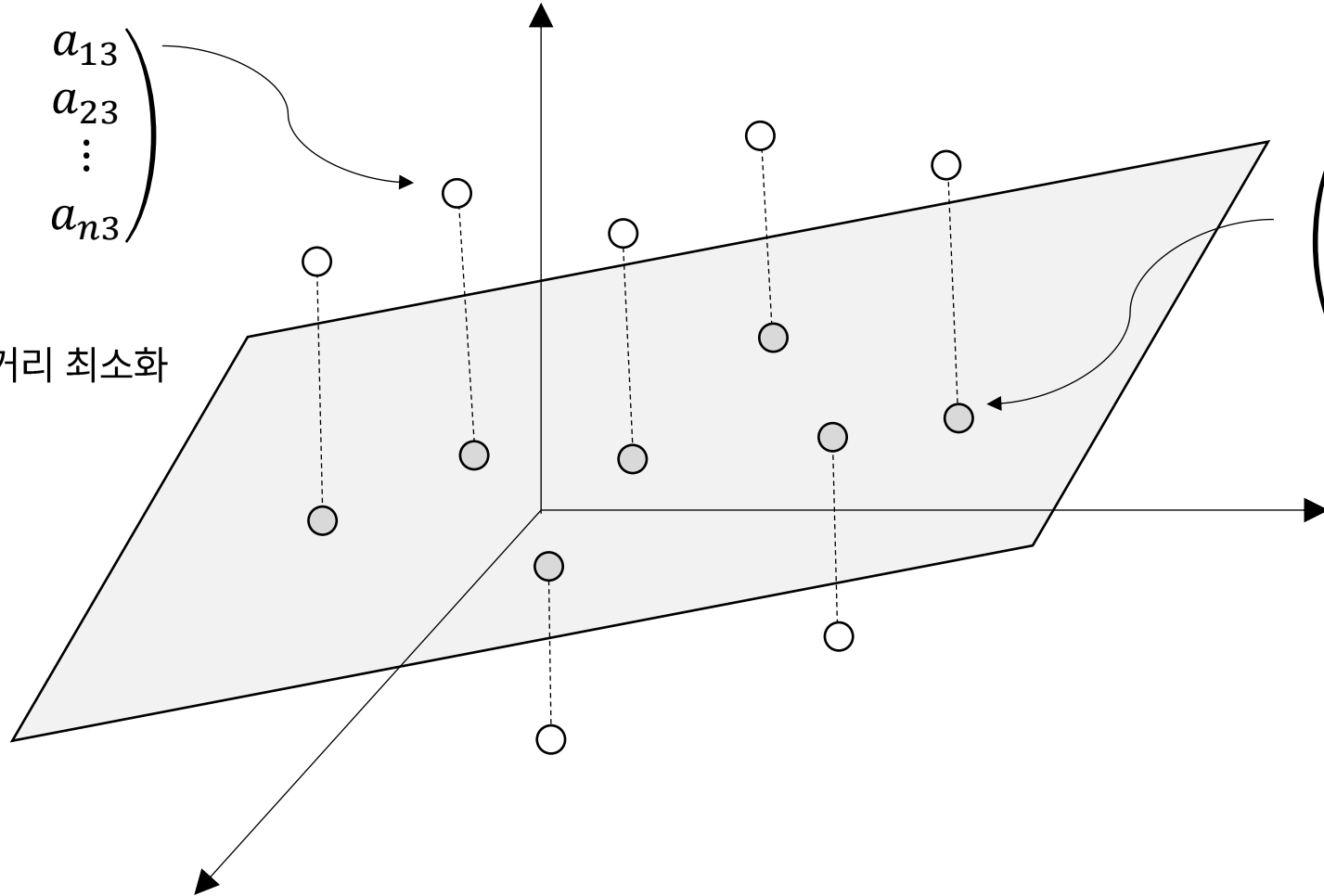
$$\begin{pmatrix} a_{11} & a_{12} & a_{13} \\ a_{21} & a_{22} & a_{23} \\ \vdots & \vdots & \vdots \\ a_{n1} & a_{n2} & a_{n3} \end{pmatrix}$$

$$\begin{pmatrix} b_{12} & b_{13} \\ b_{22} & b_{23} \\ \vdots & \vdots \\ b_{n2} & b_{n3} \end{pmatrix}$$

데이터와 부분 공간의 수직 거리 최소화

제곱합을 최소화

기존 차원 3보다 작은 차원인 2차원 부분 공간을 찾는 문제



$$\mathbf{A}^T \mathbf{A} = \begin{bmatrix} \text{row 1} \\ \text{row 2} \\ \vdots \\ \text{row } n \end{bmatrix} \begin{bmatrix} \text{col 1} & \text{col 2} & \dots & \text{col } p \end{bmatrix} \quad \begin{matrix} p \times n \\ n \times p \end{matrix} = \text{행렬 } \mathbf{A} \text{의 열벡터의 내적}$$

$$\mathbf{A} \mathbf{A}^T = \begin{bmatrix} \text{col 1} & \text{col 2} & \dots & \text{col } p \end{bmatrix} \begin{bmatrix} \text{row 1} \\ \text{row 2} \\ \vdots \\ \text{row } n \end{bmatrix} \quad \begin{matrix} n \times p \\ p \times n \end{matrix} = \text{행렬 } \mathbf{A} \text{의 행벡터의 내적}$$

특이값 분해(singular value decomposition)

$\mathbf{A}^T \mathbf{A}$ 의 고윳값: $\lambda_1, \lambda_2, \dots, \lambda_d$

특이값: $\sigma_1 = \sqrt{\lambda_1}, \sigma_2 = \sqrt{\lambda_2}, \dots, \sigma_d = \sqrt{\lambda_d}$

특이값 분해(singular value decomposition)

$$A = U\Sigma V^T$$

$$= (\mathbf{u}_1 \quad \mathbf{u}_2 \quad \cdots \quad \mathbf{u}_d \quad \mathbf{u}_{d+1} \quad \cdots \quad \mathbf{u}_n) \left(\begin{array}{cccc|cccc} \sigma_1 & 0 & \cdots & 0 & 0 & 0 & \cdots & 0 \\ 0 & \sigma_2 & \cdots & 0 & 0 & 0 & \cdots & 0 \\ \vdots & \vdots & \ddots & \vdots & \vdots & \vdots & \ddots & \vdots \\ 0 & 0 & \cdots & \sigma_d & 0 & 0 & \cdots & 0_{d \times (p-d)} \\ \hline 0 & 0 & \cdots & 0 & 0 & 0 & \cdots & 0 \\ 0 & 0 & \cdots & 0 & 0 & 0 & \cdots & 0 \\ \vdots & \vdots & \ddots & \vdots & \vdots & \vdots & \ddots & \vdots \\ 0 & 0 & \cdots & 0_{(n-d) \times d} & 0 & 0 & \cdots & 0_{(n-d) \times (p-d)} \end{array} \right) \begin{pmatrix} \mathbf{v}_1^T \\ \mathbf{v}_2^T \\ \vdots \\ \mathbf{v}_d^T \\ - \\ \mathbf{v}_{d+1}^T \\ \vdots \\ \mathbf{v}_p^T \end{pmatrix}$$

특이값 분해(singular value decomposition)

$$\begin{array}{c}
 \left[\begin{array}{c} \text{column 1} \\ \text{column 2} \\ \vdots \\ \text{column } p \end{array} \right] = \left[\begin{array}{c} \text{left singular vector} \\ \downarrow \\ \mathbf{u}_1 \\ \mathbf{u}_2 \\ \vdots \\ \mathbf{u}_d \\ \vdots \\ \mathbf{u}_{d+1} \\ \vdots \\ \mathbf{u}_n \end{array} \right] \left[\begin{array}{cc} \begin{array}{cccc} \sigma_1 & 0 & \cdots & 0 \\ 0 & \sigma_2 & \cdots & 0 \\ \vdots & \vdots & \ddots & \vdots \\ 0 & 0 & \cdots & \sigma_d \end{array} & \mathbf{0}_{d \times (p-d)} \\ \mathbf{0}_{(n-d) \times d} & \mathbf{0}_{(n-d) \times (p-d)} \end{array} \right] \left[\begin{array}{c} \text{right singular vector} \\ \downarrow \\ \mathbf{v}_1^T \\ \mathbf{v}_2^T \\ \vdots \\ \mathbf{v}_d^T \\ \mathbf{v}_{d+1}^T \\ \vdots \\ \mathbf{v}_p^T \end{array} \right] \\
 \mathbf{A}_{n \times p} & \mathbf{U}_{n \times n} & \mathbf{\Sigma}_{n \times p} & \mathbf{V}_{p \times p}^T
 \end{array}$$

$\mathbf{A}\mathbf{A}^T$ 의 고유벡터
 $\mathbf{A}^T\mathbf{A}$ 의 고유벡터

Section

대각화

```
1 e, v = np.linalg.eig(cov_x)
```

```
1 e
```

```
52]: array([1.01109837e+08, 3.74919516e+05, 2.99553705e+04, 2.40312204e+03,
          2.90022330e+00])
```

```
1 v
```

```
53]: array([[ 1.17248568e-02, -4.34541539e-02, -9.89273583e-01,
            1.38967257e-01, -3.78876853e-04],
          [ 1.21545113e-03,  3.97729110e-03,  5.48368537e-03,
            3.74537281e-02, -9.99274663e-01],
          [ 1.28461807e-02,  4.11504926e-02,  1.37240647e-01,
            9.88869506e-01,  3.79962757e-02],
          [ 1.54544362e-01,  9.86199762e-01, -4.66395497e-02,
            -3.66695593e-02,  2.48287726e-03],
          [ 9.87832003e-01, -1.54313263e-01,  1.72471505e-02,
            -8.81831838e-03,  3.51466736e-04]])
```

```
1 u, s, vt = np.linalg.svd(cov_x)
```

```
1 u
```

```
array([[ -1.17248568e-02,  4.34541539e-02,  9.89273583e-01,
         -1.38967257e-01, -3.78876853e-04],
       [ -1.21545113e-03, -3.97729110e-03, -5.48368537e-03,
         -3.74537281e-02, -9.99274663e-01],
       [ -1.28461807e-02, -4.11504926e-02, -1.37240647e-01,
         -9.88869506e-01,  3.79962757e-02],
       [ -1.54544362e-01, -9.86199762e-01,  4.66395497e-02,
          3.66695593e-02,  2.48287726e-03],
       [ -9.87832003e-01,  1.54313263e-01, -1.72471505e-02,
          8.81831838e-03,  3.51466736e-04]])
```

```
1 s
```

```
array([1.01109837e+08, 3.74919516e+05, 2.99553705e+04, 2.40312204e+03,
        2.90022330e+00])
```

```
1 vt
```

```
array([[ -1.17248568e-02, -1.21545113e-03, -1.28461807e-02,
         -1.54544362e-01, -9.87832003e-01],
       [  4.34541539e-02, -3.97729110e-03, -4.11504926e-02,
         -9.86199762e-01,  1.54313263e-01],
       [  9.89273583e-01, -5.48368537e-03, -1.37240647e-01,
          4.66395497e-02, -1.72471505e-02],
       [ -1.38967257e-01, -3.74537281e-02, -9.88869506e-01,
          3.66695593e-02,  8.81831838e-03],
       [ -3.78876853e-04, -9.99274663e-01,  3.79962757e-02,
          2.48287726e-03,  3.51466736e-04]])
```



```
1 u2, s2, vt2 = np.linalg.svd(X)
```

```
1 u2
```

```
: array([[ -0.03008946,  0.03086023,  0.01712144, ..., -0.00411115,
          -0.00427013, -0.00365704],
        [ -0.02499491,  0.06652264,  0.00780423, ..., -0.05447262,
          -0.05387641, -0.05180949],
        [ -0.02472231,  0.03790476,  0.0132068 , ..., -0.01726598,
          -0.01167245,  0.01252377],
        ...,
        [ -0.02822706,  0.07823426,  0.00788495, ...,  0.99280739,
          -0.00687219, -0.00535758],
        [ -0.02820076,  0.07548049,  0.00791964, ..., -0.00689953,
           0.99336613, -0.00536756],
        [ -0.02806907,  0.0617432 ,  0.00995976, ..., -0.00551636,
          -0.00549819,  0.99463978]])
```

```
1 s2
```

```
: array([1.00553387e+04, 6.12306718e+02, 1.73076199e+02, 4.90216487
         e+01,
         1.70300420e+00])
```

```
1 vt2
```

```
: array([[ -1.17248568e-02, -1.21545113e-03, -1.28461807e-02,
          -1.54544362e-01, -9.87832003e-01],
        [ -4.34541539e-02,  3.97729110e-03,  4.11504926e-02,
           9.86199762e-01, -1.54313263e-01],
        [ -9.89273583e-01,  5.48368537e-03,  1.37240647e-01,
          -4.66395497e-02,  1.72471505e-02],
        [ -1.38967257e-01, -3.74537281e-02, -9.88869506e-01,
           3.66695593e-02,  8.81831838e-03],
        [ -3.78876853e-04, -9.99274663e-01,  3.79962757e-02,
           2.48287726e-03,  3.51466736e-04]])
```

[63]:

```
s
```

```
[63]: array([1.01109837e+08, 3.74919516e+05, 2.99553705e+04, 2.40312204e+03,  
          2.90022330e+00])
```

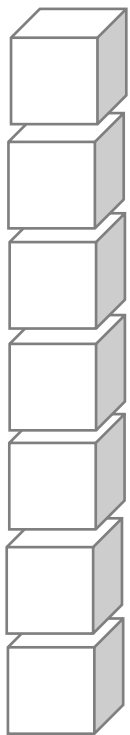
[65]:

```
s2**2
```

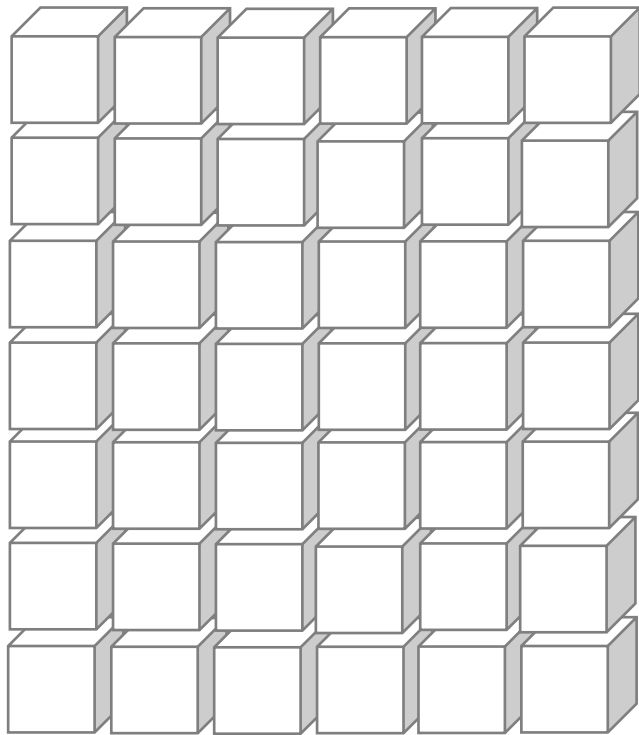
```
[65]: array([1.01109837e+08, 3.74919516e+05, 2.99553705e+04, 2.40312204e+03,  
          2.90022330e+00])
```

$\mathbf{X}^T \mathbf{X}$ 의 고유값: $\lambda_1, \lambda_2, \dots, \lambda_d$

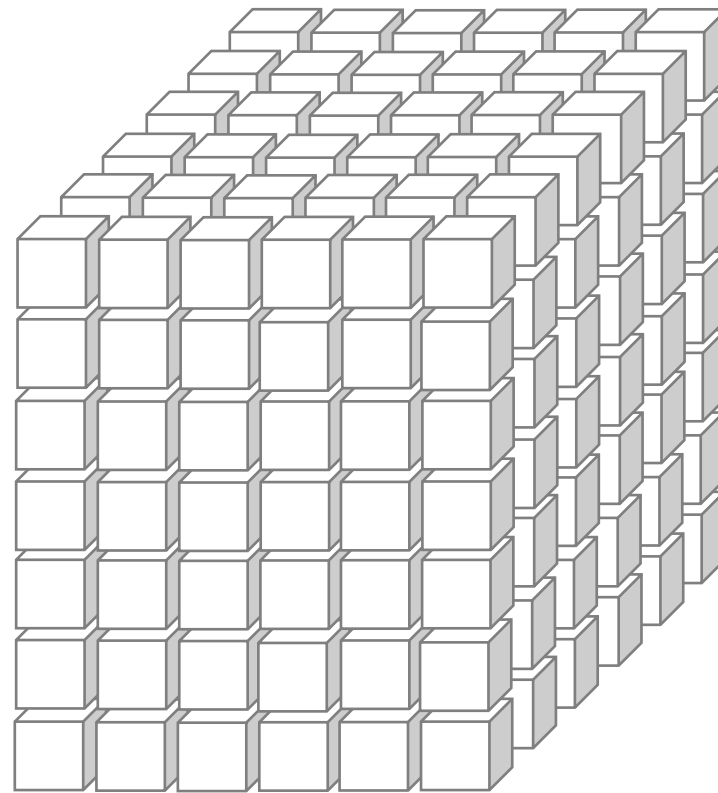
특이값: $\sigma_1 = \sqrt{\lambda_1}, \sigma_2 = \sqrt{\lambda_1}, \dots, \sigma_d = \sqrt{\lambda_d}$



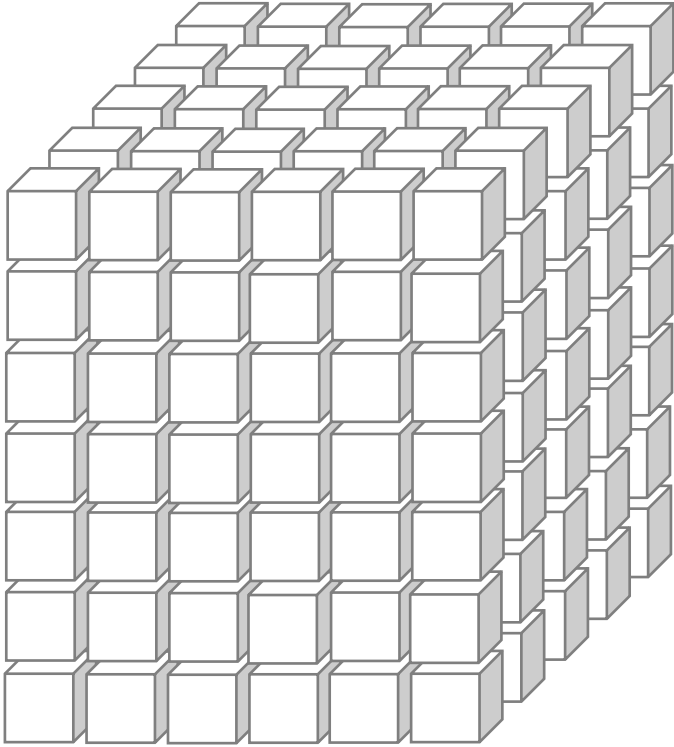
벡터
(vector)



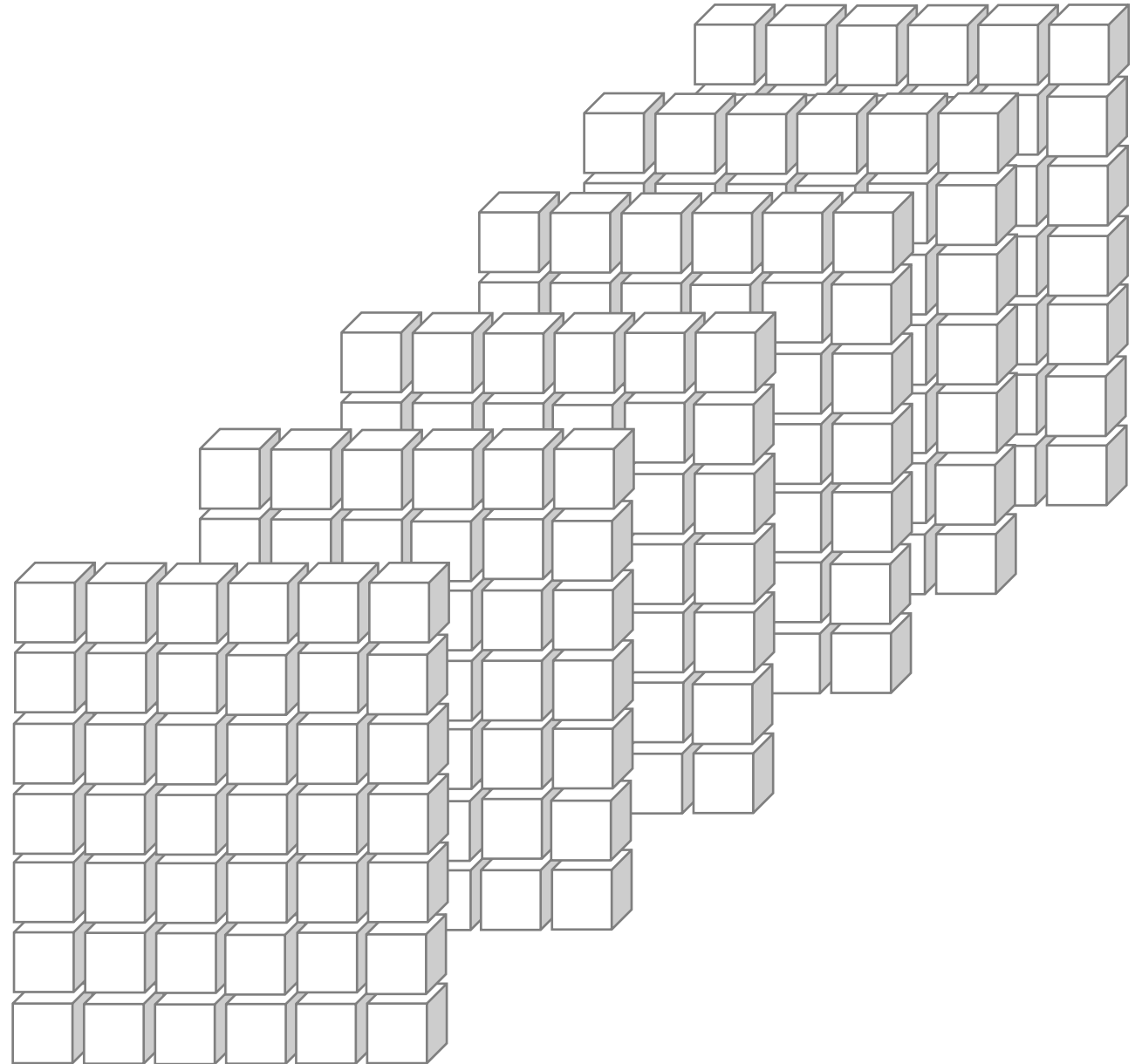
행렬
(matrix)



텐서
(tensor)



텐서
(tensor)



$$\mathbf{x} = \begin{pmatrix} x_1 \\ x_2 \\ \vdots \\ x_n \end{pmatrix}$$

$$\mathbf{X} = \begin{pmatrix} x_{11} & x_{12} & \cdots & x_{1p} \\ x_{21} & x_{22} & \cdots & x_{2p} \\ \vdots & \vdots & \ddots & \vdots \\ x_{n1} & x_{n2} & \cdots & x_{np} \end{pmatrix}$$

$$\mathcal{X} = \begin{pmatrix} x_{111} & x_{121} & \cdots & x_{1J1} \\ x_{211} & x_{221} & \cdots & x_{2J1} \\ \vdots & \vdots & \ddots & \vdots \\ x_{I11} & x_{I21} & \cdots & x_{IJ1} \end{pmatrix}, \begin{pmatrix} x_{112} & x_{122} & \cdots & x_{1J2} \\ x_{212} & x_{222} & \cdots & x_{2J2} \\ \vdots & \vdots & \ddots & \vdots \\ x_{I12} & x_{I22} & \cdots & x_{IJ2} \end{pmatrix}, \dots, \begin{pmatrix} x_{11K} & x_{12K} & \cdots & x_{1JK} \\ x_{21K} & x_{22K} & \cdots & x_{2JK} \\ \vdots & \vdots & \ddots & \vdots \\ x_{I1K} & x_{I2K} & \cdots & x_{IJK} \end{pmatrix}$$

감사합니다.

Q & A